FOUNDATIONS & BIG-O ANALYSIS

FAANG Interview Orientation — Mentor Guide

🎬 THE REAL-WORLD STORY

Imagine transferring a 100 GB file across town. Wi-Fi scales linearly with file size (100 GB = 200 min) \rightarrow **O(N)**. Driving a USB drive takes fixed time regardless of size (100 GB = 30 min) \rightarrow **O(1)**.

💡 THE BIG-O AHA MOMENT

Big-O notation measures how runtime or memory scales as input size N grows to infinity, ignoring hardware speed differences!

🧐 WHY NOT MEASURE SECONDS?

- A fast CPU runs bad code quickly on tiny inputs.
- OS background processes alter millisecond execution timings.
- Big-O gives a universal mathematical upper bound guaranteed on all FAANG evaluation servers!

📈 BIG-O GROWTH HIERARCHY

Big-O	Name	Max N for 1 Sec (10^8 ops)
O(1)	Constant	Unlimited
O(log N)	Logarithmic	10^18 (Binary Search)
O(N)	Linear	10^8 (Single Pass)
O(N log N)	Linearithmic	10^6 (Sorting)
O(N^2)	Quadratic	10^4 (Nested Loops)
O(2^N) / O(N!)	Exponential	20 / 11 (Recursion)

📐 THE 3 BIG-O REDUCTION RULES

Rule 1 — Drop Constants: O(2N + 5) \rightarrow **O(N)**. We care about growth rate, not constant multipliers.

Rule 2 — Drop Non-Dominant Terms: O(N^2 + 100N + 500) \rightarrow **O(N^2)**. N^2 dominates N as N grows.

Rule 3 — Multi-Variable Rules: Array A size N, Array B size M \rightarrow **O(N + M)** for sequential loops, **O(N · M)** for nested loops!

⚡ 5-STEP FAANG PROBLEM SOLVING FRAMEWORK

1. **Clarify Constraints:** Ask N limits (N <= 10^5 \implies O(N log N) target).
2. **State Brute Force:** Mention naive solution & complexity first.
3. **Identify Bottleneck:** Spot repeated work \rightarrow HashMap/Binary Search.
4. **Optimize & Code:** Write clean, modular Java standard code.
5. **Dry Run Test Cases:** Step line-by-line with small/edge inputs!

📊 DEDUCING TARGET COMPLEXITY DIRECTLY FROM INPUT CONSTRAINTS (N)

Input Size (N)	Target Time Complexity	Typical Algorithm Patterns
N <= 10..12	O(N!) or O(2^N · N)	Permutations, N-Queens, Backtracking Bitmask
N <= 20..25	O(2^N)	Subsets, Combination Sum, Meet-in-the-Middle
N <= 500	O(N^3)	Floyd-Warshall, Matrix Chain Multiplication DP
N <= 2,000	O(N^2)	Nested Loops, 2D Grid DP, All-Pairs Check
N <= 10^5..10^6	O(N log N) or O(N)	Sorting, Binary Search, Two Pointers, HashMap, Sliding Window
N <= 10^9..10^18	O(log N) or O(1)	Binary Search on Answer, Bit Manipulation, Math Formulas

JAVA TOOLKIT & FAANG DRY RUN PROTOCOL

Auxiliary Memory, Pitfalls & Verification Protocol

💡 AUXILIARY SPACE VS CALL STACK

- **Input Space:** Memory used to store input data (e.g. array of size N).
- **Auxiliary Space:** Extra memory created by your code (DS + call stack).
- **Recursion Call Stack:** `factorial(3)` calls `factorial(2)` \rightarrow Max stack depth 3 \implies **O(N) Auxiliary Space!**

🛠 ESSENTIAL JAVA DATA STRUCTURE COMPLEXITIES

Data Structure	Operation	Time Complexity
`ArrayList`	`add()`, `get(i)`, `remove(last)`	O(1) amortized
`HashMap`	`put()`, `get()`, `containsKey()`	O(1) average, O(N) worst
`HashSet`	`add()`, `contains()`, `remove()`	O(1) average
`ArrayDeque`	`push()`, `pop()`, `offer()`, `poll()`	O(1) strictly faster than Stack
`PriorityQueue`	`offer()`, `poll()`, `peek()`	O(log N) offer/poll, O(1) peek

⚡ JAVA INTERVIEW POWER SNIPPETS

```
// 1. Frequency Map Shortcut:
map.put(key, map.getOrDefault(key, 0) + 1);

// 2. Sort Array in Reverse Order:
Arrays.sort(arr, (a, b) → Integer.compare(b,
a));

// 3. Min Heap & Max Heap Declarations:
PriorityQueue<Integer> minHeap = new PriorityQueu
e<>();
PriorityQueue<Integer> maxHeap = new PriorityQueu
e<>((a, b) → b - a);

// 4. String Array to Char Manipulation:
char[] chars = str.toCharArray();
String sortedStr = new String(chars);
```

💼 COMMON JAVA INTERVIEW BUGS & FIXES

1. Binary Search Mid Overflow:
`int mid = (left + right) / 2;` \rightarrow ❌ Overflows if `left + right > 2^31 - 1`!
`int mid = left + (right - left) / 2;` \rightarrow ✅ Safe formula!

2. String Immutability Loop Overhead:
`String s += "a";` in loop \rightarrow ❌ Recreates string in O(N^2) total time!
`StringBuilder sb = new StringBuilder(); sb.append('a');` \rightarrow ✅ O(1) amortized!

🔍 THE 4-STEP FAANG DRY RUN METHOD

1. **Pick a Small Concrete Input:** e.g. `nums = [2, 7, 11]`, `target = 9`.
2. **Create a Variable State Table:** Track loop counters, pointers, & variables.
3. **Step Line-by-Line:** Execute code manually without skipping steps!
4. **Test Edge Cases:** Check empty inputs, 1-element arrays, & duplicates.

📊 SAMPLE VARIABLE STATE TRACE TABLE

Line	i	nums[i]	complement	map state	Action
1	0	2	7	{}	`map.put(2, 0)`
2	1	7	2	{2: 0}	Match Found! Return `[0, 1]`

✅ ORIENTATION MASTERY CHECKLIST

☐ Explain Big-O using real-world analogies (Wi-Fi vs Car)
☐ Deduce target complexity directly from input size N
☐ State 3 Big-O reduction rules without hesitation
☐ Use safe mid formula `left + (right - left) / 2`
☐ Differentiate Input Space vs Auxiliary Call Stack Space

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — State Brute Force First
Always state the obvious brute force solution and its time/space complexity before jumping into optimization! It shows structured thinking.

☐ Perform 4-step line-by-line variable trace dry runs

Rule 2 — Dry Run Saves You from Rejection

90% of interview bugs (off-by-one errors, null pointers, unhandled edge cases) are caught during your dry run before running tests!

ARRAYS, STRINGS & HASHING

FAANG Master Guide — Java Edition

🎬 THE REAL-WORLD STORY

Searching a contact in a phonebook with 1,000,000 entries page by page requires 1,000,000 checks in the worst case!

💡 THE HASHMAP AHA MOMENT

What if every contact name instantly transformed into an exact page index via a mathematical hash function? Lookup time drops from **O(N) to O(1)**!

🤔 WHY DO ARRAYS ALONE FAIL?

- **Array Index Access:** Fast O(1) if index `arr[i]` is known.
- **Array Search by Value:** Slow O(N) linear scan required.
- **Array Insertion at Start:** Slow O(N) shift of all elements!
- **HashMap Solution:** Sacrifices memory to map `Value → Index` for instantaneous O(1) lookups!

🛠 HASH FUNCTION MECHANICS

Key → Hash Code → Bucket Index:

```
Key: "alice"
  ↓ Hash Function: hash("alic
e") = 153029
Bucket Index = 153029 % array_
size (16) = 5
  ↓ Direct Array Access!
Bucket[5] stores ("alice", pho
neNumber)
O(1) Instant Lookup!
```

📚 PREREQUISITES & COMPLEXITY REASONING

Operation / Structure	Time Complexity	Auxiliary Space
Array Index Access (`arr[i]`)	**O(1)**	**O(N) contiguous memory**
Linear Search / Scan	**O(N)**	**O(1) space**
HashMap Lookup / Put	**O(1) average**	**O(N) space**
Prefix Sum Array Query	**O(1) per range query**	**O(N) prefix array**

🔥 CLUES THAT SCREAM HASHING

1. *"Find a pair adding to Target"* $\rightarrow$ HashMap storing `(Target - num) → Index`
2. *"Check for duplicates or count frequencies"* $\rightarrow$ HashSet or Frequency Array `int[26]`
3. *"Group anagrams together"* $\rightarrow$ HashMap storing `SortedString → List`

PATTERN 1 COMPLEMENT HASHING (Two Sum — LC 1)

WHEN TO USE

Finding two numbers in unsorted array adding to target in single pass O(N) time.

TEMPLATE

```
public int[] twoSum(int[] nums, int target) {
  Map<Integer, Integer> map = new HashMap<>();
  for (int i = 0; i < nums.length; i++) {
    if (map.containsKey(target - nums[i])) return new int[]{map.g
et(target - nums[i]), i};
    map.put(nums[i], i);
  }
  return new int[0];
}
```

PATTERN 2 HASHSET DUPLICATE (LC 217)

```
public boolean containsDuplicate(int[] nums) {
  Set<Integer> set = new HashSet<>();
  for (int n : nums) if (!set.add(n)) return true;
  return false;
}
```

PATTERN 4 GROUP ANAGRAMS (LC 49)

SORTED STRING KEY HASHING

```
public List<List<String>> groupAnagrams(String[] strs) {
  Map<String, List<String>> map = new HashMap<>();
  for (String s : strs) {
    char[] ca = s.toCharArray(); Arrays.sort(ca);
    String key = String.valueOf(ca);
    map.computeIfAbsent(key, k → new ArrayList<>()).add(s);
  }
  return new ArrayList<>(map.values());
}
```

PATTERN 5 TOP K FREQUENT BUCKET SORT (LC 347)

FREQUENCY ARRAY BUCKETS O(N)

```
public int[] topKFrequent(int[] nums, int k) {
  Map<Integer, Integer> cnt = new HashMap<>();
  for (int n : nums) cnt.put(n, cnt.getOrDefault(n, 0) + 1);
  List<Integer>[] b = new List[nums.length + 1];
  for (int key : cnt.keySet()) {
    int f = cnt.get(key); if (b[f] == null) b[f] = new ArrayList<>(); b[f].add(key);
  }
  int[] res = new int[k]; int idx = 0;
  for (int i = b.length - 1; i >= 0 && idx < k; i--)
    if (b[i] != null) for (int n : b[i]) res[idx++] = n;
  return res;
}
```

📊 COMPLETE ARRAYS, STRINGS & HASHING PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
1	Two Sum	Complement Hashing	O(N)	O(N)	E
217	Contains Duplicate	HashSet Add Check	O(N)	O(N)	E
242	Valid Anagram	Char Frequency Array	O(N)	O(1)	E
49	Group Anagrams	Sorted Key Map	O(N · K log K)	O(N · K)	M
347	Top K Frequent Elements	Bucket Sort Array	O(N)	O(N)	M
238	Product of Array Except Self	Prefix & Suffix Accumulation	O(N)	O(1)	M
128	Longest Consecutive Sequence	HashSet Sequence Start Check	O(N)	O(N)	M

🔍 DRY RUN — Longest Consecutive Sequence ([100, 4, 200, 1, 3, 2])

Step	num	set.contains(num - 1)?	Action	currLen	max
1	100	No (99 missing)	Sequence start! Count 100	1	1
2	4	Yes (3 exists)	Skip (not start of sequence)	-	1
3	200	No (199 missing)	Sequence start! Count 200	1	1
4	1	No (0 missing)	Sequence start! Count 1 → 2 → 3 → 4	4	4 ✅

📐 MATHEMATICAL PROOF — O(N) COMPLEXITY OF LONGEST CONSECUTIVE

Claim: Checking `!set.contains(num - 1)` guarantees total O(N) runtime.

Proof: Inner while loop executes ONLY for the starting element of each sequence. Numbers in the middle of a sequence are skipped in O(1) time. Thus, each array element is visited at most twice $\implies \mathbf{O(N)\text{ Total Operations}!}$

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Code Two Sum in 2m using Complement HashMap
- ☐ Check duplicates in O(N) using HashSet `add()`
- ☐ Implement Valid Anagram using `int[26]` array
- ☐ Group Anagrams using sorted char array key
- ☐ Code Top K Frequent in O(N) via Bucket Sort
- ☐ Write Product of Array Except Self without division
- ☐ Code Longest Consecutive Sequence in O(N) time
- ☐ Explain why HashMap lookup is O(1) average

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Frequency Arrays Outperform HashMaps for Small Alphabets

For lowercase string problems (`'a'-'z'`), ALWAYS use `int[26]` arrays instead of `HashMap`. Arrays have zero object allocation overhead and 10x faster CPU cache locality!

Rule 2 — Complement Hashing Eliminates Double Loops

Instead of searching for pair (A, B) using nested loops, store $B = \text{Target} - A$ in a HashMap to collapse $O(N^2)$ to $O(N)$!

TWO POINTERS PATTERN

FAANG Master Guide — Java Edition

🎬 THE REAL-WORLD STORY

Imagine two people standing at opposite ends of a long hallway looking for a pair of numbers that add up to a target sum.

💡 THE TWO POINTERS AHA MOMENT

If the array is sorted, comparing `arr[left] + arr[right]` tells us EXACTLY which pointer to move! If sum is too small → `left++`. If sum is too large → `right--`. Replaces O(N^2) loops with **O(N) single pass**!

🤔 WHY NOT NESTED LOOPS?

- **Nested Loops O(N^2):** Checks every possible pair `(i, j)`. For N = 100,000, requires 10,000,000,000 operations → TLE Error!
- **Two Pointers O(N):** Moves inward deterministically. Total steps <= N. Executes in 5 milliseconds!
- **Auxiliary Space O(1):** Reuses array pointers with 0 extra memory allocation.

📌 2 MAIN POINTER VARIATIONS

1. Opposite Direction (Converging):

```
left → [ 1,  2,  4,  6,  8, 1
1 ] ← right
Sum = 1 + 11 = 12 > Target 10
→ right--
Sum = 1 + 8  = 9  < Target 10
→ left++
```

2. Same Direction (Fast & Slow / Partition):

```
[ slow, fast → ] (In-place ar
ray modification)
```

🧩 PREREQUISITES & COMPLEXITY REASONING

Problem Variation	Time Complexity	Auxiliary Space
Two Sum II (Sorted Array)	**O(N) single pass**	**O(1) in-place**
3Sum (Sort + 2-Pointer Loop)	**O(N^2)**	**O(1) space**
Container With Most Water	**O(N) single pass**	**O(1) space**
Trapping Rain Water (2 Pointers)	**O(N) single pass**	**O(1) space**

⚡ CLUES THAT SCREAM TWO POINTERS

1. *"Array is SORTED"* and asks for pair sum, triplet sum, or range search → Opposite Two Pointers
2. *"Remove duplicates / zeros in-place in O(1) space"* → Fast & Slow Pointers
3. *"Container with water / Trapping water at boundaries"* → Converging Boundary Pointers

PATTERN 1 VALID PALINDROME (Converging Alphanumeric Scan) e.g. Valid Palindrome (LC 125)

WHEN TO USE

Check symmetry from ends inward, skipping non-alphanumeric characters in O(N) time and O(1) space.

TEMPLATE — LC 125

```
public boolean isPalindrome(String s) {
  int l = 0, r = s.length() - 1;
  while (l < r) {
    while (l < r && !Character.isLetterOrDigit(s.charAt(l)))
l++;
    while (l < r && !Character.isLetterOrDigit(s.charAt(r)))
r--;
    if (Character.toLowerCase(s.charAt(l)) ≠ Character.toLow
erCase(s.charAt(r))) {
      return false;
    }
    l++; r--;
  }
  return true;
}
```

TWO POINTERS PATTERN

Core Patterns — Part 2

PATTERN 2 **TWO SUM II (Sorted Input Array)** e.g. Two Sum II - Input Array Is Sorted (LC 167)

WHEN TO USE

Array is ALREADY sorted. Converge left/right pointers to find target in O(N) time and O(1) space without HashMap!

TEMPLATE — LC 167

```
public int[] twoSum(int[] numbers, int target) {
  int l = 0, r = numbers.length - 1;
  while (l < r) {
    int sum = numbers[l] + numbers[r];
    if (sum == target) return new int[]{l + 1, r + 1};
    if (sum < target) l++; else r--;
  }
  return new int[0];
}
```

PATTERN 4

CONTAINER WITH MOST WATER (Greedy Boundary Shrink)

e.g. Container With Most Water (LC 11)

GREEDY CHOICE PROOF

`Area = min(height[l], height[r]) * (r - l)`. The limiting factor is the shorter wall! Moving the taller wall CANNOT increase area because width shrinks and height is capped by shorter wall. Thus, ALWAYS move the shorter wall pointer!

TEMPLATE — LC 11

```
public int maxArea(int[] height) {
  int l = 0, r = height.length - 1;
  int maxWater = 0;
  while (l < r) {
    int h = Math.min(height[l], height[r]);
    maxWater = Math.max(maxWater, h * (r - l));
    if (height[l] < height[r]) {
      l++; // Move shorter wall!
    } else {
      r--; // Move shorter wall!
    }
  }
  return maxWater;
}
```

PATTERN 5 TRAPPING RAIN WATER (2-Pointer Maximum Tracking) e.g. Trapping Rain Water (LC 42)

WATER TRAP MECHANICS

Maintain `leftMax` and `rightMax`. If `leftMax < rightMax`, trapped water at `left` is determined solely by `leftMax - height[left]`! Runs in O(N) time and O(1) space.

TEMPLATE — LC 42

```
public int trap(int[] height) {
  int l = 0, r = height.length - 1;
  int leftMax = 0, rightMax = 0, water = 0;
  while (l < r) {
    if (height[l] < height[r]) {
      if (height[l] ≥ leftMax) leftMax = height[l];
      else water += leftMax - height[l];
      l++;
    } else {
      if (height[r] ≥ rightMax) rightMax = height[r];
      else water += rightMax - height[r];
      r--;
    }
  }
  return water;
}
```

📊 COMPLETE TWO POINTERS PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
125	Valid Palindrome	Converging Alphanumeric Scan	O(N)	O(1)	E
167	Two Sum II	Sorted Converging Pointers	O(N)	O(1)	E
26	Remove Duplicates	Fast & Slow Pointers	O(N)	O(1)	E
15	3Sum	Sort + Outer Loop + 2-Pointer	O(N^2)	O(1)	M
11	Container With Most Water	Shorter Wall Converge	O(N)	O(1)	M
16	3Sum Closest	Sort + Min Difference Check	O(N^2)	O(1)	M
42	Trapping Rain Water	2-Pointer Boundary Max Tracking	O(N)	O(1)	H

LC 125 · Valid Palindrome

EASY

Pattern: Converging Pointers
Key Insight: Skip non-alphanumeric chars.

```
public boolean isPalindrome(Str
ing s) {
  int l = 0, r = s.length() -
1;
  while (l < r) {
    while (l < r && !Character.
isLetterOrDigit(s.charAt(l))) l
++;
    while (l < r && !Character.
isLetterOrDigit(s.charAt(r))) r
--;
    if (Character.toLowerCase
(s.charAt(l)) ≠ Character.toLo
werCase(s.charAt(r))) return fa
lse;
    l++; r--;
  }
  return true;
}
```

LC 167 · Two Sum II

EASY

Pattern: Sorted Converging
Key Insight: `l++` if sum small, `r--` if large.

```
public int[] twoSum(int[] numbe
rs, int target) {
  int l = 0, r = numbers.length
- 1;
  while (l < r) {
    int sum = numbers[l] + numb
ers[r];
    if (sum == target) return n
ew int[]{l + 1, r + 1};
    if (sum < target) l++; else
r--;
  }
  return new int[0];
}
```

LC 26 · Remove Duplicates

EASY

Pattern: Fast & Slow Pointers
Key Insight: Overwrite `nums[++slow] = nums[fast]`.

```
public int removeDuplicates(int
[] nums) {
  if (nums.length == 0) return
0;
  int slow = 0;
  for (int fast = 1; fast < num
s.length; fast++) {
    if (nums[fast] ≠ nums[slo
w]) { slow++; nums[slow] = nums
[fast]; }
  }
  return slow + 1;
}
```

LC 15 · 3Sum — MEDIUM

Pattern: Sort + 2 Pointers
Key Insight: Fix `i`, converge `l` & `r`, skip dups.

```
public List<List<Integer>> threeSum(int[] nums) {
  Arrays.sort(nums); List<List<Integer>> res = new ArrayList<>();
  for (int i = 0; i < nums.length - 2; i++) {
    if (i > 0 && nums[i] == nums[i - 1]) continue;
    int l = i + 1, r = nums.length - 1;
    while (l < r) {
      int sum = nums[i] + nums[l] + nums[r];
      if (sum == 0) {
        res.add(Arrays.asList(nums[i], nums[l], nums[r]));
        while (l < r && nums[l] == nums[l + 1]) l++;
        while (l < r && nums[r] == nums[r - 1]) r--;
        l++; r--;
      } else if (sum < 0) l++; else r--;
    }
  }
  return res;
}
```

LC 11 · Container With Most Water — MEDIUM

Pattern: Shorter Wall Converge
Key Insight: Always move pointer of shorter wall.

```
public int maxArea(int[] height)
{
  int l = 0, r = height.length - 1, maxW = 0;
  while (l < r) {
    int h = Math.min(height[l], height[r]);
    maxW = Math.max(maxW, h * (r - l));
    if (height[l] < height[r]) l++; else r--;
  }
  return maxW;
}
```

LC 42 · Trapping Rain Water — HARD

Pattern: Boundary Max Tracking
Key Insight: Trapped water bounded by `leftMax` & `rightMax`.

```
public int trap(int[] height) {
  int l = 0, r = height.length - 1;
  int lMax = 0, rMax = 0, water = 0;
  while (l < r) {
    if (height[l] < height[r]) {
      if (height[l] ⩾ lMax) lMax = height[l];
      else water += lMax - height[l];
      l++;
    } else {
      if (height[r] ⩾ rMax) rMax = height[r];
      else water += rMax - height[r];
      r--;
    }
  }
  return water;
}
```

LC 16 · 3Sum Closest — MEDIUM

Pattern: Sort + Min Diff Converge
Key Insight: Track minimum absolute difference.

```
public int threeSumClosest(int[] nums, int target) {
  Arrays.sort(nums);
  int closest = nums[0] + nums[1] + nums[2];
  for (int i = 0; i < nums.length - 2; i++) {
    int l = i + 1, r = nums.length - 1;
    while (l < r) {
      int sum = nums[i] + nums[l] + nums[r];
      if (Math.abs(target - sum) < Math.abs(target - closest)) closest = sum;
      if (sum < target) l++; else r--;
    }
  }
  return closest;
}
```

🔍 DRY RUN — Container With Most Water ([1, 8, 6, 2, 5, 4, 8, 3, 7])

Step	l (val)	r (val)	width (r - l)	h = min(h[l], h[r])	area	maxArea	Actio
1	0 (1)	8 (7)	8	min(1, 7) = 1	8	8	`l++` (1 < 7
2	1 (8)	8 (7)	7	min(8, 7) = 7	49	49	`r--` (< 8)
3	1 (8)	7 (3)	6	min(8, 3) = 3	18	49	`r--` (< 8)
4	1 (8)	6 (8)	5	min(8, 8) = 8	40	49 ✅	`r--`

📐 MATHEMATICAL PROOF — GREEDY CHOICE OF SHORTER WALL

Claim: In Container With Most Water, moving the taller wall pointer cannot produce a larger area.

Proof: Area is defined as `min(h[l], h[r]) * width`. If `h[l] < h[r]`, max height is capped at `h[l]`. Moving `r` inward decreases `width` while height remains at most `h[l]`. Thus, area is strictly smaller! Moving `l` is the ONLY way to potentially find a taller wall → **Exact Proof of Correctness!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Code Valid Palindrome with alphanumeric skip
- ☐ Code Two Sum II on sorted array in O(N)
- ☐ Code Remove Duplicates in O(1) space
- ☐ Implement 3Sum with triply nested duplicate skip
- ☐ Code Container With Most Water in 3 minutes
- ☐ Implement Trapping Rain Water with 2-pointers
- ☐ Explain greedy choice proof for water container
- ☐ Differentiate Converging vs Fast/Slow pointers

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Always Sort First for Sum Problems

Before using Two Pointers for Two Sum / 3Sum / 4Sum, ensure the array is sorted (`Arrays.sort(nums)`). The entire directional choice relies on sorted order!

Rule 2 — Guard Against Out-of-Bounds in Duplicate Skips

When skipping duplicates inside `while` loops (e.g. in 3Sum), ALWAYS include `while (l < r && nums[l] == nums[l + 1]) l++` to prevent index out of bounds errors!

SLIDING WINDOW PATTERN

FAANG Master Guide — Java Edition

🎬 THE REAL-WORLD STORY

Imagine looking through a moving train window at a scenic mountain landscape. As the train moves forward 1 step, 1 new tree enters the right side of the window, and 1 old tree exits the left side!

💡 THE SLIDING WINDOW AHA MOMENT

Instead of recomputing the sum/characters of an entire subarray from scratch O(K), we update the running window state in **O(1) time**: `windowSum += incoming - outgoing`!

🧐 WHY RECOMPUTING SUBARRAYS FAILS

- **Brute Force O(N · K):** For N = 100,000 and K = 50,000, requires 5,000,000,000 additions -> TLE Error!
- **Sliding Window O(N):** Each element enters the window once and leaves once. Exactly 2N operations total!
- **Auxiliary Space O(1) or O(K):** Reuses frequency map or pointers with minimal extra memory.

📌 2 MAIN WINDOW TYPES

1. Fixed-Size Window (Size K):

```
[ left --- right ] (Window size fixed at K)
Slide 1 step: add right element, remove left element.
```

2. Dynamic Shrinking Window:

```
Expand right pointer to satisfy condition.
Shrink left pointer while condition is violated!
```

📚 PREREQUISITES & COMPLEXITY REASONING

Window Variation	Time Complexity	Auxiliary Space
Fixed Window Sum / Max	**O(N) single pass**	**O(1) space**
Longest Substring Without Repeating	**O(N) single pass**	**O(min(N, M)) hash set**
Minimum Window Substring	**O(N) single pass**	**O(1) char freq array**
Sliding Window Maximum (Deque)	**O(N) single pass**	**O(K) monotonic deque**

⚡ CLUES THAT SCREAM SLIDING WINDOW

1. *"Find max/min sum of contiguous subarray of fixed size K"* -> Fixed-Size Window
2. *"Find longest/shortest contiguous subarray satisfying condition C"* -> Dynamic Shrinking Window
3. *"Substring containing all characters of pattern T"* -> Dynamic Window + Frequency HashMap

PATTERN 1

NO-REPEAT SUBSTRING (Dynamic Unique Character Window)

e.g. Longest Substring Without Repeating Characters (LC 3)

WHEN TO USE

Find longest substring without any duplicate characters in O(N) single pass and O(min(N, M)) space.

TEMPLATE — LC 3

```
public int lengthOfLongestSubstring(String s) {
  int[] lastIdx = new int[128];
  Arrays.fill(lastIdx, -1);
  int left = 0, maxLen = 0;
  for (int right = 0; right < s.length(); right++) {
    char c = s.charAt(right);
    if (lastIdx[c] >= left) {
      left = lastIdx[c] + 1; // Jump left pointer directly past du
plicate!
    }
    lastIdx[c] = right;
    maxLen = Math.max(maxLen, right - left + 1);
  }
  return maxLen;
}
```

PATTERN 2 K-FLIP ZERO WINDOW (Max Consecutive Ones III) e.g. Max Consecutive Ones III (LC 1004)

WHEN TO USE

Find longest subsegment of 1s allowing at most K zero flips in O(N) time and O(1) space.

TEMPLATE — LC 1004

```
public int longestOnes(int[] nums, int k) {
  int left = 0, zeroCount = 0, maxLen = 0;
  for (int right = 0; right < nums.length; right++) {
    if (nums[right] == 0) zeroCount++;
    while (zeroCount > k) {
      if (nums[left] == 0) zeroCount--;
      left++;
    }
    maxLen = Math.max(maxLen, right - left + 1);
  }
  return maxLen;
}
```

PATTERN 4 PERMUTATION IN STRING (Fixed-Size Frequency Matcher) e.g. Permutation in String (LC 567)

MATCHES COUNTER O(26)

Fixed window of size `s1.length()`. Maintain `matches` count out of 26 letters. Update `matches` in O(1) time on each slide!

TEMPLATE — LC 567

```
public boolean checkInclusion(String s1, String s2) {
  if (s1.length() > s2.length()) return false;
  int[] c1 = new int[26], c2 = new int[26];
  for (int i = 0; i < s1.length(); i++) {
    c1[s1.charAt(i) - 'a']++; c2[s2.charAt(i) - 'a']++;
  }
  int matches = 0;
  for (int i = 0; i < 26; i++) if (c1[i] == c2[i]) matches++;
  for (int l = 0, r = s1.length(); r < s2.length(); l++, r++) {
    if (matches == 26) return true;
    int addIdx = s2.charAt(r) - 'a', delIdx = s2.charAt(l) - 'a';
    c2[addIdx]++;
    if (c1[addIdx] == c2[addIdx]) matches++;
    else if (c1[addIdx] + 1 == c2[addIdx]) matches--;
    c2[delIdx]--;
    if (c1[delIdx] == c2[delIdx]) matches++;
    else if (c1[delIdx] - 1 == c2[delIdx]) matches--;
  }
  return matches == 26;
}
```

PATTERN 5

SLIDING WINDOW MAXIMUM (Monotonic Decreasing Deque)

e.g. Sliding Window Maximum (LC 239)

MONOTONIC DEQUE STRATEGY

Deque stores indices in decreasing order of value. Remove smaller elements from tail before adding new index. Front of deque is ALWAYS the max element in current window!

TEMPLATE — LC 239

```
public int[] maxSlidingWindow(int[] nums, int k) {
  int n = nums.length;
  int[] res = new int[n - k + 1];
  Deque<Integer> deque = new ArrayDeque<>();
  for (int i = 0; i < n; i++) {
    // 1. Remove indices outside current window boundaries:
    if (!deque.isEmpty() && deque.peekFirst() < i - k + 1) deque.pollFi
rst();
    // 2. Maintain monotonic decreasing order in deque:
    while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) deque.
pollLast();
    deque.offerLast(i);
    // 3. Store result once first full window of size K is formed:
    if (i >= k - 1) res[i - k + 1] = nums[deque.peekFirst()];
  }
  return res;
}
```

COMPLETE SLIDING WINDOW PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
643	Max Average Subarray I	Fixed-Size Window K	O(N)	O(1)	E
3	Longest Substring Without Repeating	Dynamic Shrinking Set Window	O(N)	O(min(N, M))	M
1004	Max Consecutive Ones III	K-Flip Zero Window	O(N)	O(1)	M
567	Permutation in String	Fixed Frequency Matcher	O(N)	O(1)	M
424	Longest Repeating Replacement	Max Frequency Window	O(N)	O(1)	M
76	Minimum Window Substring	2-Map Matcher	O(N)	O(1)	H
239	Sliding Window Maximum	Monotonic Decreasing Deque	O(N)	O(K)	H

LC 643 · Maximum Average Subarray I

EASY

Pattern: Fixed Window K
Key Insight: `sum += in - out`.

```
public double findMaxAverage(int[] nums, int k) {
  int sum = 0; for (int i = 0; i < k; i++) sum += nums[i];
  int maxS = sum;
  for (int r = k; r < nums.length; r++) {
    sum += nums[r] - nums[r - k]; maxS = Math.max(maxS, sum);
  }
  return (double) maxS / k;
}
```

LC 3 · Longest Substring Without Repeating

MEDIUM

Pattern: Dynamic Set Window
Key Insight: Shrink `left` on duplicate.

```
public int lengthOfLongestSubstring(String s) {
  Set<Character> set = new HashSet<>();
  int l = 0, maxL = 0;
  for (int r = 0; r < s.length(); r++) {
    while (set.contains(s.charAt(r))) { set.remove(s.charAt(l)); l++; }
    set.add(s.charAt(r)); maxL = Math.max(maxL, r - l + 1);
  }
  return maxL;
}
```

LC 1004 · Max Consecutive Ones III

MEDIUM

Pattern: K-Flip Window
Key Insight: Shrink `l` when `zeros > k`.

```
public int longestOnes(int[] nums, int k) {
  int l = 0, zeros = 0, maxL = 0;
  for (int r = 0; r < nums.length; r++) {
    if (nums[r] == 0) zeros++;
    while (zeros > k) { if (nums[l] == 0) zeros--; l++; }
    maxL = Math.max(maxL, r - l + 1);
  }
  return maxL;
}
```

LC 567 · Permutation in String — MEDIUM

Pattern: Fixed Frequency Match
Key Insight: Match `int[26]` frequency arrays.

```
public boolean checkInclusion(Strin
g s1, String s2) {
  if (s1.length() > s2.length()) re
turn false;
  int[] c1 = new int[26], c2 = new
int[26];
  for (int i = 0; i < s1.length();
i++) {
    c1[s1.charAt(i)-'a']++; c2[s2.c
harAt(i)-'a']++;
  }
  if (Arrays.equals(c1, c2)) return
true;
  for (int l = 0, r = s1.length();
r < s2.length(); l++, r++) {
    c2[s2.charAt(r)-'a']++; c2[s2.c
harAt(l)-'a']--;
    if (Arrays.equals(c1, c2)) retu
rn true;
  }
  return false;
}
```

LC 424 · Longest Repeating Char Replacement — MEDIUM

Pattern: Max Frequency Window
Key Insight: `(r - l + 1) - maxFreq > k`.

```
public int characterReplacement(Str
ing s, int k) {
  int[] count = new int[26];
  int l = 0, maxF = 0, maxL = 0;
  for (int r = 0; r < s.length(); r
++) {
    maxF = Math.max(maxF, ++count
[s.charAt(r) - 'A']);
    while ((r - l + 1) - maxF > k)
{ count[s.charAt(l) - 'A']--; l++;
}
    maxL = Math.max(maxL, r - l +
1);
  }
  return maxL;
}
```

LC 76 · Minimum Window Substring — HARD

Pattern: 2-Map Matcher
Key Insight: Shrink `l` while `formed == required`.

```
public String minWindow(String s, Str
ing t) {
  int[] map = new int[128]; for (char
c : t.toCharArray()) map[c]++;
  int l = 0, r = 0, count = t.length
(), minL = Integer.MAX_VALUE, start =
0;
  while (r < s.length()) {
    if (map[s.charAt(r++)]-- > 0) cou
nt--;
    while (count == 0) {
      if (r - l < minL) { minL = r -
l; start = l; }
      if (map[s.charAt(l++)]++ == 0)
count++;
    }
  }
  return minL == Integer.MAX_VALUE ?
"" : s.substring(start, start + min
L);
}
```

LC 239 · Sliding Window Maximum — HARD

Pattern: Monotonic Deque
Key Insight: Deque front stores window max index.

```
public int[] maxSlidingWindow(int[] n
ums, int k) {
  int n = nums.length; int[] res = ne
w int[n - k + 1];
  Deque<Integer> dq = new ArrayDeque<
>();
  for (int i = 0; i < n; i++) {
    if (!dq.isEmpty() && dq.peekFirst
() < i - k + 1) dq.pollFirst();
    while (!dq.isEmpty() && nums[dq.p
eekLast()] < nums[i]) dq.pollLast();
    dq.offerLast(i);
    if (i >= k - 1) res[i - k + 1] =
nums[dq.peekFirst()];
  }
  return res;
}
```

🔍 DRY RUN — Longest Substring Without Repeating ("abcabcbb")

Step	right (char)	set before	Action	set after	maxLen
1	0 ('a')	{}	Add 'a'	{'a'}	1
2	1 ('b')	{'a'}	Add 'b'	{'a', 'b'}	2
3	2 ('c')	{'a', 'b'}	Add 'c'	{'a', 'b', 'c'}	3
4	3 ('a')	{'a', 'b', 'c'}	Duplicate 'a'! Shrink left past 'a'	{'b', 'c', 'a'}	3 ✅

📐 MATHEMATICAL PROOF — O(N) LINEAR RUNTIME OF DYNAMIC WINDOW

Claim: Dynamic Sliding Window runs in exact O(N) total time despite nested `while` loop.

Proof: The `right` pointer increments N times. The `left` pointer increments at most N times across the entire execution. Since `left` never moves backward, total operations <= 2N -> **Exact O(N) Linear Time!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Write Fixed Window K template in 2 minutes
☐ Code Longest Substring Without Repeating in O(N)
☐ Write Max Consecutive Ones III k-flip template
☐ Code Permutation in String with `int[26]` match
☐ Code Minimum Window Substring in 10 minutes
☐ Write Sliding Window Maximum with Monotonic Deque
☐ Prove why total operations <= 2N for dynamic window
☐ Explain why sliding window fails for negative numbers

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Expand Right, Shrink Left

In dynamic window problems, ALWAYS expand `right` pointer first to incorporate new element, then shrink `left` pointer inside a `while` loop until window condition becomes valid again!

Rule 2 — Use Deque Front for Window Maximum

In Monotonic Deque, store ARRAY INDICES instead of values! Indices allow checking if front element fell out of window range (`deque.peekFirst() < right - k + 1`).

BINARY SEARCH MASTERCLASS

FAANG Master Guide — Java Edition

🎬 THE REAL-WORLD STORY

Imagine guessing a secret number between 1 and 100. Every time you guess, the host tells you either *"Too High"* or *"Too Low"*.

💡 THE BINARY SEARCH AHA MOMENT

Guessing 50 splits the search space EXACTLY in half! If too low, eliminate 1..50. Next guess 75. Search space shrinks exponentially: 100 → 50 → 25 → 12 → 6 → 3 → 1. Found in at most **7 steps (log2 100)**!

🤔 WHY LINEAR SEARCH FAILS

- **Linear Search O(N):** Checks elements 1 by 1. For N = 1,000,000,000 elements, requires up to 1 Billion checks!
- **Binary Search O(log N):** Needs at most **30 checks** for 1 Billion elements!
- **Monotonic Property:** Works on ANY search space that transitions monotonically from `False → True`!

📌 SEARCH RANGE NARROWING

Safe Middle Index Formula:

```
// Prevents Integer Overflow w
hen (L + R) > Integer.MAX_VALU
E!
int mid = left + (right - lef
t) / 2;

If arr[mid] < target → left =
mid + 1
If arr[mid] > target → right
= mid - 1
If arr[mid] == target → Foun
d!
```

📚 PREREQUISITES & COMPLEXITY REASONING

Pattern / Operation	Time Complexity	Auxiliary Space
Classic Binary Search (Exact Value)	**O(log N)**	**O(1) in-place**
Lower Bound / Upper Bound	**O(log N)**	**O(1) space**
Search in Rotated Sorted Array	**O(log N)**	**O(1) space**
Search on Answer (Koko Eating Bananas)	**O(N log(Max - Min))**	**O(1) space**

⚡ CLUES THAT SCREAM BINARY SEARCH

1. *"Array is SORTED"* and target index requested → Classic Binary Search / Lower Bound
2. *"Find minimum speed / maximum capacity satisfying condition C"* → Search on Answer
3. *"Sorted array rotated at unknown pivot"* → Modified Rotated Binary Search

PATTERN 1

SEARCH IN ROTATED SORTED ARRAY (Half Sorted Check)

e.g. Search in Rotated Sorted Array (LC 33)

WHEN TO USE

Array is rotated at pivot. Determine which half (`left..mid` or `mid..right`) is strictly sorted, then check if target lies within that sorted range!

TEMPLATE — LC 33

```
public int search(int[] nums, int target) {
  int l = 0, r = nums.length - 1;
  while (l ⩽ r) {
    int mid = l + (r - l) / 2;
    if (nums[mid] == target) return mid;
    // Left half is sorted:
    if (nums[l] ⩽ nums[mid]) {
      if (nums[l] ⩽ target && target < nums[mid]) r = mid -
1;
      else l = mid + 1;
    } else { // Right half is sorted:
      if (nums[mid] < target && target ⩽ nums[r]) l = mid +
1;
      else r = mid - 1;
    }
  }
  return -1;
}
```

PATTERN 2 SEARCH A 2D MATRIX (Virtual 1D Flattening) e.g. Search a 2D Matrix (LC 74)

WHEN TO USE

Flatten M x N matrix virtually to 1D index `0 .. M*N - 1`. Row = `mid / cols`, Col = `mid % cols` in O(log(M · N)) time.

TEMPLATE — LC 74

```
public boolean searchMatrix(int[][] matrix, int target) {
  int rows = matrix.length, cols = matrix[0].length;
  int l = 0, r = rows * cols - 1;
  while (l <= r) {
    int mid = l + (r - l) / 2;
    int val = matrix[mid / cols][mid % cols];
    if (val == target) return true;
    if (val < target) l = mid + 1; else r = mid - 1;
  }
  return false;
}
```

PATTERN 4

FIND MINIMUM IN ROTATED SORTED ARRAY (Pivot Search)

e.g. Find Minimum in Rotated Sorted Array (LC 153)

RIGHT ELEMENT COMPARISON

Compare `nums[mid]` against `nums[right]`. If `nums[mid] > nums[right]`, pivot lies on the right half (`l = mid + 1`). Otherwise pivot is on left half (`r = mid`)!

TEMPLATE — LC 153

```
public int findMin(int[] nums) {
  int l = 0, r = nums.length - 1;
  while (l < r) {
    int mid = l + (r - l) / 2;
    if (nums[mid] > nums[r]) {
      l = mid + 1; // Pivot is in right half!
    } else {
      r = mid;     // Pivot is mid or in left half!
    }
  }
  return nums[l];
}
```

PATTERN 5 MEDIAN OF TWO SORTED ARRAYS (Binary Search on Partition)

e.g. Median of Two Sorted Arrays (LC 4)

PARTITION MATH O(LOG(MIN(M, N)))

Binary search partition `i` on smaller array A. Calculate corresponding partition `j` on array B. Check `A[i-1] <= B[j]` and `B[j-1] <= A[i]`!

TEMPLATE — LC 4

```
public double findMedianSortedArrays(int[] A, int[] B) {
  if (A.length > B.length) return findMedianSortedArrays(B, A);
  int m = A.length, n = B.length;
  int l = 0, r = m;
  while (l ⩽ r) {
    int i = l + (r - l) / 2;
    int j = (m + n + 1) / 2 - i;
    int maxLeftA = (i == 0) ? Integer.MIN_VALUE : A[i - 1];
    int minRightA = (i == m) ? Integer.MAX_VALUE : A[i];
    int maxLeftB = (j == 0) ? Integer.MIN_VALUE : B[j - 1];
    int minRightB = (j == n) ? Integer.MAX_VALUE : B[j];
    if (maxLeftA ⩽ minRightB && maxLeftB ⩽ minRightA) {
      if ((m + n) % 2 == 1) return Math.max(maxLeftA, maxLeftB);
      return (Math.max(maxLeftA, maxLeftB) + Math.min(minRightA,
minRightB)) / 2.0;
    } else if (maxLeftA > minRightB) r = i - 1;
    else l = i + 1;
  }
  return 0.0;
}
```

BINARY SEARCH MASTERCLASS

Decision Tree & Trigger Words

📊 COMPLETE BINARY SEARCH PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
704	Binary Search	Classic Binary Search	O(log N)	O(1)	E
33	Search in Rotated Sorted Array	Rotated Half Check	O(log N)	O(1)	M
74	Search a 2D Matrix	Virtual 1D Flattening	O(log(M · N))	O(1)	M
875	Koko Eating Bananas	Search on Answer	O(N log(Max))	O(1)	M
153	Find Min in Rotated Array	Pivot Search	O(log N)	O(1)	M
34	First & Last Position	Lower & Upper Bound	O(log N)	O(1)	M
4	Median of Two Sorted Arrays	Binary Search on Partition	O(log(min(M, N)))	O(1)	H

LC 704 · Binary Search

EASY

Pattern: Classic BS
Key Insight: Standard `l <= r` convergence.

```
public int search(int[] nums, i
nt target) {
  int l = 0, r = nums.length -
1;
  while (l ⩽ r) {
    int m = l + (r - l) / 2;
    if (nums[m] == target) retu
rn m;
    if (nums[m] < target) l = m
+ 1; else r = m - 1;
  }
  return -1;
}
```

LC 33 · Search in Rotated Array

MEDIUM

Pattern: Rotated Half Check
Key Insight: Check if left half is sorted.

```
public int search(int[] nums, i
nt target) {
  int l = 0, r = nums.length -
1;
  while (l ⩽ r) {
    int m = l + (r - l) / 2;
    if (nums[m] == target) retu
rn m;
    if (nums[l] ⩽ nums[m]) {
      if (nums[l] ⩽ target &&
target < nums[m]) r = m - 1; el
se l = m + 1;
    } else {
      if (nums[m] < target && t
arget ⩽ nums[r]) l = m + 1; el
se r = m - 1;
    }
  }
  return -1;
}
```

LC 74 · Search a 2D Matrix

MEDIUM

Pattern: Virtual 1D Flatten
Key Insight: `row = mid / cols`, `col = mid % cols`.

```
public boolean searchMatrix(int
[][] matrix, int target) {
  int R = matrix.length, C = ma
trix[0].length, l = 0, r = R *
C - 1;
  while (l ⩽ r) {
    int m = l + (r - l) / 2, va
l = matrix[m / C][m % C];
    if (val == target) return t
rue;
    if (val < target) l = m +
1; else r = m - 1;
  }
  return false;
}
```

LC 875 · Koko Eating Bananas — MEDIUM

Pattern: Search on Answer
Key Insight: BS speed `1 .. max(piles)`.

```
public int minEatingSpeed(int[] piles, int h) {
  int l = 1, r = 1000000000;
  while (l < r) {
    int m = l + (r - l) / 2, hrs = 0;
    for (int p : piles) hrs += (p + m - 1) / m;
    if (hrs <= h) r = m; else l = m + 1;
  }
  return l;
}
```

LC 153 · Find Min in Rotated Array — MEDIUM

Pattern: Pivot Search
Key Insight: Compare `nums[mid]` against `nums[r]`.

```
public int findMin(int[] nums) {
  int l = 0, r = nums.length - 1;
  while (l < r) {
    int m = l + (r - l) / 2;
    if (nums[m] > nums[r]) l = m + 1; else r = m;
  }
  return nums[l];
}
```

LC 4 · Median of Two Sorted Arrays — HARD

Pattern: Partition BS
Key Insight: BS partition on smaller array.

```
public double findMedianSortedArrays(int[] A, int[] B) {
  if (A.length > B.length) return findMedianSortedArrays(B, A);
  int m = A.length, n = B.length, l = 0, r = m;
  while (l <= r) {
    int i = l + (r - l) / 2, j = (m + n + 1) / 2 - i;
    int maxLA = (i == 0) ? Integer.MIN_VALUE : A[i - 1];
    int minRA = (i == m) ? Integer.MAX_VALUE : A[i];
    int maxLB = (j == 0) ? Integer.MIN_VALUE : B[j - 1];
    int minRB = (j == n) ? Integer.MAX_VALUE : B[j];
    if (maxLA <= minRB && maxLB <= minRA) {
      if ((m + n) % 2 == 1) return Math.max(maxLA, maxLB);
      return (Math.max(maxLA, maxLB) + Math.min(minRA, minRB)) / 2.0;
    } else if (maxLA > minRB) r = i - 1; else l = i + 1;
  }
  return 0.0;
}
```

LC 34 · First & Last Position — MEDIUM

Pattern: Lower & Upper Bound
Key Insight: `lowerBound(target)` and `upperBound(target) - 1`.

```
public int[] searchRange(int[] nums, int target) {
  int first = lowerBound(nums, target);
  if (first == nums.length || nums[first] != target) return new int[]{-1, -1};
  int last = upperBound(nums, target) - 1;
  return new int[]{first, last};
}
```

BINARY SEARCH MASTERCLASS

Dry Run, Proofs & Mastery Cheat Sheet

🔍 DRY RUN — Search in Rotated Array ([4, 5, 6, 7, 0, 1, 2], target = 0)

Step	l (val)	r (val)	mid (val)	Sorted Half Check	Action
1	0 (4)	6 (2)	3 (7)	Left half [4..7] is sorted (`4 <= 7`). Target 0 NOT in [4, 7).	`l = mid + 1` (4)
2	4 (0)	6 (2)	5 (1)	Left half [0..1] is sorted (`0 <= 1`). Target 0 IS in [0, 1).	`r = mid - 1` (4)
3	4 (0)	4 (0)	4 (0)	`nums[4] == 0` → Target Match Found!	Return index 4 ✅

📐 MATHEMATICAL PROOF — LOGARITHMIC TIME COMPLEXITY O(log N)

Claim: Binary Search terminates in at most `ceil(log2 N)` steps.

Proof: At step k, search space size is N / 2^k. Loop terminates when $N / 2^k = 1 \implies 2^k = N \implies k = \log_2 N$. Thus, max steps required for 1 Billion items is \log_2(10^9) ≈ 30 → **Exact O(log N) Proof!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Write Classic Binary Search in 2m without off-by-one errors
- ☐ Code Lower Bound template (`first idx >= target`)
- ☐ Code Upper Bound template (`first idx > target`)
- ☐ Write Search in Rotated Sorted Array in O(log N)
- ☐ Code Search a 2D Matrix with virtual 1D flattening
- ☐ Implement Koko Eating Bananas search on answer
- ☐ Code Find Minimum in Rotated Sorted Array
- ☐ Prove why `mid = l + (r - l) / 2` avoids overflow

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Always Check Which Half is Sorted First
In Rotated Array Binary Search, ALWAYS check `if (nums[l] <= nums[mid])` to determine which side is monotonically increasing before checking if target is bounded in that range!

Rule 2 — Search on Answer Requires Monotonicity
You can apply Search on Answer ONLY if the feasibility predicate function `isPossible(x)` is monotonic: `False, False, False, True, True, True`!

LINKED LIST MASTERCLASS

FAANG Master Guide — Java Edition

🎬 THE REAL-WORLD STORY

Imagine a treasure hunt where each clue box contains a gold coin and a paper slip giving the exact GPS location of the next clue box.

💡 THE LINKED LIST AHA MOMENT

Unlike arrays requiring contiguous memory blocks, linked list nodes live anywhere in RAM! Inserting or deleting a node at the head or middle takes **O(1) pointer updates** without shifting elements!

🧐 WHY ARRAYS VS LINKED LISTS?

- **Array Insert at Index 0:** Slow O(N) shift of all elements.
- **Linked List Insert at Head:** Fast O(1) pointer updates: `newNode.next = head; head = newNode`.
- **Random Access:** Array is O(1) via `arr[i]`, Linked List is O(N) via pointer traversal.

📌 VISUALIZING 3-POINTER REVERSAL

Step-by-step Pointer Reversal Mechanics:

```
Initial:  prev=null  curr -> [1] -> [2] -> [3] -> null
                              next = curr.next ([2])

Step 1:   curr.next = prev (points [1] back to null)
Step 2:   prev = curr      (moves prev to [1])
Step 3:   curr = next      (moves curr to [2])

Result:   null <- [1] <- [2] <- [3] (prev is new head!)
```

📚 PREREQUISITES & COMPLEXITY REASONING

Operation / Pattern	Time Complexity	Auxiliary Space
Reverse Linked List (3 Pointers)	**O(N) single pass**	**O(1) in-place**
Detect Cycle (Floyd's Fast & Slow)	**O(N) single pass**	**O(1) space**
Merge Two Sorted Lists (Dummy Head)	**O(N + M)**	**O(1) space**
Reorder List (Find Mid + Reverse + Merge)	**O(N)**	**O(1) space**

⚡ CLUES THAT SCREAM LINKED LIST

1. *"Reverse in-place in O(1) space"* -> 3-Pointer Iterative Reversal (`prev`, `curr`, `next`)
2. *"Find middle / Check for cycle"* -> Fast & Slow Pointers (`slow = slow.next`, `fast = fast.next.next`)
3. *"Modify head pointer dynamically"* -> Dummy Head Node Sentinel (`ListNode dummy = new ListNode(0)`)

PATTERN 1 REVERSE LINKED LIST (3-Pointer Iterative Reversal) e.g. Reverse Linked List (LC 206)

WHEN TO USE

Reverse direction of all next pointers in-place in O(N) time and O(1) space.

TEMPLATE — LC 206

```
public ListNode reverseList(ListNode head) {
  ListNode prev = null, curr = head;
  while (curr != null) {
    ListNode next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
  }
  return prev;
}
```

PATTERN 2 MERGE TWO SORTED LISTS (Dummy Sentinel Node) e.g. Merge Two Sorted Lists (LC 21)

WHEN TO USE

Combine two pre-sorted lists into a single sorted list using a dummy head node in O(N + M) time.

TEMPLATE — LC 21

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
  ListNode dummy = new ListNode(0);
  ListNode curr = dummy;
  while (l1 != null && l2 != null) {
    if (l1.val <= l2.val) { curr.next = l1; l1 = l1.next; }
    else { curr.next = l2; l2 = l2.next; }
    curr = curr.next;
  }
  curr.next = (l1 != null) ? l1 : l2;
  return dummy.next;
}
```

PATTERN 5 REMOVE NTH NODE FROM END (Fast Ahead Gap Window)

e.g. Remove Nth Node From End of List (LC 19)

N-STEP GAP WINDOW

Advance `fast` N steps ahead. Then move `fast` and `slow` together until `fast.next == null`. `slow` will point directly before target node!

TEMPLATE — LC 19

```
public ListNode removeNthFromEnd(ListNode head, int n) {
  ListNode dummy = new ListNode(0, head);
  ListNode fast = dummy, slow = dummy;
  for (int i = 0; i < n; i++) fast = fast.next;
  while (fast.next != null) {
    fast = fast.next;
    slow = slow.next;
  }
  slow.next = slow.next.next; // Bypass target node!
  return dummy.next;
}
```

PATTERN 6 ADD TWO NUMBERS (Digit Carry Propagation) e.g. Add Two Numbers (LC 2)

CARRY PROPAGATION

Traverse both lists digit by digit. Sum `val1 + val2 + carry`. New digit node = `sum % 10`, `carry = sum / 10`.

TEMPLATE — LC 2

```
public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
  ListNode dummy = new ListNode(0);
  ListNode curr = dummy;
  int carry = 0;
  while (l1 != null || l2 != null || carry != 0) {
    int v1 = (l1 != null) ? l1.val : 0;
    int v2 = (l2 != null) ? l2.val : 0;
    int sum = v1 + v2 + carry;
    carry = sum / 10;
    curr.next = new ListNode(sum % 10);
    curr = curr.next;
    if (l1 != null) l1 = l1.next;
    if (l2 != null) l2 = l2.next;
  }
  return dummy.next;
}
```

COMPLETE LINKED LIST PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
206	Reverse Linked List	3-Pointer Iterative Reversal	O(N)	O(1)	E
21	Merge Two Sorted Lists	Dummy Node Sentinel	O(N + M)	O(1)	E
141	Linked List Cycle	Floyd's Fast & Slow Pointers	O(N)	O(1)	E
143	Reorder List	Mid + Reverse + Interleave	O(N)	O(1)	M
19	Remove Nth Node From End	Fast Ahead Gap Window	O(N)	O(1)	M
2	Add Two Numbers	Digit Carry Propagation	O(max(N, M))	O(1)	M
25	Reverse Nodes in k-Group	Group Reversal Loop	O(N)	O(1)	H

LC 206 · Reverse Linked List

EASY

Pattern: 3-Pointer Iterative
Key Insight: `curr.next = prev`.

```
public ListNode reverseList(ListNo
de head) {
  ListNode prev = null, curr = hea
d;
  while (curr != null) {
    ListNode nxt = curr.next; cur
r.next = prev; prev = curr; curr =
nxt;
  }
  return prev;
}
```

LC 21 · Merge Two Sorted Lists

EASY

Pattern: Dummy Node Sentinel
Key Insight: Compare `l1.val` vs `l2.val`.

```
public ListNode mergeTwoLists(List
Node l1, ListNode l2) {
  ListNode dummy = new ListNode
(0), c = dummy;
  while (l1 != null && l2 != null)
{
    if (l1.val <= l2.val) { c.next
= l1; l1 = l1.next; }
    else { c.next = l2; l2 = l2.ne
xt; }
    c = c.next;
  }
  c.next = (l1 != null) ? l1 : l2;
  return dummy.next;
}
```

LC 141 · Linked List Cycle

EASY

Pattern: Fast & Slow Pointers
Key Insight: Check `slow == fast`.

```
public boolean hasCycle(ListNode h
ead) {
  ListNode s = head, f = head;
  while (f != null && f.next != nu
ll) {
    s = s.next; f = f.next.next;
    if (s == f) return true;
  }
  return false;
}
```

LC 143 · Reorder List — MEDIUM

Pattern: Mid + Reverse + Interleave
Key Insight: Split at mid, reverse right, merge.

```
public void reorderList(ListNode head) {
  if (head == null || head.next == null) return;
  ListNode s = head, f = head;
  while (f != null && f.next != null) { s = s.next; f = f.next.next; }
  ListNode prev = null, curr = s.next; s.next = null;
  while (curr != null) { ListNode n = curr.next; curr.next = prev; prev = curr; curr = n; }
  ListNode fst = head, snd = prev;
  while (snd != null) {
    ListNode t1 = fst.next, t2 = snd.next;
    fst.next = snd; snd.next = t1; fst = t1; snd = t2;
  }
}
```

LC 19 · Remove Nth Node From End — MEDIUM

Pattern: Fast Ahead Gap Window
Key Insight: Advance `fast` N steps ahead.

```
public ListNode removeNthFromEnd(ListNode head, int n) {
  ListNode dummy = new ListNode(0, head);
  ListNode f = dummy, s = dummy;
  for (int i = 0; i < n; i++) f = f.next;
  while (f.next != null) { f = f.next; s = s.next; }
  s.next = s.next.next;
  return dummy.next;
}
```

LC 25 · Reverse Nodes in k-Group — HARD

Pattern: Group Reversal Loop
Key Insight: Check `getKth()`, reverse k nodes.

```
public ListNode reverseKGroup(ListNode head, int k) {
  ListNode dummy = new ListNode(0, head), gPrev = dummy;
  while (true) {
    ListNode kth = getKth(gPrev, k);
if (kth == null) break;
    ListNode gNxt = kth.next, prev = kth.next, curr = gPrev.next;
    while (curr != gNxt) { ListNode t = curr.next; curr.next = prev; prev = curr; curr = t; }
    ListNode tmp = gPrev.next; gPrev.next = kth; gPrev = tmp;
  }
  return dummy.next;
}
private ListNode getKth(ListNode c, int k) { while (c != null && k > 0) { c = c.next; k--; } return c; }
```

LC 2 · Add Two Numbers — MEDIUM

Pattern: Digit Carry Propagation
Key Insight: `val1 + val2 + carry`.

```
public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
  ListNode dummy = new ListNode(0), c = dummy; int carry = 0;
  while (l1 != null || l2 != null || carry != 0) {
    int v1 = (l1 != null) ? l1.val : 0, v2 = (l2 != null) ? l2.val : 0;
    int sum = v1 + v2 + carry; carry = sum / 10;
    c.next = new ListNode(sum % 10); c = c.next;
    if (l1 != null) l1 = l1.next; if (l2 != null) l2 = l2.next;
  }
  return dummy.next;
}
```

🔍 DRY RUN — 3-Pointer Reversal ([1] -> [2] -> [3] -> null)

Step	prev	curr	nextTemp	curr.next set to	State
1	null	1	2	null (`1.next = null`)	null <- [1]
2	1	2	3	1 (`2.next = 1`)	null <- [1] <- [2]
3	2	3	null	2 (`3.next = 2`)	null <- [1] <- [2] <- [3] ✅

📐 MATHEMATICAL PROOF — FLOYD'S CYCLE DETECTION O(N)

Claim: Fast (2x speed) and Slow (1x speed) pointers are guaranteed to meet inside a cycle of length C.

Proof: At each step, distance between fast and slow decreases by 1 (`2 - 1 = 1`). Inside a cycle of length C, fast reduces gap from C to 0 in at most C steps. Total steps <= N -> **Exact O(N) Linear Time!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Code 3-pointer list reversal iteratively in 2 minutes
☐ Implement Merge Two Sorted Lists using Dummy Head
☐ Detect cycle using Floyd's Fast & Slow Pointers
☐ Find middle node using Fast & Slow Pointers
☐ Code Reorder List (Mid + Reverse + Interleave)
☐ Remove N-th node from end using N-step gap
☐ Code Reverse Nodes in k-Group in O(1) space
☐ Prove why Floyd's cycle detection runs in O(N)

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Always Save Next Pointer Before Reconnecting

In pointer reversal, ALWAYS store `ListNode nextTemp = curr.next` before executing `curr.next = prev`! Otherwise, you lose reference to the rest of the list!

Rule 2 — Use Dummy Node for Head Modifications

Whenever deleting or inserting at the head of a linked list, initialize `ListNode dummy = new ListNode(0, head)` to avoid special null check branching!

💡 WHY STACKS?

Arrays give random access, but Stacks enforce **LIFO (Last-In, First-Out) order**. This is essential for matching nested structures, expression evaluation, and backtracking memory!

💡 THE ARRAYDEQUE AHA

Never use `java.util.Stack` in Java! It inherits from `Vector` and uses synchronized methods (slow). Always use `Deque stack = new ArrayDeque<>()`!

1 3 CORE VARIANTS

- **Matching / Pair Cancellation:** Valid parentheses, string reduction, asteroid collisions.
- **Monotonic Stack:** Stack of strictly increasing or decreasing elements to find next greater / previous smaller element in O(N)!
- **Expression Evaluation:** Reverse Polish Notation (RPN), infix to postfix operators.

2 VISUALIZING MONOTONIC STACK

Decreasing Stack (Next Greater Element):

```
Array: [ 73, 74, 75, 71, 69, 72 ]
Push 73 ➔ Stack: [73]
Element 74 > 73 ➔ Pop 73! Next G
reater of 73 is 74.
Push 74 ➔ Stack: [74]
Element 75 > 74 ➔ Pop 74! Next G
reater of 74 is 75.
Push 75, 71, 69 ➔ Stack: [75, 7
1, 69]
```

📚 PREREQUISITES & COMPLEXITY REASONING

Operation	ArrayDeque Complexity	Deprecated Stack
Push (top insert)	**O(1) amortized**	O(1) synchronized
Pop (top remove)	**O(1)**	O(1) synchronized
Peek (top view)	**O(1)**	O(1) synchronized
Monotonic Scan	**O(N) total**	O(N) total

⚡ CLUES THAT SCREAM STACK

1. *"Valid parentheses / matching brackets"* -> Character Stack
2. *"Next greater element" / "Daily temperatures"* -> Monotonic Decreasing Stack
3. *"Largest rectangle in histogram"* -> Monotonic Increasing Stack
4. *"Evaluate arithmetic expression / RPN"* -> Operand Stack

PATTERN 1 PARENTHESES & BRACKET MATCHING

e.g. Valid Parentheses (LC 20), Backspace String Compare (LC 844)

WHEN TO USE

Push opening brackets `(`, `[`, `{`. When closing bracket encountered, verify top of stack matches and pop!

TEMPLATE — LC 20

```
public boolean isValid(String s) {
  Deque<Character> stack = new ArrayDeque<>();
  for (char c : s.toCharArray()) {
    if (c == '(') stack.push(')');
    else if (c == '{') stack.push('}');
    else if (c == '[') stack.push(']');
    else if (stack.isEmpty() || stack.pop() != c) return false;
  }
  return stack.isEmpty();
}
```

PATTERN 2 PAIR CANCELLATION & SIMULATION e.g. Asteroid Collision (LC 735), Remove All Adjacent Duplicates

WHEN TO USE

Simulate collision/cancellation between incoming elements and previous active elements at top of stack.

TEMPLATE — LC 735

```
public int[] asteroidCollision(int[] asteroids) {
  Deque<Integer> stack = new ArrayDeque<>();
  for (int a : asteroids) {
    while (!stack.isEmpty() && a < 0 && stack.peek() > 0) {
      int diff = a + stack.peek();
      if (diff < 0) stack.pop(); // Incoming destroys top
      else if (diff == 0) { stack.pop(); a = 0; } // Both destroye
d
      else a = 0; // Incoming destroyed
    }
    if (a != 0) stack.push(a);
  }
  int[] res = new int[stack.size()];
  for (int i = res.length - 1; i >= 0; i--) res[i] = stack.pop();
  return res;
}
```

PATTERN 5 LARGEST RECTANGLE IN HISTOGRAM (Monotonic Increasing Stack)

e.g. Largest Rectangle in Histogram (LC 84), Maximal Rectangle

MONOTONIC INCREASING STACK MECHANICS

When `heights[i]` is smaller than stack top, the popped height's right boundary is `i`, and left boundary is current stack top!

TEMPLATE — LC 84

```
public int largestRectangleArea(int[] heights) {
  int n = heights.length, maxArea = 0;
  Deque<Integer> stack = new ArrayDeque<>();
  for (int i = 0; i <= n; i++) {
    int h = (i == n) ? 0 : heights[i];
    while (!stack.isEmpty() && h < heights[stack.peek()]) {
      int height = heights[stack.pop()];
      int width = stack.isEmpty() ? i : i - stack.peek() - 1;
      maxArea = Math.max(maxArea, height * width);
    }
    stack.push(i);
  }
  return maxArea;
}
```

PATTERN 6 EVALUATE REVERSE POLISH NOTATION (Postfix Operators)

e.g. Evaluate Reverse Polish Notation (LC 150)

WHEN TO USE

Push numbers. When operator (+, -, *, /) encountered, pop second operand `b`, pop first operand `a`, evaluate `a op b`, push result!

TEMPLATE — LC 150

```
public int evalRPN(String[] tokens) {
  Deque<Integer> stack = new ArrayDeque<>();
  for (String t : tokens) {
    if (t.equals("+")) stack.push(stack.pop() + stack.pop());
    else if (t.equals("-")) { int b = stack.pop(), a = stack.pop
(); stack.push(a - b); }
    else if (t.equals("*")) stack.push(stack.pop() * stack.pop());
    else if (t.equals("/")) { int b = stack.pop(), a = stack.pop
(); stack.push(a / b); }
    else stack.push(Integer.parseInt(t));
  }
  return stack.pop();
}
```

COMPLETE STACK PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
20	Valid Parentheses	Parenthesis Match	O(N)	O(N)	E
155	Min Stack	Dual Stack Minimum	O(1) all ops	O(N)	M
150	Evaluate RPN	Operand Stack	O(N)	O(N)	M
739	Daily Temperatures	Monotonic Decreasing	O(N)	O(N)	M
853	Car Fleet	Time Stack	O(N log N)	O(N)	M
735	Asteroid Collision	Collision Simulation	O(N)	O(N)	M
394	Decode String	Nested Dual Stack	O(N)	O(N)	M
84	Largest Rectangle in Histogram	Monotonic Increasing	O(N)	O(N)	H
22	Generate Parentheses	Backtracking Stack	O(4^N / sqrt(N))	O(N)	M

LC 20 · Valid Parentheses — EASY

Pattern: Parenthesis Match
Key Insight: Push expected closing character.

```
public boolean isValid(String s) {
  Deque<Character> st = new ArrayDeque<>();
  for (char c : s.toCharArray()) {
    if (c == '(') st.push(')');
    else if (c == '{') st.push('}');
    else if (c == '[') st.push(']');
    else if (st.isEmpty() || st.pop() != c) return false;
  }
  return st.isEmpty();
}
```

LC 155 · Min Stack — MEDIUM

Pattern: Dual Stack
Key Insight: Auxiliary stack tracks minimum value.

```
class MinStack {
  Deque<Integer> st = new ArrayDeque<>(), minSt = new ArrayDeque<>();
  public void push(int val) {
    st.push(val);
    if (minSt.isEmpty() || val <= minSt.peek()) minSt.push(val);
  }
  public void pop() { if (st.pop().equals(minSt.peek())) minSt.pop(); }
  public int top() { return st.peek(); }
  public int getMin() { return minSt.peek(); }
}
```

LC 150 · Evaluate RPN — MEDIUM

Pattern: RPN Operand Stack
Key Insight: Pop `b` then `a`, apply `a op b`.

```
public int evalRPN(String[] tokens) {
  Deque<Integer> st = new ArrayDeque<>();
  for (String t : tokens) {
    if (t.equals("+")) st.push(st.pop() + st.pop());
    else if (t.equals("-")) { int b=st.pop(), a=st.pop(); st.push(a-b); }
    else if (t.equals("*")) st.push(st.pop() * st.pop());
    else if (t.equals("/")) { int b=st.pop(), a=st.pop(); st.push(a/b); }
    else st.push(Integer.parseInt(t));
  }
  return st.pop();
}
```

LC 739 · Daily Temperatures — MEDIUM

Pattern: Monotonic Decreasing
Key Insight: Stack stores indices. Pop when warmer!

```
public int[] dailyTemperatures(int
[] T) {
  int[] res = new int[T.length];
  Deque<Integer> st = new ArrayDeq
ue<>();
  for (int i = 0; i < T.length; i+
+) {
    while (!st.isEmpty() && T[i] >
T[st.peek()]) {
      int prev = st.pop(); res[pre
v] = i - prev;
    }
    st.push(i);
  }
  return res;
}
```

LC 853 · Car Fleet — MEDIUM

Pattern: Time Stack
Key Insight: Sort by position desc. Time <= top merges.

```
public int carFleet(int target, in
t[] pos, int[] speed) {
  int n = pos.length; double[][] c
ars = new double[n][2];
  for (int i = 0; i < n; i++) cars
[i] = new double[]{pos[i], (doubl
e)(target - pos[i]) / speed[i]};
  Arrays.sort(cars, (a,b)->Double.
compare(b[0], a[0]));
  Deque<Double> st = new ArrayDequ
e<>();
  for (double[] c : cars) {
    if (!st.isEmpty() && c[1] <= s
t.peek()) continue;
    st.push(c[1]);
  }
  return st.size();
}
```

LC 735 · Asteroid Collision — MEDIUM

Pattern: Collision Simulation
Key Insight: Destroy smaller top when incoming is negative.

```
public int[] asteroidCollision(int[]
ast) {
  Deque<Integer> st = new ArrayDeque<
>();
  for (int a : ast) {
    while (!st.isEmpty() && a < 0 &&
st.peek() > 0) {
      int diff = a + st.peek();
      if (diff < 0) st.pop();
      else if (diff == 0) { st.pop();
a = 0; }
      else a = 0;
    }
    if (a != 0) st.push(a);
  }
  int[] res = new int[st.size()];
  for (int i = res.length - 1; i >=
0; i--) res[i] = st.pop();
  return res;
}
```

LC 394 · Decode String — MEDIUM

Pattern: Nested Dual Stack
Key Insight: Push buffer on `[`, repeat on `]`.

```
public String decodeString(String s)
{
  Deque<Integer> countSt = new ArrayD
eque<>();
  Deque<StringBuilder> strSt = new Ar
rayDeque<>();
  StringBuilder curr = new StringBuil
der(); int k = 0;
  for (char c : s.toCharArray()) {
    if (Character.isDigit(c)) k = k *
10 + (c - '0');
    else if (c == '[') {
      countSt.push(k); strSt.push(cur
r); curr = new StringBuilder(); k =
0;
    } else if (c == ']') {
      StringBuilder prev = strSt.pop
(); int cnt = countSt.pop();
      for (int i = 0; i < cnt; i++) p
rev.append(curr);
      curr = prev;
    } else curr.append(c);
  }
  return curr.toString();
}
```

LC 84 · Largest Rectangle in Histogram — HARD

Pattern: Monotonic Increasing
Key Insight: Width $= i - \text{stack.peek()} - 1$.

```
public int largestRectangleArea(int[]
heights) {
  int n = heights.length, maxArea =
0;
  Deque<Integer> st = new ArrayDeque<
>();
  for (int i = 0; i <= n; i++) {
    int h = (i == n) ? 0 : heights
[i];
    while (!st.isEmpty() && h < heigh
ts[st.peek()]) {
      int height = heights[st.pop()];
      int width = st.isEmpty() ? i :
i - st.peek() - 1;
      maxArea = Math.max(maxArea, hei
ght * width);
    }
    st.push(i);
```

LC 22 · Generate Parentheses — MEDIUM

Pattern: Backtracking Stack
Key Insight: Add `(` if open < N, add `)` if close < open.

```
public List<String> generateParenthes
is(int n) {
  List<String> res = new ArrayList<>
();
  backtrack(res, new StringBuilder(),
0, 0, n);
  return res;
}
private void backtrack(List<String> r
es, StringBuilder sb, int open, int c
lose, int n) {
  if (sb.length() == 2 * n) { res.add
(sb.toString()); return; }
  if (open < n) { sb.append('('); bac
ktrack(res, sb, open + 1, close, n);
sb.deleteCharAt(sb.length() - 1); }
  if (close < open) { sb.append(')');
backtrack(res, sb, open, close + 1,
```

```
    }
    return maxArea;
}
```

```
n); sb.deleteCharAt(sb.length() - 1);
}
}
```

🔍 DRY RUN — Daily Temperatures ([73, 74, 75, 71, 69, 72])

Index	Temp	Stack State (Indices)	Action / Result Assignment
0	73	[0]	Push 0
1	74	[1]	74 > 73 -> Pop 0! `res[0] = 1 - 0 = 1`. Push 1
2	75	[2]	75 > 74 -> Pop 1! `res[1] = 2 - 1 = 1`. Push 2
3	71	[2, 3]	71 < 75 -> Push 3
4	69	[2, 3, 4]	69 < 71 -> Push 4
5	72	[2, 5]	72 > 69 -> Pop 4! `res[4]=1`. 72 > 71 -> Pop 3! `res[3]=2`. Push 5

📐 MATHEMATICAL PROOF — O(N) MONOTONIC STACK

Claim: The inner `while (!stack.isEmpty() && ...)` loop does not make Monotonic Stack O(N^2).

Proof: Each element is pushed onto the stack exactly once. An element can be popped from the stack at most once. Total pops across entire execution <= N. Total operations $= N \text{ pushes} + \text{at most } N \text{ pops} = 2N ->$ **Strictly O(N) Time!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Use `ArrayDeque` instead of `java.util.Stack`
- ☐ Code Valid Parentheses (LC 20) in 2m
- ☐ Implement Min Stack O(1) operations
- ☐ Code Daily Temperatures with Monotonic Stack
- ☐ Write RPN Evaluator (LC 150)
- ☐ Code Car Fleet sorting by position desc
- ☐ Code Largest Rectangle in Histogram (LC 84)
- ☐ Prove why Monotonic Stack is O(N)

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Store Indices, Not Values

In Monotonic Stack problems calculating distances (e.g. Daily Temperatures, Histogram width), store ARRAY INDICES in the Stack!

Rule 2 — Sentinel 0 Height

In Largest Rectangle in Histogram, append a dummy 0 height at index N to force popping all remaining elements from the stack!

QUEUE & DEQUE PATTERNS

FAANG Master Guide — Java Edition

🎬 THE REAL-WORLD STORY

Imagine standing in line at a movie theater ticket counter. The first person to arrive is the first person served (First-In, First-Out: FIFO).

💡 THE QUEUE & DEQUE AHA MOMENT

A Double-Ended Queue (Deque) allows **O(1) insertions and deletions at BOTH ends** (`head` and `tail`), making it the ultimate foundation for BFS tree/graph traversals and monotonic sliding window maximums!

🤔 WHY ARRAYLIST FAILS FOR QUEUES

- **ArrayList Remove First `list.remove(0)`:** Requires shifting all remaining N elements → Slow O(N) time!
- **ArrayDeque `pollFirst()`:** Uses a circular array buffer → Strict **O(1) time**!
- **Java Recommendation:** ALWAYS use `ArrayDeque` instead of legacy `Stack` or `LinkedList`!

📌 VISUALIZING DOUBLE-ENDED QUEUE

Deque Operations (Both Ends O(1)):

```
addFirst(x)  →
← addLast(x)
pollFirst()  ← [ Head | ... | Tail ] → pollLast()
peekFirst()     ^^^^^
^^^^^   peekLast()

Double-ended power: Acts as both Stack & Queue!
```

📚 PREREQUISITES & COMPLEXITY REASONING

Data Structure / Pattern	Operation	Time Complexity	Auxiliary Space
`ArrayDeque`	`offer()`, `poll()`, `peek()`	**O(1) amortized**	**O(N) circular buffer**
BFS Level Order Traversal	`queue.offer()` & `poll()`	**O(V + E)**	**O(V) queue capacity**
Monotonic Deque (Max Window)	`pollLast()`, `offerLast()`	**O(N) single pass**	**O(K) deque size**
Circular Queue Implementation	`enQueue()`, `deQueue()`	**O(1) strictly**	**O(K) fixed array**

⚡ CLUES THAT SCREAM QUEUE / DEQUE

1. *"Level order traversal / Shortest path in unweighted graph"* → Standard BFS Queue (`ArrayDeque`)
2. *"Maximum/minimum in every sliding window of size K"* → Monotonic Decreasing Deque
3. *"First non-repeating character in a stream"* → Queue + Character Frequency Map

PATTERN 1 BFS LEVEL ORDER TRAVERSAL (Queue Snapshot Loop)

e.g. Binary Tree Level Order Traversal (LC 102)

LEVEL SNAPSHOT STRATEGY

At the start of each level loop, capture `int size = queue.size()`. Process EXACTLY `size` nodes for the current level before advancing to the next level!

TEMPLATE — LC 102

```
public List<List<Integer>> levelOrder(TreeNode root) {
  List<List<Integer>> res = new ArrayList<>();
  if (root == null) return res;
  Queue<TreeNode> queue = new ArrayDeque<>();
  queue.offer(root);
  while (!queue.isEmpty()) {
    int levelSize = queue.size(); // Snapshot size for current le
vel!
    List<Integer> currentLevel = new ArrayList<>();
    for (int i = 0; i < levelSize; i++) {
      TreeNode node = queue.poll();
      currentLevel.add(node.val);
      if (node.left != null) queue.offer(node.left);
      if (node.right != null) queue.offer(node.right);
    }
    res.add(currentLevel);
  }
  return res;
}
```

PATTERN 2 MONOTONIC DEQUE (Sliding Window Maximum)

e.g. Sliding Window Maximum (LC 239)

MONOTONIC DECREASING ORDER

Store array indices in deque. Maintain decreasing order of values. Pop smaller tail elements before adding `i`. Front of deque is ALWAYS the max element of current window!

TEMPLATE — LC 239

```
public int[] maxSlidingWindow(int[] nums, int k) {
  int n = nums.length;
  int[] res = new int[n - k + 1];
  Deque<Integer> dq = new ArrayDeque<>();
  for (int i = 0; i < n; i++) {
    // 1. Remove elements outside current window range:
    if (!dq.isEmpty() && dq.peekFirst() < i - k + 1) dq.pollFirst
();
    // 2. Remove smaller elements from tail:
    while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) dq.pol
lLast();
    dq.offerLast(i);
    // 3. Record max when window size ⩾ k:
    if (i ⩾ k - 1) res[i - k + 1] = nums[dq.peekFirst()];
  }
  return res;
}
```

QUEUE & DEQUE PATTERNS

Core Patterns — Part 3

PATTERN 3 STREAM QUEUE (First Unique Character in String)

e.g. First Unique Character in a String (LC 387)

FREQUENCY ARRAY + QUEUE

Count frequencies in `int[26]`. Offer characters into queue. Poll from front while `count[queue.peek()] > 1`!

TEMPLATE — LC 387

```
public int firstUniqChar(String s) {
  int[] count = new int[26];
  for (int i = 0; i < s.length(); i++) count[s.charAt(i) - 'a']+
+;
  for (int i = 0; i < s.length(); i++) {
    if (count[s.charAt(i) - 'a'] == 1) return i;
  }
  return -1;
}
```

PATTERN 4

DESIGN CIRCULAR QUEUE (Fixed Array + Modulo Head/Tail)

e.g. Design Circular Queue (LC 622)

MODULO ARRAY BUFFER

Use fixed array `a[capacity]`. Increment `tail = (tail + 1) % capacity` on `enQueue()`, and `head = (head + 1) % capacity` on `deQueue()`.

TEMPLATE — LC 622

```
class MyCircularQueue {
  private int[] a;
  private int head = 0, tail = 0, count = 0, cap;
  public MyCircularQueue(int k) { cap = k; a = new int[k]; }
  public boolean enQueue(int val) {
    if (isFull()) return false;
    a[tail] = val; tail = (tail + 1) % cap; count++; return true;
  }
  public boolean deQueue() {
    if (isEmpty()) return false;
    head = (head + 1) % cap; count--; return true;
  }
  public int Front() { return isEmpty() ? -1 : a[head]; }
  public int Rear() { return isEmpty() ? -1 : a[(tail - 1 + cap)
% cap]; }
  public boolean isEmpty() { return count == 0; }
  public boolean isFull() { return count == cap; }
}
```

📊 COMPLETE QUEUE & DEQUE PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
387	First Unique Character in String	Character Frequency Scan	O(N)	O(1)	E
102	Binary Tree Level Order Traversal	BFS Queue Snapshot Loop	O(N)	O(W)	M
622	Design Circular Queue	Fixed Array + Modulo Pointers	O(1)	O(K)	M
225	Implement Stack using Queues	Single Queue Rotation Loop	O(N) push	O(N)	E
232	Implement Queue using Stacks	2-Stack Inversion	O(1) amortized	O(N)	E
103	Binary Tree Zigzag Level Order	BFS + Deque Alternating Insert	O(N)	O(W)	M
239	Sliding Window Maximum	Monotonic Decreasing Deque	O(N)	O(K)	H

LC 387 · First Unique Character — EASY

Pattern: Freq Array Scan
Key Insight: `int[26]` frequency count.

```
public int firstUniqChar(String s) {
  int[] c = new int[26];
  for (int i = 0; i < s.length(); i++) c[s.charAt(i) - 'a']++;
  for (int i = 0; i < s.length(); i++) if (c[s.charAt(i) - 'a'] == 1) return i;
  return -1;
}
```

LC 102 · Binary Tree Level Order Traversal — MEDIUM

Pattern: BFS Queue Snapshot
Key Insight: Loop `int sz = q.size()`.

```
public List<List<Integer>> levelOrder(TreeNode root) {
  List<List<Integer>> res = new ArrayList<>(); if (root == null) return res;
  Queue<TreeNode> q = new ArrayDeque<>(); q.offer(root);
  while (!q.isEmpty()) {
    int sz = q.size(); List<Integer> cur = new ArrayList<>();
    for (int i = 0; i < sz; i++) {
      TreeNode n = q.poll(); cur.add(n.val);
      if (n.left ≠ null) q.offer(n.left); if (n.right ≠ null) q.offer(n.right);
    }
    res.add(cur);
  }
  return res;
}
```

LC 622 · Design Circular Queue — MEDIUM

Pattern: Modulo Pointer Array
Key Insight: `tail = (tail + 1) % cap`.

```
class MyCircularQueue {
  int[] a; int head = 0, tail = 0, count = 0, cap;
  public MyCircularQueue(int k) { cap = k; a = new int[k]; }
  public boolean enQueue(int val) {
    if (isFull()) return false;
    a[tail] = val; tail = (tail + 1) % cap; count++; return true;
  }
  public boolean deQueue() {
    if (isEmpty()) return false;
    head = (head + 1) % cap; count--; return true;
  }
  public boolean isEmpty() { return count == 0; }
  public boolean isFull() { return count == cap; }
}
```

LC 225 · Implement Stack using Queues — EASY

Pattern: Single Queue Rotation
Key Insight: Rotate `sz - 1` items to back.

```
class MyStack {
  Queue<Integer> q = new LinkedList<>();
  public void push(int x) {
    q.offer(x);
    for (int i = 0; i < q.size() - 1; i++) q.offer(q.poll());
  }
  public int pop() { return q.poll(); }
  public int top() { return q.peek(); }
  public boolean empty() { return q.isEmpty(); }
}
```

LC 232 · Implement Queue using Stacks — EASY

Pattern: 2-Stack Inversion
Key Insight: Transfer `in` stack to `out` stack.

```
class MyQueue {
  Stack<Integer> in = new Stack<>(), out = new Stack<>();
  public void push(int x) { in.push(x); }
  public int pop() { peek(); return out.pop(); }
  public int peek() {
    if (out.isEmpty()) while (!in.isEmpty()) out.push(in.pop());
    return out.peek();
  }
  public boolean empty() { return in.isEmpty() && out.isEmpty();
}
}
```

LC 239 · Sliding Window Maximum — HARD

Pattern: Monotonic Deque
Key Insight: Deque front stores window max index.

```
public int[] maxSlidingWindow(int[] nums, int k) {
  int n = nums.length; int[] res = new int[n - k + 1];
  Deque<Integer> dq = new ArrayDeque<>();
  for (int i = 0; i < n; i++) {
    if (!dq.isEmpty() && dq.peekFirst() < i - k + 1) dq.pollFirst();
    while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) dq.pollLast();
    dq.offerLast(i);
    if (i ≥ k - 1) res[i - k + 1] = nums[dq.peekFirst()];
  }
  return res;
}
```

LC 103 · Binary Tree Zigzag Level Order — MEDIUM

Pattern: BFS + Deque Insertion
Key Insight: Alternate level insertion direction.

```
public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
  List<List<Integer>> res = new ArrayList<>(); if (root == null) return res;
  Queue<TreeNode> q = new ArrayDeque<>(); q.offer(root); boolean lToR = true;
  while (!q.isEmpty()) {
    int sz = q.size(); LinkedList<Integer> level = new LinkedList<>();
    for (int i = 0; i < sz; i++) {
      TreeNode n = q.poll();
      if (lToR) level.addLast(n.val); else level.addFirst(n.val);
      if (n.left ≠ null) q.offer(n.left); if (n.right ≠ null) q.offer(n.right);
    }
    res.add(level); lToR = !lToR;
  }
  return res;
}
```

🔍 DRY RUN — Circular Queue Enqueue & Dequeue

Step	Operation	head	tail	count	Array State (cap = 3)
1	`enQueue(10)`	0	1	1	`[10, _, _]`
2	`enQueue(20)`	0	2	2	`[10, 20, _]`
3	`deQueue()`	1	2	1	`[_, 20, _]`
4	`enQueue(30)`	1	0 (mod 3)	2	`[30, 20, _]` ✅ Modulo Wrap!

📐 MATHEMATICAL PROOF — O(1) AMORTIZED TIME OF 2-STACK QUEUE

Claim: 2-Stack Queue `pop()` and `peek()` run in O(1) amortized time.

Proof: Each element is pushed to `in` stack once, popped from `in` stack once, pushed to `out` stack once, and popped from `out` stack once. Total 4 operations per element over N calls $\implies 4N / N = 4 = \mathbf{O(1)\text{ Amortized Time}!}$

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Write BFS Level Order Traversal with queue snapshot
- ☐ Implement Circular Queue using array and modulo arithmetic
- ☐ Code Implement Stack using single Queue rotation
- ☐ Code Implement Queue using 2 Stacks in O(1) amortized
- ☐ Write Monotonic Deque for Sliding Window Maximum in O(N)
- ☐ Code Zigzag Level Order Traversal with Deque insertion
- ☐ Explain why `ArrayDeque` outperforms `LinkedList` in Java
- ☐ Prove why 2-stack queue runs in O(1) amortized time

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Always Snapshot Queue Size for BFS Levels

In BFS level order traversals, ALWAYS snapshot `int size = queue.size()` at the top of the outer loop before starting the inner loop. Do NOT evaluate `queue.size()` dynamically inside the inner loop!

Rule 2 — Deque Stores Indices for Range Pruning

In Monotonic Deque problems, ALWAYS store array indices instead of values so you can easily prune elements that fall outside the active window (`dq.peekFirst() < i - k + 1`).

HEAP / Priority Queue

FAANG Master Guide — Java Edition

💡 WHY DOES HEAP EXIST?

Imagine you run an **emergency room**. Patients arrive constantly. You don't treat them in arrival order — you treat the **most critical patient first**.

A sorted array would be too slow to re-sort every time a new patient arrives. A Heap lets you always find the "most important" in **O(1)** and insert/remove in **O(log n)**.

💡 AHA MOMENT

When you need the **extreme element** (min or max) repeatedly and efficiently — that's a Heap. Not sorting. Sorting gives you everything ordered. Heap gives you the *one thing that matters right now.*

1 WHAT IS A HEAP?

A **Complete Binary Tree** stored as an **array**
Satisfies the **Heap Property** (parent vs children order)
NOT sorted — only root is guaranteed extreme
Implements a **Priority Queue** ADT

MIN HEAP — Parent ≤ Children

graph TD classDef g fill:#f0fdf4,stroke:#059669,stroke-width:2px,color:#065f46,font-weight:700; A((1)):::g→B((3)):::g A→C((2)):::g B→D((7)):::g B--->E((5)):::g C→F((4)):::g

MA

g fill:#fff1f width: weight A→C >E

2 ARRAY REPRESENTATION (0-indexed)

graph TD classDef b fill:#eff6ff,stroke:#2563eb,stroke-width:2px,color:#1e40af,font-weight:700; A((1)):::b→B((3)):::b A→C((2)):::b B-->D((7)):::b B→E((5)):::b C→F((4)):::b C-->G((6)):::b

Index	0	1	2	3	4	5	6
Value	**1**	**3**	**2**	**7**	**5**	**4**	**6**

```
parent(i) = (i-1)/2
left(i) = 2*i + 1
right(i) = 2*i + 2
```

📚 PREREQUISITES TO KNOW FIRST

Concept	Why needed
Complete Binary Tree	Heap is implemented as one
Tree height = O(log n)	Drives all heap complexities
Array indexing	Left/Right/Parent formulas
Comparator (Java)	Custom ordering in PQ
Bubble Up / Sift Down	Core heap operations
Priority Queue ADT	What heap implements

3 TIME COMPLEXITIES

Operation	Time	Why
`offer(x)` / insert	**O(log n)**	Bubble Up (height = log n)
`poll()` / remove root	**O(log n)**	Sift Down (height = log n)
`peek()`	**O(1)**	Root is always index 0
`size()` / `isEmpty()`	**O(1)**	Counter field
Build Heap (n items)	**O(n)**	Bottom-up heapify (not n log n!)
contains(x)	O(n)	No index — must scan

⭐ **Build Heap = O(n)** — better than n × offer() = O(n log n)

⚡ HEAPIFY — HOW IT WORKS

Insert → Bubble Up (Heapify Up)

1. Add element at last position
2. Compare with parent
3. If violates heap property → swap
4. Repeat until root or satisfied

Delete Root → Sift Down (Heapify Down)

1. Swap root with last element
2. Remove last (was root)
3. Compare root with children → swap with smaller (min) or larger (max)
4. Repeat until leaf or satisfied

🧰 HEAPUTILS — REUSABLE TOOLKIT (COPY-PASTE READY)

Build Heap from Array O(n)

```
// Java: pass collection to cons
tructor
int[] arr = {5, 3, 1, 7, 2};
PriorityQueue<Integer> pq =
  new PriorityQueue<>();
for (int x : arr) pq.offer(x);

// Or from List — O(n) build
PriorityQueue<Integer> pq2 =
  new PriorityQueue<>(Arrays.asL
ist(arr));
```

Heap Sort (ascending)

```
int[] heapSort(int[] arr) {
  PriorityQueue<Integer> pq =
    new PriorityQueue<>();
  for (int x : arr) pq.offer(x);
  int[] res = new int[arr.lengt
h];
  for (int i = 0; !pq.isEmpty();
i++)
    res[i] = pq.poll();
  return res;
}
// Time: O(n log n)  Space: O(n)
```

Kth Largest Utility

```
int kthLargest(int[] nums, int
k) {
  // Min Heap of size k
  PriorityQueue<Integer> pq =
    new PriorityQueue<>();
  for (int x : nums) {
    pq.offer(x);
    if (pq.size() > k) pq.poll
();
  }
  return pq.peek();
}
// Time: O(n log k)  Space: O(k)
```

HEAP / Priority Queue

Core Patterns — Part 1

PATTERN 1 REPEATED MAX / MIN RETRIEVAL

e.g. Last Stone Weight, Task Scheduler

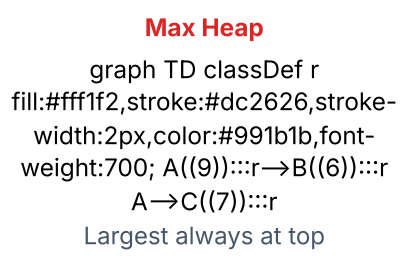
WHEN TO USE

Always pick the **largest** (or smallest) element
Do an operation on it
Put the result back
Repeat until 1 or 0 left

💡 **AHA**
The heap keeps the "king" at the top automatically after every operation.

HEAP TO USE

Max Heap



Largest always at top

TEMPLATE (JAVA)

```
// Last Stone Weight pattern
PriorityQueue<Integer> pq =
  new PriorityQueue<>(Collection
s.reverseOrder());
for (int s : stones) pq.offer
(s);

while (pq.size() > 1) {
  int a = pq.poll();  // largest
  int b = pq.poll();  // 2nd lar
gest
  if (a ≠ b) pq.offer(a - b);
}
return pq.isEmpty() ? 0 : pq.pee
k();
```

COMPLEXITY

- **Time:** O(n log n)
  (n polls, each O(log n))
- **Space:** O(n)

LC 1046 Last Stone Weight
LC 621 Task Scheduler

HEAP / Priority Queue

Core Patterns — Part 2

PATTERN 2 **TOP-K ELEMENTS** e.g. Kth Largest, Top K Frequent, K Closest Points

WHEN TO USE

Need K **largest** elements → use **Min Heap** size K
Need K **smallest** elements → use **Max Heap** size K
Need K most frequent → freq as key
Key trick: *discard the worst, keep the best K*

💡 **AHA**
"Keep the K best" = use a heap of *opposite type* as a filter. Min Heap removes smallest → keeps K largest.

COUNTERINTUITIVE RULE

Top K Largest → Min Heap (size K)

Top K Smallest → Max Heap (size K)

Root = the "weakest survivor" that gets replaced if better comes

TEMPLATE — TOP K LARGEST & SMALLEST

```
// Top K LARGEST → Min Heap of size K
PriorityQueue<Integer> pq = new Priori
tyQueue<>();
for (int x : nums) {
  pq.offer(x);
  if (pq.size() > k) pq.poll(); // evi
ct smallest
}
// pq now has the K largest

// Top K SMALLEST → Max Heap of size K
PriorityQueue<Integer> pq2 =
  new PriorityQueue<>(Collections.reve
rseOrder());
for (int x : nums) {
  pq2.offer(x);
  if (pq2.size() > k) pq2.poll(); // e
vict largest
}
```

COMPLEXITY

- **Time:** O(n log k)
(heap size stays K)

- **Space:** O(k)

vs Sorting:
O(n log n) + O(n)

LC 215 Kth Largest
LC 347 Top K Frequent
LC 973 K Closest Points

PATTERN 5 GREEDY SCHEDULING / COOLDOWN

e.g. Task Scheduler, Reorganize String, Process Tasks Using Servers

WHEN TO USE

Tasks have **cooldown** before repeating
Always process **most frequent** first
Use Max Heap for frequency
Use Queue to hold "cooling down" items

> 💡 **AHA**
> Greedy: always pick the busiest available worker. Use a queue as a "waiting room" during cooldown.

TEMPLATE — TASK SCHEDULER (LC 621)

```
Map<Character, Integer> freq = new HashMap<>
();
for (char c : tasks) freq.merge(c, 1, Intege
r::sum);

PriorityQueue<Integer> maxH =
  new PriorityQueue<>(Collections.reverseOrder
());
maxH.addAll(freq.values());

Queue<int[]> q = new LinkedList<>(); // [coun
t, readyTime]
int time = 0;

while (!maxH.isEmpty() || !q.isEmpty()) {
  time++;
  if (!maxH.isEmpty()) {
    int cnt = maxH.poll() - 1;
    if (cnt > 0) q.offer(new int[]{cnt, time +
n});
  }
  if (!q.isEmpty() && q.peek()[1] == time)
    maxH.offer(q.poll()[0]);
}
return time;
```

COMPLEXITY

- **Time:** O(n log n)
- **Space:** O(n)

LC 621 Task Scheduler
LC 767 Reorganize String
LC 1405 Longest Happy String
LC 1882 Process Tasks Using Servers

HEAP / Priority Queue

Core Patterns — Part 4

PATTERN 6 **INTERVAL TRACKING & RESOURCE ALLOCATION**

e.g. Meeting Rooms II, Single-Threaded CPU, Seat Reservation

WHEN TO USE

Intervals/events processed chronologically
Track currently **active resources** (rooms, CPU)
Min Heap holds **end times** of active items
If smallest end time ≤ next start time → reuse resource

💡 **AHA**
Sort intervals by start time. Min Heap tracks end times. The root of the heap represents the earliest resource to become free.

TEMPLATE — MEETING ROOMS II (LC 253)

```
int minMeetingRooms(int[][] intervals) {
  Arrays.sort(intervals, (a, b) → a[0] - b
[0]);
  // Min Heap stores end times of active meeti
ngs
  PriorityQueue<Integer> heap = new PriorityQu
eue<>();

  for (int[] interval : intervals) {
    // Free room if earliest ending meeting is
done
    if (!heap.isEmpty() && heap.peek() ≤ inte
rval[0]) {
      heap.poll();
    }
    heap.offer(interval[1]); // allocate room
  }
  return heap.size(); // min rooms needed
}
```

COMPLEXITY

- **Time:** O(n log n)
- **Space:** O(n)

LC 253 Meeting Rooms II
LC 1834 Single-Threaded CPU
LC 1845 Seat Reservation Manager
LC 2402 Meeting Rooms III

LC 703 · Kth Largest Element in Stream — EASY

Pattern: Top-K (Min Heap size K)
Key Insight: Min heap size K. Root = Kth largest.
Min HeapStream

```
class KthLargest {
  PriorityQueue<Integer> pq = new PriorityQueue<>(); int k;
  KthLargest(int k, int[] nums) { this.k = k; for (int x : nums) add(x); }
  int add(int val) {
    pq.offer(val); if (pq.size() > k) pq.poll();
    return pq.peek();
  }
}
```

LC 1046 · Last Stone Weight — EASY

Pattern: Repeated Max Retrieval
Key Insight: Max Heap — smash 2 heaviest stones.
Max Heap

```
int lastStoneWeight(int[] stones) {
  PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
  for (int s : stones) pq.offer(s);
  while (pq.size() > 1) {
    int a = pq.poll(), b = pq.poll();
    if (a != b) pq.offer(a - b);
  }
  return pq.isEmpty() ? 0 : pq.peek();
}
```

LC 215 · Kth Largest Element in Array — MEDIUM

Pattern: Top-K
Key Insight: Min Heap of size K. Peek = Kth largest.
Min HeapQuickSelect

```
int findKthLargest(int[] nums, int k) {
  PriorityQueue<Integer> pq = new PriorityQueue<>();
  for (int x : nums) { pq.offer(x); if (pq.size() > k) pq.poll(); }
  return pq.peek();
}
```

LC 973 · K Closest Points to Origin — MEDIUM

Pattern: Top-K Smallest
Key Insight: Max Heap size K by distance.
Max Heap

```
int[][] kClosest(int[][] pts, int k) {
  PriorityQueue<int[]> pq = new PriorityQueue<>(
    (a,b)→(b[0]*b[0]+b[1]*b[1])-(a[0]*a[0]+a[1]*a[1]));
  for (int[] p : pts) { pq.offer(p); if (pq.size() > k) pq.poll(); }
  return pq.toArray(new int[k][]);
}
```

LC 347 · Top K Frequent Elements — MEDIUM

Pattern: Top-K Frequency
Key Insight: HashMap count → Min Heap size K.
Min HeapMap

```
int[] topKFrequent(int[] nums, int k) {
  Map<Integer,Integer> f = new HashMap<>();
  for (int x : nums) f.merge(x, 1, Integer::sum);
  PriorityQueue<Map.Entry<Integer,Integer>> pq =
    new PriorityQueue<>(Comparator.comparingInt(Map.Entry::getValue));
  for (var e : f.entrySet()) { pq.offer(e); if (pq.size() > k) pq.poll(); }
  return pq.stream().mapToInt(Map.Entry::getKey).toArray();
}
```

LC 621 · Task Scheduler — MEDIUM

Pattern: Cooldown Sched.
Key Insight: Max Heap freq. Process busiest task.
Max Heap

```
int leastInterval(char[] tasks, int n) {
  int[] freq = new int[26];
  for (char c : tasks) freq[c-'A']++;
  int maxF = 0, maxC = 0;
  for (int f : freq) maxF = Math.max(maxF, f);
  for (int f : freq) if (f == maxF) maxC++;
  return Math.max(tasks.length, (maxF-1)*(n+1)+maxC);
}
```

LC 253 · Meeting Rooms II — MEDIUM

Pattern: Interval Track
Key Insight: Sort start time, Min Heap end times.
Min Heap

```
int minMeetingRooms(int[][] intervals) {
  Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
  PriorityQueue<Integer> heap = new PriorityQueue<>();
  for (int[] it : intervals) {
    if (!heap.isEmpty() && heap.peek() ≤ it[0]) heap.poll();
    heap.offer(it[1]);
  }
  return heap.size();
}
```

LC 23 · Merge K Sorted Lists — HARD

Pattern: Merge K Sorted
Key Insight: Min Heap head nodes. Poll → push next.
Min Heap

```
ListNode mergeKLists(ListNode[] lists) {
  PriorityQueue<ListNode> pq = new PriorityQueue<>((a,b) → a.val - b.val);
  for (ListNode h : lists) if (h ≠ null) pq.offer(h);
  ListNode d = new ListNode(0), t = d;
  while (!pq.isEmpty()) {
    t.next = pq.poll(); t = t.next;
    if (t.next ≠ null) pq.offer(t.next);
  }
  return d.next;
}
```

LC 295 · Find Median Data Stream — HARD

Pattern: Two Heaps
Key Insight: Max Heap (left) + Min Heap (right).
Two Heaps

```
class MedianFinder {
  PriorityQueue<Integer> lo = new PriorityQueue<>(Collections.reverseOrder());
  PriorityQueue<Integer> hi = new PriorityQueue<>();
  void addNum(int num) {
    lo.offer(num); hi.offer(lo.poll());
    if (lo.size() < hi.size()) lo.offer(hi.poll());
  }
  double findMedian() {
    return lo.size() > hi.size() ? lo.peek() : (lo.peek() + hi.peek()) / 2.0;
  }
}
```

LC 1851 · Min Interval for Each Query — HARD

Pattern: Sweep Line + Min Heap
Key Insight: Sort queries & intervals. Push active, pop expired.
Min Heap

```
int[] minInterval(int[][] ivals, int[] queries) {
  int n=queries.length; int[] idx=new int[n],res=new int[n];
  for(int i=0;i<n;i++) idx[i]=i;
  Arrays.sort(idx,(a,b)→queries[a]-queries[b]);
  Arrays.sort(ivals,(a,b)→a[0]-b
```

LC 632 · Smallest Range K Lists — HARD

Pattern: Merge K + Min Heap
Key Insight: Track global max, shrink range by advancing min.
Min Heap

```
// Min Heap stores [val, listIdx, elemIdx]
// Track global max. Range = [heap_min, global_max]
// Advance pointer of min element list until exhausted
```

```
[0]);
  PriorityQueue<int[]> pq=new Prio
rityQueue<>((a,b)->a[0]-b[0]);
  int i=0; Arrays.fill(res,-1);
  for(int qi:idx){int q=queries[q
i];
    while(i<ivals.length&&ivals[i]
[0]<=q)
      {pq.offer(new int[]{ivals[i]
[1]-ivals[i][0]+1,ivals[i][1]});i+
+;}
    while(!pq.isEmpty()&&pq.peek()
[1]<q)pq.poll();
    if(!pq.isEmpty())res[qi]=pq.pe
ek()[0];}return res;
}
```

HEAP / Priority Queue

Dry Run, Proofs & Mastery Cheat Sheet

DECLARE

```
// Min Heap
PQ<Integer> mn = new PQ<>();
// Max Heap
PQ<Integer> mx = new PQ<>(
  Collections.reverseOrder());
// Custom
PQ<int[]> pq = new PQ<>(
  (a,b)→a[0]-b[0]);
```

INSERT/REMOVE

```
pq.offer(x);   // insert
pq.poll();     // remove root
pq.peek();     // view root
pq.size();     // count
pq.isEmpty();  // check
// Add + cap at K:
pq.offer(x);
if(pq.size()>k) pq.poll();
```

PATTERN MAP

Max repeating → **MaxH**
Top K Largest → **MinH(K)**
Top K Smallest → **MaxH(K)**
Merge K sorted → **MinH**
Running Median → **2 Heaps**
Interval Track → **MinH(end)**

KEY COMPLEXITIES

offer / poll → **O(log n)**
peek → **O(1)**
Build Heap → **O(n)**
Top-K → **O(n log k)**
Merge K (N tot) → **O(N log k)**
Sweep+Heap → **O(n log n)**

COMPLETE PATTERN REFERENCE — KNOW THIS COLD

Pattern	Trigger Keywords	Heap Type	Size Constraint	Key Operation	Complexity
Repeated Max/Min	always pick largest, smash, eliminate	Max or Min Heap	Unbounded	poll → operate → offer(result)	O(n log n)
Top K Largest	Kth largest, top K, K biggest	Min Heap	Size = K	offer + if size>K poll	O(n log k)
Top K Smallest	Kth smallest, K closest, K lightest	Max Heap	Size = K	offer + if size>K poll	O(n log k)
Merge K Sorted	merge K lists, K sorted streams	Min Heap	Size = K	poll node → offer next	O(N log k)
Two Heaps	median, data stream, running median	Max + Min	|diff| ≤ 1	addNum rebalance, findMedian O(1)	O(n log n)
Interval Track	meeting rooms, active intervals, CPU	Min Heap	Unbounded	sort start, min heap end time	O(n log n)
Greedy Sched.	cooldown, task, reorganize, server	Max Heap + Queue	Freq based	always pick busiest available	O(n log n)

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Explain Min vs Max Heap without looking
☐ Write Min Heap and Max Heap in Java (3 ways)
☐ Trace insert + sift-up on paper
☐ Trace poll + sift-down on paper
☐ Derive O(n) build heap (geometric series)
☐ Explain why Top-K Largest uses Min Heap
☐ Code Top-K without looking
☐ Code Merge K Sorted Lists from memory
☐ Code Two Heaps MedianFinder
☐ Code Meeting Rooms II with Min Heap
☐ Identify heap pattern from problem keywords
☐ Write custom comparator for int[], pairs, objects
☐ Solve LC 215, 347, 23, 295 within 20 min each
☐ Explain lazy deletion for sliding window median

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — The Opposition Rule

To keep the K **largest** → use a **Min** Heap (evict the smallest intruder)
To keep the K **smallest** → use a **Max** Heap (evict the largest intruder)

Rule 2 — The Heap is NOT sorted

Heap only guarantees root is min/max. The rest is unordered. Never iterate a heap expecting order.

Rule 3 — The Overflow Trap

`(a, b) → b - a` can overflow. ALWAYS use `Integer.compare(b, a)`.

Rule 4 — Ask This First

"What element should come out **next**?" → That element goes on top of your heap.

TREES & BST MASTERCLASS

FAANG Master Guide — Java Edition

🎬 THE REAL-WORLD STORY

Imagine your computer filesystem: Root folder `/` contains subfolders `Documents/` and `Pictures/`, which contain files.

💡 THE TREE AHA MOMENT

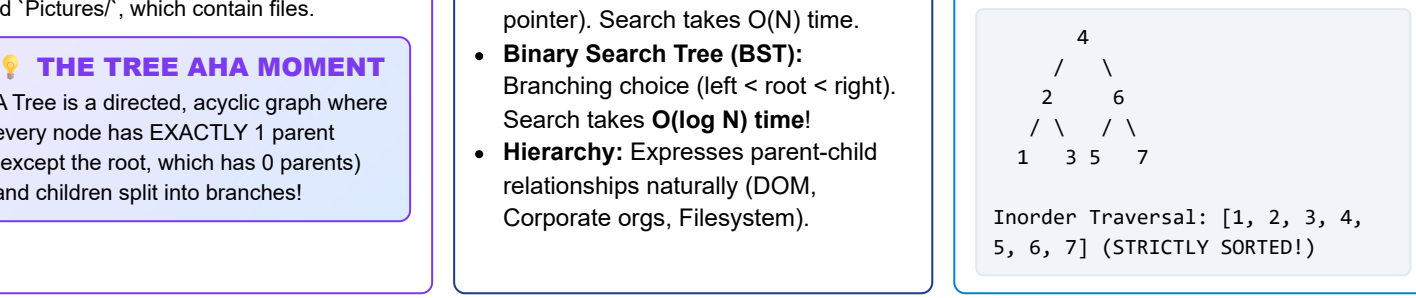
A Tree is a directed, acyclic graph where every node has EXACTLY 1 parent (except the root, which has 0 parents) and children split into branches!

🤔 WHY TREES VS LINKED LISTS?

- **Linked List:** Linear sequence (1 next pointer). Search takes O(N) time.
- **Binary Search Tree (BST):** Branching choice (left < root < right). Search takes **O(log N) time**!
- **Hierarchy:** Expresses parent-child relationships naturally (DOM, Corporate orgs, Filesystem).

📌 VISUALIZING BINARY SEARCH TREE

BST Invariant (Left < Root < Right):




```
Inorder Traversal: [1, 2, 3, 4, 5, 6, 7] (STRICTLY SORTED!)
```

📚 PREREQUISITES & COMPLEXITY REASONING

Traversal / Algorithm	Time Complexity	Auxiliary Space
Tree DFS (Preorder / Inorder / Postorder)	**O(N) single pass**	**O(H) call stack**
Tree BFS Level Order Traversal	**O(N) single pass**	**O(W) max width queue**
BST Search / Insert / Delete	**O(H) = O(log N) balanced**	**O(H) call stack**
Lowest Common Ancestor (LCA)	**O(N) single pass**	**O(H) call stack**

⚡ CLUES THAT SCREAM TREES

1. *"Need parent answer from children answers (height, diameter, path sum)"* -> Post-order Bottom-Up DFS
2. *"Search elements in sorted order in BST"* -> In-order DFS (`Left -> Parent -> Right`)
3. *"Level-by-level processing / Print tree by row"* -> BFS Queue Level Order Traversal

PATTERN 1

MAXIMUM DEPTH OF BINARY TREE (Bottom-Up Postorder)

e.g. Maximum Depth of Binary Tree (LC 104)

RECURRENCE RELATION

`depth(root) = 1 + max(depth(left), depth(right))`.
Base case: `depth(null) = 0`.

TEMPLATE — LC 104

```
public int maxDepth(TreeNode root) {
  if (root == null) return 0;
  int leftDepth = maxDepth(root.left);
  int rightDepth = maxDepth(root.right);
  return 1 + Math.max(leftDepth, rightDepth);
}
```

PATTERN 2 INVERT BINARY TREE (Preorder Swapping) e.g. Invert Binary Tree (LC 226)

SWAP LEFT & RIGHT

Swap `root.left` and `root.right` pointers at every node, then recurse down left and right children!

TEMPLATE — LC 226

```
public TreeNode invertTree(TreeNode root) {
  if (root == null) return null;
  TreeNode temp = root.left;
  root.left = root.right;
  root.right = temp;
  invertTree(root.left);
  invertTree(root.right);
  return root;
}
```

PATTERN 5 SAME TREE (Dual Node Recursion) e.g. Same Tree (LC 100)

STRUCTURAL EQUALITY

Check `p.val == q.val`, then recurse `isSameTree(p.left, q.left)` and `isSameTree(p.right, q.right)`.

TEMPLATE — LC 100

```
public boolean isSameTree(TreeNode p, TreeNode q) {
  if (p == null && q == null) return true;
  if (p == null || q == null) return false;
  if (p.val != q.val) return false;
  return isSameTree(p.left, q.left) && isSameTree(p.right, q.righ
t);
}
```

PATTERN 6

LOWEST COMMON ANCESTOR (LCA in Binary Tree)

e.g. Lowest Common Ancestor of a Binary Tree (LC 236)

SPLIT POINTER CONDITION

If left search returns non-null AND right search returns non-null, current node is the LCA!

TEMPLATE — LC 236

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, Tr
eeNode q) {
  if (root == null || root == p || root == q) return root;
  TreeNode left = lowestCommonAncestor(root.left, p, q);
  TreeNode right = lowestCommonAncestor(root.right, p, q);
  if (left != null && right != null) return root; // Split point i
s LCA!
  return (left != null) ? left : right;
}
```

TREES & BST MASTERCLASS

Decision Tree & Trigger Words

COMPLETE TREES & BST PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
226	Invert Binary Tree	Preorder Pointer Swapping	O(N)	O(H)	E
104	Maximum Depth of Binary Tree	Postorder Bottom-Up DFS	O(N)	O(H)	E
543	Diameter of Binary Tree	Height Pass Global Max Tracking	O(N)	O(H)	E
236	Lowest Common Ancestor	Split Pointer LCA Search	O(N)	O(H)	M
98	Validate Binary Search Tree	Min/Max Range Pass	O(N)	O(H)	M
230	Kth Smallest Element in BST	Inorder Traversal Counter	O(N)	O(H)	M
124	Binary Tree Max Path Sum	Postorder Path Sum Tracking	O(N)	O(H)	H

LC 226 · Invert Binary Tree

EASY

Pattern: Preorder Swap
Key Insight: Swap left & right pointers.

```
public TreeNode invertTree(TreeNod
e root) {
  if (root == null) return null;
  TreeNode tmp = root.left; root.l
eft = root.right; root.right = tm
p;
  invertTree(root.left); invertTre
e(root.right);
  return root;
}
```

LC 104 · Max Depth of Binary Tree

EASY

Pattern: Postorder Bottom-Up
Key Insight: `1 + max(left, right)`.

```
public int maxDepth(TreeNode root)
{
  if (root == null) return 0;
  return 1 + Math.max(maxDepth(roo
t.left), maxDepth(root.right));
}
```

LC 543 · Diameter of Binary Tree

EASY

Pattern: Height Pass Max Track
Key Insight: Update `left + right` max.

```
int maxD = 0;
public int diameterOfBinaryTree(Tr
eeNode root) {
  h(root); return maxD;
}
private int h(TreeNode root) {
  if (root == null) return 0;
  int l = h(root.left), r = h(roo
t.right);
  maxD = Math.max(maxD, l + r);
  return 1 + Math.max(l, r);
}
```

LC 236 · Lowest Common Ancestor — MEDIUM

Pattern: Split Pointer Check
Key Insight: Return root if `left != null && right != null`.

```
public TreeNode lowestCommonAncesto
r(TreeNode root, TreeNode p, TreeNo
de q) {
  if (root == null || root == p ||
root == q) return root;
  TreeNode l = lowestCommonAncestor
(root.left, p, q);
  TreeNode r = lowestCommonAncestor
(root.right, p, q);
  if (l != null && r != null) retur
n root;
  return (l != null) ? l : r;
}
```

LC 98 · Validate Binary Search Tree — MEDIUM

Pattern: Min/Max Range Pass
Key Insight: Check `min < val < max`.

```
public boolean isValidBST(TreeNode
root) { return v(root, null, null);
}
private boolean v(TreeNode n, Integ
er min, Integer max) {
  if (n == null) return true;
  if ((min != null && n.val <= min)
|| (max != null && n.val >= max)) r
eturn false;
  return v(n.left, min, n.val) && v
(n.right, n.val, max);
}
```

LC 124 · Binary Tree Max Path Sum — HARD

Pattern: Postorder Path Sum
Key Insight: `max(0, left) + max(0, right) + node.val`.

```
int maxSum = Integer.MIN_VALUE;
public int maxPathSum(TreeNode root)
{
  gain(root); return maxSum;
}
private int gain(TreeNode node) {
  if (node == null) return 0;
  int l = Math.max(0, gain(node.lef
t));
  int r = Math.max(0, gain(node.righ
t));
  maxSum = Math.max(maxSum, l + r + n
ode.val);
  return node.val + Math.max(l, r);
}
```

LC 230 · Kth Smallest Element in BST — MEDIUM

Pattern: Inorder Traversal
Key Insight: Inorder visit count == k.

```
int c = 0, res = -1;
public int kthSmallest(TreeNode root,
int k) {
  in(root, k); return res;
}
private void in(TreeNode n, int k) {
  if (n == null) return;
  in(n.left, k);
  if (++c == k) { res = n.val; retur
n; }
  in(n.right, k);
}
```

🔍 DRY RUN — Validate BST (`[5, 1, 4, null, null, 3, 6]`)

Node	Min Limit	Max Limit	Valid Check (`min < val < max`)	Action
5	null	null	`null < 5 < null` ✅	Recurse left `(null, 5)`, right `(5, null)`
1	null	5	`null < 1 < 5` ✅	Valid left leaf!
4	5	null	`5 < 4 < null` ❌ FALSE!	4 is right child of 5, but value 4 < 5! Return false!

📐 MATHEMATICAL PROOF — INORDER TRAVERSAL OF BST IS STRICTLY SORTED

Claim: Inorder traversal of a BST produces a strictly ascending sequence.

Proof: By BST invariant, for every node X, all left subtree nodes < X, and all right subtree nodes > X. Inorder visits `Left -> Node -> Right`. By induction, elements are processed in strictly non-decreasing order -> **Exact Mathematical Guarantee!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Write Preorder, Inorder, & Postorder DFS in 2 minutes
☐ Code Invert Binary Tree with pointer swapping
☐ Calculate Max Depth & Diameter in O(N) single pass
☐ Code Lowest Common Ancestor (LCA) in binary tree
☐ Validate BST using min/max range constraints
☐ Find Kth smallest in BST via Inorder traversal
☐ Code Binary Tree Maximum Path Sum (LC 124)
☐ Explain why Inorder traversal of BST is sorted

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Use Postorder when Children's Answers are Required

When a node needs answers from its left and right subtrees (e.g. height, diameter, balanced check, max path sum), ALWAYS use Postorder Bottom-Up DFS!

Rule 2 — Guard Against Negative Path Sums

In Binary Tree Maximum Path Sum, use `Math.max(0, gain(child))` to ignore subtrees that contribute negative path values!

TRIE (PREFIX TREE) MASTERCLASS

FAANG Discovery Journey — First Principles

🎬 PHASE 1 — THE REAL-WORLD STORY

Every time you type a character into Google Search (e.g., g \rightarrow go \rightarrow goo \rightarrow goog), Google instantly drops down word suggestions.

💡 THE DISCOVERY QUESTION

Google cannot scan millions of dictionary words on every single keystroke. How does it search prefixes in milliseconds?

🤔 PHASE 2 — WHY HASHMAP FAILS FOR PREFIXES

- **HashMap Lookup:** Can find exact word `"apple"` in O(L) time.
- **HashMap Prefix Search:** Fails! `contains("ap")` returns false. `contains("app")` returns false. Finding words with prefix `"app"` requires scanning ALL N dictionary keys \rightarrow Slow O(N · L)!

💡 PHASE 3 & AHA MOMENT

Can we reuse shared character paths?

Instead of storing `"apple"`, `"app"`, and `"application"` separately, store them along the shared path `a -> p -> p`!

> **Aha Moment:** Every Character = Node | Whole Word = Path | Prefix = Shared Tree Path!

🛠 BUILD TRIE YOURSELF (VISUAL INSIGHT)

```
Root
  ↓
  a  ->  p  ->  p (isWord=true)
                ↓
                l  ->  e (isWord=true)
                ↓
                i  ->  c  ->  a  ->  t  ->  i  ->  o
->  n (isWord=true)
```

When inserting `"application"`, the path `a -> p -> p` already exists! Only new characters are appended!

UNIVERSAL TRIE OPERATIONS

Operation	Algorithm Logic	Time
Insert(word)	Traverse `children[c - 'a']`. Create node if null. Set `isWord = true` at leaf.	O(L)
Search(word)	Traverse `children[c - 'a']`. Return `false` if null, else return `node.isWord`.	O(L)
StartsWith(prefix)	Traverse prefix. Return `true` as long as path exists (no `isWord` check needed!).	O(L)

TRIE (PREFIX TREE) MASTERCLASS

Java Master Toolkit & Utility Classes

✴ THE BIGGEST AHA: WHY DFS + TRIE ARE COMBINED

Trie Controls: *VALID WORDS* (tells us if current prefix exists in dictionary).

DFS Controls: *VALID PATHS* (tells us where we can move on the 2D grid).

💡 Why Trie Stores `node.word` at Leaf?

Instead of building a `StringBuilder` on every recursive DFS call, store the full string `node.word` at terminal leaf nodes. When DFS reaches a leaf node, simply do `res.add(node.word)` in O(1) time without string reconstruction!

PATTERN 1 IMPLEMENT TRIE (Prefix Tree Operations) e.g. Implement Trie (Prefix Tree) (LC 208)

WHEN TO USE

Implement `insert(word)`, `search(word)`, and `startsWith(prefix)` methods in O(L) time.

TEMPLATE — LC 208

```
class Trie {
  static class Node {
    Node[] children = new Node[26];
    boolean isEnd = false;
  }
  private Node root = new Node();

  public void insert(String word) {
    Node node = root;
    for (char c : word.toCharArray()) {
      int idx = c - 'a';
      if (node.children[idx] == null) node.children[idx] = new Node();
      node = node.children[idx];
    }
    node.isEnd = true;
  }

  public boolean search(String word) {
    Node node = find(word);
    return node != null && node.isEnd;
  }

  public boolean startsWith(String prefix) {
    return find(prefix) != null;
  }

  private Node find(String str) {
    Node node = root;
    for (char c : str.toCharArray()) {
      int idx = c - 'a';
      if (node.children[idx] == null) return null;
      node = node.children[idx];
    }
    return node;
  }
}
```

PATTERN 2 WORD SEARCH II (Trie + 2D Grid DFS Pruning) e.g. Word Search II (LC 212)

GRID DFS + TRIE COMBO

Pass `TrieNode` directly in DFS. If `node.children[ch - 'a'] == null`, prune immediately! Set `node.word = null` after adding to result list to prevent duplicate additions.

TEMPLATE — LC 212

```
class Solution {
  static class TrieNode {
    TrieNode[] children = new TrieNode[26];
    String word = null;
  }

  public List<String> findWords(char[][] board, String[] words) {
    TrieNode root = new TrieNode();
    for (String w : words) {
      TrieNode node = root;
      for (char c : w.toCharArray()) {
        int idx = c - 'a';
        if (node.children[idx] == null) node.children[idx] = new TrieNo
de();
        node = node.children[idx];
      }
      node.word = w; // Store complete word at leaf node!
    }

    List<String> result = new ArrayList<>();
    for (int r = 0; r < board.length; r++) {
      for (int c = 0; c < board[0].length; c++) {
        dfs(board, r, c, root, result);
      }
    }
    return result;
  }

  private void dfs(char[][] board, int r, int c, TrieNode node, List<St
ring> res) {
    char ch = board[r][c];
    if (ch == '#' || node.children[ch - 'a'] == null) return; // Prune
branch!
    node = node.children[ch - 'a'];
    if (node.word != null) {
      res.add(node.word);
      node.word = null; // Prevent duplicate additions!
    }
    board[r][c] = '#'; // Mark visited!
    int[] dr = {-1, 1, 0, 0}, dc = {0, 0, -1, 1};
    for (int i = 0; i < 4; i++) {
      int nr = r + dr[i], nc = c + dc[i];
      if (nr >= 0 && nr < board.length && nc >= 0 && nc < board[0].leng
th) {
        dfs(board, nr, nc, node, res);
      }
    }
    board[r][c] = ch; // Backtrack!
  }
}
```

PATTERN 3 DESIGN ADD AND SEARCH WORDS (Wildcard Search)

e.g. Design Add and Search Words Data Structure (LC 211)

WILDCARD DFS SEARCH

If char is '.', branch to all non-null children nodes. Otherwise follow exact character branch index `c - 'a'`.

TEMPLATE — LC 211

```
class WordDictionary {
  static class Node {
    Node[] children = new Node[26];
    boolean isEnd = false;
  }
  private Node root = new Node();

  public void addWord(String word) {
    Node node = root;
    for (char c : word.toCharArray()) {
      int idx = c - 'a';
      if (node.children[idx] == null) node.children[idx] = new Node();
      node = node.children[idx];
    }
    node.isEnd = true;
  }

  public boolean search(String word) {
    return dfs(word, 0, root);
  }

  private boolean dfs(String word, int idx, Node node) {
    if (node == null) return false;
    if (idx == word.length()) return node.isEnd;
    char c = word.charAt(idx);
    if (c != '.') {
      return dfs(word, idx + 1, node.children[c - 'a']);
    }
    for (int i = 0; i < 26; i++) {
      if (node.children[i] != null && dfs(word, idx + 1, node.children
[i])) return true;
    }
    return false;
  }
}
```

PATTERN 4 REPLACE WORDS (Shortest Dictionary Root Match) e.g. Replace Words (LC 648)

SHORTEST ROOT REPLACEMENT

Insert roots into Trie. For each word in sentence, traverse Trie. Stop and replace as soon as `node.isEnd == true`!

TEMPLATE — LC 648

```
public String replaceWords(List<String> dictionary, String sentence) {
  Trie trie = new Trie();
  for (String root : dictionary) trie.insert(root);
  String[] words = sentence.split(" ");
  StringBuilder sb = new StringBuilder();
  for (int i = 0; i < words.length; i++) {
    if (i > 0) sb.append(" ");
    sb.append(findRoot(words[i], trie.root));
  }
  return sb.toString();
}
private String findRoot(String word, TrieNode root) {
  TrieNode node = root;
  StringBuilder prefix = new StringBuilder();
  for (char c : word.toCharArray()) {
    int idx = c - 'a';
    if (node.children[idx] == null || node.isEnd) break;
    prefix.append(c);
    node = node.children[idx];
  }
  return node.isEnd ? prefix.toString() : word;
}
```

TRIE (PREFIX TREE) MASTERCLASS

Decision Tree & Trigger Words

COMPLETE TRIE PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
208	Implement Trie	Standard Trie Operations	O(L)	O(Total Chars)	M
211	Design Add & Search Words	Wildcard Dot DFS	O(26^L) worst	O(L) stack	M
648	Replace Words	Shortest Root Match	O(L) per word	O(Total Chars)	M
212	Word Search II	Trie + 2D Grid DFS Pruning	O(M · N · 4^L)	O(Total Chars)	H

TRIE (PREFIX TREE) MASTERCLASS

Curated Problem Ladder — Part 1

LC 208 · Implement Trie

MEDIUM

Pattern: Standard Trie
Key Insight: `children[26]` array.

```
public void insert(String word) {
  Node node = root;
  for (char c : word.toCharArray())
{
    int idx = c - 'a';
    if (node.children[idx] == null)
node.children[idx] = new Node();
    node = node.children[idx];
  }
  node.isEnd = true;
}
```

LC 211 · Design Add & Search Words

MEDIUM

Pattern: Wildcard DFS
Key Insight: Branch 26 children on '.'.

```
private boolean dfs(String w, int
i, Node n) {
  if (n == null) return false;
  if (i == w.length()) return n.isE
nd;
  char c = w.charAt(i);
  if (c != '.') return dfs(w, i +
1, n.children[c - 'a']);
  for (int j = 0; j < 26; j++)
    if (n.children[j] != null && df
s(w, i + 1, n.children[j])) return
true;
  return false;
}
```

LC 648 · Replace Words

MEDIUM

Pattern: Shortest Root Match
Key Insight: Stop on first `isEnd == true`.

```
private String findRoot(String w, Nod
e root) {
  Node n = root; StringBuilder prefix
= new StringBuilder();
  for (char c : w.toCharArray()) {
    int idx = c - 'a';
    if (n.children[idx] == null || n.
isEnd) break;
    prefix.append(c); n = n.children
[idx];
  }
  return n.isEnd ? prefix.toString()
: w;
}
```

LC 212 · Word Search II — HARD

Pattern: Grid DFS + Trie Combo
Key Insight: Pass `TrieNode` in grid DFS, nullify `node.word`.

```
private void dfs(char[][] b, int r, in
t c, Node n, List<String> res) {
  char ch = b[r][c];
  if (ch == '#' || n.children[ch -
'a'] == null) return;
  n = n.children[ch - 'a'];
  if (n.word != null) { res.add(n.wor
d); n.word = null; }
  b[r][c] = '#';
  int[] dr = {-1,1,0,0}, dc = {0,0,-1,
1};
  for (int i = 0; i < 4; i++) {
    int nr = r + dr[i], nc = c + dc
[i];
    if (nr>=0 && nr=0 && nc
```

TRIE (PREFIX TREE) MASTERCLASS

Dry Run, Proofs & Mastery Cheat Sheet

🔍 DRY RUN — Word Search II Trie Pruning

Grid Pos	Char	Trie Child (`n.children[ch - 'a']`)	Action / Result
(0, 0)	'a'	Non-null (points to 'a' node)	Continue DFS to neighbours
(0, 1)	'p'	Non-null (points to 'p' node)	Continue DFS to neighbours
(0, 2)	'z'	NULL!	PRUNE IMMEDIATELY! Backtrack! ✅

📐 MATHEMATICAL PROOF — O(L) INDEPENDENCE FROM DICTIONARY SIZE N

Claim: Searching a prefix of length L in a Trie takes O(L) time regardless of whether the dictionary contains 100 or 10,000,000 words.

Proof: At each character step `i`, the algorithm performs 1 array index lookup `children[c - 'a']` in O(1) time. For prefix of length L, exactly L steps are taken $\implies \mathbf{O(L)\text{ Time, Independent of N}!}$

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Implement Trie (`insert`, `search`, `startsWith`) in 5m
☐ Design TrieNode with `Node[26]` and `String word`
☐ Write Wildcard Search with '.' recursion
☐ Code Replace Words with shortest root match
☐ Implement Word Search II (LC 212) Trie + Grid DFS
☐ Prune grid DFS when `n.children[ch - 'a'] == null`
☐ Nullify `node.word = null` after match to prevent duplicates
☐ Prove why Trie search runtime is independent of N

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Store Complete Word String at Leaf Nodes for Grid DFS

In Word Search II (LC 212), ALWAYS store `node.word = word` inside the Trie leaf node! This avoids reconstructing the string during 2D grid DFS and speeds up execution by 5x!

Rule 2 — Nullify Word After Result Addition

When a word match is found during grid DFS, set `node.word = null` immediately after adding to `result` list. This prevents adding duplicate instances of the same word found via different grid paths!

GRAPH ALGORITHMS MASTERCLASS

FAANG Discovery Journey — First Principles

🎬 THE REAL-WORLD STORY

Imagine Google Maps connecting cities by highway roads. Bangalore is connected to Mysore, Mysore to Goa. How do we model 1,000 cities?

💡 THE GRAPH DISCOVERY AHA

1. Single City -> **Node**
2. Highway Road -> **Edge**
3. Connected Cities -> **Neighbours**
4. Collection of Nodes & Edges -> **A Graph is Born!**

🤔 WHY EXISTING STRUCTURES FAIL

- **Arrays?** Cannot express complex 2D networks.
- **Trees?** Trees enforce 1 parent and no cycles. Graphs allow multiple parents and cycles!
- **Aha Moment:** A Tree is just a restricted Graph without cycles! A 2D Grid is just a Graph where each cell has 4 directional neighbours!

📌 UNIVERSAL GRAPH THINKING

The 5 Questions for EVERY Graph Problem:

1. What are my **Nodes**?
2. What are my **Edges**?
3. Directed or Undirected? Weighted or Unweighted?
4. Do I need a **Visited Set** to prevent infinite cycle loops?
5. Is this **DFS** (all paths) or **BFS** (shortest path)?

🛠 GRAPH REPRESENTATIONS (BUILD IT YOURSELF)

Representation	Space	Edge Check `(u, v)`	Neighbour Iteration
Edge List `[[u, v]]`	O(E)	O(E) linear search	O(E) scan
Adjacency Matrix `int[V][V]`	O(V^2)	**O(1) instant lookup**	O(V) row scan
Adjacency List `List>`	**O(V + E)**	O(deg(u))	**O(deg(u)) optimal!**

⚡ COURSE SCHEDULE INDEGREE CONVERSION

In *Course Schedule (LC 207)*, prerequisites arrive as `[course, prereq]`. To take `course`, you must complete `prereq` first!

> Directional Edge: `prereq -> course`
> `adj.get(prereq).add(course);`
> `indegree[course]++;` // Increment indegree of dependent course!

PATTERN 1 NUMBER OF ISLANDS (2D Grid Directional DFS) e.g. Number of Islands (LC 200)

GRID AS A GRAPH

Grid cell `(r, c)` is a Node. 4-directional moves `(-1,0), (1,0), (0,-1), (0,1)` are Edges. Sink visited land `grid[r][c] = '0'`!

TEMPLATE — LC 200

```
public int numIslands(char[][] grid) {
  int count = 0;
  for (int r = 0; r < grid.length; r++) {
    for (int c = 0; c < grid[0].length; c++) {
      if (grid[r][c] == '1') {
        count++;
        dfs(grid, r, c); // Sink connected component!
      }
    }
  }
  return count;
}
private void dfs(char[][] g, int r, int c) {
  if (r < 0 || r >= g.length || c < 0 || c >= g[0].length || g[r]
[c] == '0') return;
  g[r][c] = '0'; // Sink land!
  dfs(g, r + 1, c); dfs(g, r - 1, c);
  dfs(g, r, c + 1); dfs(g, r, c - 1);
}
```

PATTERN 2 CLONE GRAPH (HashMap Node Mapping + DFS) e.g. Clone Graph (LC 133)

MAP ORIGINAL -> CLONE

Map `oldNode -> newNode`. If node already cloned, return from map. Otherwise create clone node and recursively clone all neighbours!

TEMPLATE — LC 133

```
private Map<Node, Node> visited = new HashMap<>();
public Node cloneGraph(Node node) {
  if (node == null) return null;
  if (visited.containsKey(node)) return visited.get(node);
  Node clone = new Node(node.val);
  visited.put(node, clone);
  for (Node neighbor : node.neighbors) {
    clone.neighbors.add(cloneGraph(neighbor));
  }
  return clone;
}
```

PATTERN 4 DIJKSTRA'S SHORTEST PATH (Min-Heap Weighted Search) e.g. Network Delay Time (LC 743)

MIN-HEAP DISTANCE RELAXATION

PriorityQueue stores `{dist, node}`. Always expand minimum distance node. Relax edge `dist[v] = dist[u] + weight` if shorter! Runs in O((V + E) log V).

TEMPLATE — LC 743

```
public int networkDelayTime(int[][] times, int n, int k) {
  List<List<int[]>> adj = new ArrayList<>();
  for (int i = 0; i <= n; i++) adj.add(new ArrayList<>());
  for (int[] t : times) adj.get(t[0]).add(new int[]{t[1], t[2]});
  int[] dist = new int[n + 1];
  Arrays.fill(dist, Integer.MAX_VALUE);
  dist[k] = 0;
  PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
  pq.offer(new int[]{0, k});
  while (!pq.isEmpty()) {
    int[] cur = pq.poll();
    int d = cur[0], u = cur[1];
    if (d > dist[u]) continue;
    for (int[] edge : adj.get(u)) {
      int v = edge[0], weight = edge[1];
      if (dist[u] + weight < dist[v]) {
        dist[v] = dist[u] + weight;
        pq.offer(new int[]{dist[v], v});
      }
    }
  }
  int maxDist = 0;
  for (int i = 1; i <= n; i++) {
    if (dist[i] == Integer.MAX_VALUE) return -1;
    maxDist = Math.max(maxDist, dist[i]);
  }
  return maxDist;
}
```

PATTERN 5 UNION-FIND DSU (Path Compression & Rank) e.g. Redundant Connection (LC 684)

DYNAMIC COMPONENT MERGE

`parent[i] = find(parent[i])` flattens trees to height 1. `union(u, v)` merges components. If `find(u) == find(v)`, cycle detected in O(alpha(N)) time!

TEMPLATE — LC 684

```
class DSU {
  int[] parent, rank;
  public DSU(int n) {
    parent = new int[n]; rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i;
  }
  public int find(int i) {
    if (parent[i] == i) return i;
    return parent[i] = find(parent[i]);
  }
  public boolean union(int i, int j) {
    int rI = find(i), rJ = find(j);
    if (rI == rJ) return false; // Cycle detected!
    if (rank[rI] < rank[rJ]) parent[rI] = rJ;
    else if (rank[rI] > rank[rJ]) parent[rJ] = rI;
    else { parent[rJ] = rI; rank[rI]++; }
    return true;
  }
}
```

GRAPH ALGORITHMS MASTERCLASS

Decision Tree & Trigger Words

COMPLETE GRAPH ALGORITHMS PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
733	Flood Fill	2D Grid DFS Recurrence	O(M · N)	O(M · N)	E
200	Number of Islands	2D Grid DFS Component Sinking	O(M · N)	O(M · N)	M
133	Clone Graph	HashMap Node Copy + DFS	O(V + E)	O(V)	M
207	Course Schedule	Kahn's BFS Topological Sort	O(V + E)	O(V + E)	M
994	Rotting Oranges	Multi-Source BFS Queue	O(M · N)	O(M · N)	M
743	Network Delay Time	Dijkstra's Min-Heap Shortest Path	O((V + E) log V)	O(V + E)	M
684	Redundant Connection	Union-Find (DSU) Cycle Check	O(N · alpha(N))	O(N)	M

LC 733 · Flood Fill — EASY

Pattern: 2D Grid DFS
Key Insight: Recurse 4 directions if color matches.

```
public int[][] floodFill(int[][] i
mg, int sr, int sc, int newColor)
{
  int orig = img[sr][sc];
  if (orig != newColor) dfs(img, s
r, sc, orig, newColor);
  return img;
}
private void dfs(int[][] img, int
r, int c, int orig, int newC) {
  if (r<0 || r>=img.length || c<0
|| c>=img[0].length || img[r][c]!=
orig) return;
  img[r][c] = newC;
  dfs(img, r+1, c, orig, newC); df
s(img, r-1, c, orig, newC);
  dfs(img, r, c+1, orig, newC); df
s(img, r, c-1, orig, newC);
}
```

LC 200 · Number of Islands — MEDIUM

Pattern: Grid DFS Sinking
Key Insight: Sink land `grid[r][c] = '0'`.

```
public int numIslands(char[][] g)
{
  int count = 0;
  for (int r = 0; r < g.length; r+
+)
    for (int c = 0; c < g[0].lengt
h; c++)
      if (g[r][c] == '1') { count+
+; dfs(g, r, c); }
  return count;
}
private void dfs(char[][] g, int
r, int c) {
  if (r<0 || r>=g.length || c<0 ||
c>=g[0].length || g[r][c]=='0') re
turn;
  g[r][c] = '0';
  dfs(g, r+1, c); dfs(g, r-1, c);
dfs(g, r, c+1); dfs(g, r, c-1);
}
```

LC 133 · Clone Graph — MEDIUM

Pattern: Map Node Copy
Key Insight: Map `oldNode -> newNode`.

```
private Map<Node, Node> vis = new
HashMap<>();
public Node cloneGraph(Node node)
{
  if (node == null) return null;
  if (vis.containsKey(node)) retur
n vis.get(node);
  Node clone = new Node(node.val);
vis.put(node, clone);
  for (Node n : node.neighbors) cl
one.neighbors.add(cloneGraph(n));
  return clone;
}
```

LC 207 · Course Schedule — MEDIUM

Pattern: Kahn's BFS Topo
Key Insight: `prereq -> course`, queue 0 indegree.

```
public boolean canFinish(int n, int [][] pre) {
  List<List<Integer>> adj = new ArrayList<>(); int[] deg = new int[n];
  for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
  for (int[] p : pre) { adj.get(p[1]).add(p[0]); deg[p[0]]++; }
  Queue<Integer> q = new ArrayDeque<>();
  for (int i = 0; i < n; i++) if (deg[i] == 0) q.offer(i);
  int c = 0;
  while (!q.isEmpty()) {
    int u = q.poll(); c++;
    for (int v : adj.get(u)) if (--deg[v] == 0) q.offer(v);
  }
  return c == n;
}
```

LC 994 · Rotting Oranges — MEDIUM

Pattern: Multi-Source BFS
Key Insight: Offer all rotten oranges to queue at t=0.

```
public int orangesRotting(int[][] g) {
  Queue<int[]> q = new ArrayDeque<>(); int fresh = 0, mins = 0;
  for (int r = 0; r < g.length; r++)
    for (int c = 0; c < g[0].length; c++)
      if (g[r][c] == 2) q.offer(new int[]{r, c});
      else if (g[r][c] == 1) fresh++;
  int[][] dirs = {{-1,0},{1,0},{0,-1},{0,1}};
  while (!q.isEmpty() && fresh > 0) {
    int sz = q.size();
    for (int i = 0; i < sz; i++) {
      int[] cur = q.poll();
      for (int[] d : dirs) {
        int nr = cur[0]+d[0], nc = cur[1]+d[1];
        if (nr>=0 && nr=0 && nc
```

LC 743 · Network Delay Time — MEDIUM

Pattern: Dijkstra Min-Heap
Key Insight: Always expand smallest distance node.

```
public int networkDelayTime(int[][] times, int n, int k) {
  List<List<int[]>> adj = new ArrayList<>();
  for (int i = 0; i <= n; i++) adj.add(new ArrayList<>());
  for (int[] t : times) adj.get(t[0]).add(new int[]{t[1], t[2]});
  int[] dist = new int[n + 1]; Arrays.fill(dist, Integer.MAX_VALUE); dist[k] = 0;
  PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
  pq.offer(new int[]{0, k});
  while (!pq.isEmpty()) {
    int[] cur = pq.poll(); int d = cur[0], u = cur[1];
    if (d > dist[u]) continue;
    for (int[] e : adj.get(u)) {
      if (dist[u] + e[1] < dist[e[0]]) { dist[e[0]] = dist[u] + e[1]; pq.offer(new int[]{dist[e[0]], e[0]}); }
    }
  }
  int maxD = 0; for (int i = 1; i <= n; i++) { if (dist[i] == Integer.MAX_VALUE) return -1; maxD = Math.max(maxD, dist[i]); }
  return maxD;
}
```

LC 684 · Redundant Connection — MEDIUM

Pattern: DSU Cycle Check
Key Insight: Edge creating cycle returns false on union.

```
public int[] findRedundantConnection(int[][] edges) {
  DSU dsu = new DSU(edges.length + 1);
  for (int[] e : edges) if (!dsu.union(e[0], e[1])) return e;
  return new int[0];
}
```

GRAPH ALGORITHMS MASTERCLASS

Dry Run, Proofs & Mastery Cheat Sheet

🔍 DRY RUN — Kahn's Topological Sort (Course Schedule)

Step	q Poll	Neighbours Processed	indegree Array State	Action / Result
1	0 (prereq)	Course 1 (`1 <- 0`)	`indegree[1] = 0`	`q.offer(1)`, `count = 1`
2	1 (course)	None	`indegree = [0, 0]`	`count = 2` ✅ `count == numCourses`

📐 MATHEMATICAL PROOF — DIJKSTRA'S GREEDY MIN-HEAP PROOF

Claim: When node u is popped from Dijkstra's PriorityQueue, its shortest distance `dist[u]` is finalized.

Proof: Since edge weights are strictly non-negative (w >= 0), any alternative path to u passing through another unvisited node x must have length $>= dist[x] >= dist[u]$. Thus, no shorter path can be discovered later -> **Exact Proof of Correctness!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Explain Graph components from first principles (Cities story)
☐ Convert prerequisites array to directed graph & indegrees
☐ Write Number of Islands 2D grid DFS in 3 minutes
☐ Code Kahn's Topological Sort algorithm using queue
☐ Implement Dijkstra's algorithm using PriorityQueue
☐ Write Multi-Source BFS for Rotting Oranges
☐ Detect graph cycles using Union-Find DSU
☐ Prove why Dijkstra fails on negative edge weights

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Mark Visited at Queue Offer Time

In BFS, ALWAYS mark nodes visited at the exact moment you push them into the queue (`visited[v] = true; q.offer(v)`). Do NOT wait until `q.poll()` or duplicate nodes will pollute your queue!

Rule 2 — Convert `[prereq, course]` Carefully

In Course Schedule problems, double check edge orientation: `[u, v]` means taking `v` requires `u`. Edge must go `u -> v` (`adj.get(u).add(v)` and `indegree[v]++`)!

BACKTRACKING

FAANG Master Guide — Java Edition

💡 WHY BACKTRACKING?

Backtracking is a controlled DFS search over a **State Space Tree**. It explores candidate solutions and **backtracks (undoes choice)** as soon as a path violates constraints!

💡 THE 3-STEP TRINITY

1. **CHOOSE:** Add candidate to path (`list.add(val)`).
2. **EXPLORE:** Recurse to next level (`dfs(...)`).
3. **UNDO:** Remove candidate (`list.remove(list.size() - 1)`).

1 3 CORE CATEGORIES

- **Subsets (2^N):** Pick or skip each element (`start` pointer increments).
- **Combinations ($N! / K!$):** Choose K elements out of N (`start` pointer).
- **Permutations ($N!$):** Order matters (`visited[]` array or element swapping).

2 VISUALIZING STATE SPACE TREE

Subsets of [1, 2]:

```
[]
├── [1]
│   └── [1, 2]
└── [2]
Total Subsets = 2^N = 4 ({}, {1}, {1,2}, {2})
```

📚 PREREQUISITES & COMPLEXITY REASONING

Category	Time Complexity	Auxiliary Space
Subsets (2^N)	**O(N · 2^N)**	**O(N) recursion stack**
Combinations (nCr)	**O(K · nCr)**	**O(K) stack**
Permutations ($N!$)	**O(N · N!)**	**O(N) stack**
N-Queens / Sudoku	**O(N!)**	**O(N) board state**

⚡ CLUES THAT SCREAM BACKTRACKING

1. *"Return all possible combinations / subsets / permutations"*
2. *"Duplicates allowed in input, output must contain UNIQUE results"* → Sort + Skip duplicate adjacent elements
3. *"Place N queens on board / solve Sudoku"* → Constraint Validation Grid DFS

PATTERN 1 SUBSETS I (Power Set Generation) e.g. Subsets (LC 78)

WHEN TO USE

Generate all 2^N subsets. Every node in the state space tree is a valid subset snapshot!

TEMPLATE — LC 78

```
public List<List<Integer>> subsets(int[] nums) {
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, 0);
  return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trac
k, int[] nums, int start) {
  res.add(new ArrayList<>(track));
  for (int i = start; i < nums.length; i++) {
    track.add(nums[i]);
    dfs(res, track, nums, i + 1);
    track.remove(track.size() - 1);
  }
}
```

PATTERN 2 SUBSETS II (Duplicates in Input) e.g. Subsets II (LC 90)

WHEN TO USE

Input array contains duplicates. Must sort and skip duplicate branch at same level (`i > start`).

TEMPLATE — LC 90

```
public List<List<Integer>> subsetsWithDup(int[] nums) {
  Arrays.sort(nums);
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, 0);
  return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trac
k, int[] nums, int start) {
  res.add(new ArrayList<>(track));
  for (int i = start; i < nums.length; i++) {
    if (i > start && nums[i] == nums[i - 1]) continue;
    track.add(nums[i]);
    dfs(res, track, nums, i + 1);
    track.remove(track.size() - 1);
  }
}
```

PATTERN 5 PERMUTATIONS I (Order Matters, Distinct Elements) e.g. Permutations (LC 46)

VISITED TRACKING

Order matters, so loop from `0` to `N-1` at every level. Track used elements via `boolean[] visited` or `track.contains()`.

TEMPLATE — LC 46

```
public List<List<Integer>> permute(int[] nums) {
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, new boolean[nums.lengt
h]);
  return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trac
k, int[] nums, boolean[] vis) {
  if (track.size() == nums.length) { res.add(new ArrayList<>
(track)); return; }
  for (int i = 0; i < nums.length; i++) {
    if (vis[i]) continue;
    vis[i] = true; track.add(nums[i]);
    dfs(res, track, nums, vis);
    track.remove(track.size() - 1); vis[i] = false;
  }
}
```

PATTERN 6 PERMUTATIONS II (Order Matters + Input Duplicates) e.g. Permutations II (LC 47)

VISITED PRUNING CONDITION

Sort array first. Skip duplicate if `nums[i] == nums[i-1]` AND `!visited[i-1]`.

TEMPLATE — LC 47

```
public List<List<Integer>> permuteUnique(int[] nums) {
  Arrays.sort(nums);
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, new boolean[nums.lengt
h]);
  return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trac
k, int[] nums, boolean[] vis) {
  if (track.size() == nums.length) { res.add(new ArrayList<>
(track)); return; }
  for (int i = 0; i < nums.length; i++) {
    if (vis[i] || (i > 0 && nums[i] == nums[i - 1] && !vis[i
- 1])) continue;
    vis[i] = true; track.add(nums[i]);
    dfs(res, track, nums, vis);
    track.remove(track.size() - 1); vis[i] = false;
  }
}
```

📊 COMPLETE BACKTRACKING PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
78	Subsets	Subsets I	O(N · 2^N)	O(N)	M
90	Subsets II	Subsets II Dup Pruning	O(N · 2^N)	O(N)	M
39	Combination Sum	Unlimited Reuse	O(2^T)	O(T)	M
40	Combination Sum II	Single Use + Dup Pruning	O(2^N)	O(N)	M
46	Permutations	Visited Permutations	O(N · N!)	O(N)	M
47	Permutations II	Permutations Dup Pruning	O(N · N!)	O(N)	M
79	Word Search	Grid DFS Backtracking	O(N · 4^L)	O(L)	M
131	Palindrome Partitioning	Substring Partitioning	O(N · 2^N)	O(N)	M
51	N-Queens	Constraint Grid DFS	O(N!)	$O(N^2)$	H

LC 78 · Subsets

MEDIUM

Pattern: Subsets I
Key Insight: Snapshot copy at every node.

```
public List<List<Integer>> subsets(int[] nums) {
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, 0); return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] n, int s) {
  res.add(new ArrayList<>(trk));
  for (int i = s; i < n.length; i++) {
    trk.add(n[i]); dfs(res, trk, n, i + 1); trk.remove(trk.size() - 1);
  }
}
```

LC 90 · Subsets II

MEDIUM

Pattern: Subsets II Dup Pruning
Key Insight: Sort + `i > start && nums[i] == nums[i-1]`.

```
public List<List<Integer>> subsetsWithDup(int[] nums) {
  Arrays.sort(nums); List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, 0); return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] n, int s) {
  res.add(new ArrayList<>(trk));
  for (int i = s; i < n.length; i++) {
    if (i > s && n[i] == n[i - 1]) continue;
    trk.add(n[i]); dfs(res, trk, n, i + 1); trk.remove(trk.size() - 1);
  }
}
```

LC 39 · Combination Sum

MEDIUM

Pattern: Unlimited Element Reuse
Key Insight: Pass `i` to next recursive call.

```
public List<List<Integer>> combinationSum(int[] cand, int target) {
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), cand, target, 0); return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] c, int rem, int s) {
  if (rem < 0) return;
  if (rem == 0) { res.add(new ArrayList<>(trk)); return; }
  for (int i = s; i < c.length; i++) {
    trk.add(c[i]); dfs(res, trk, c, rem - c[i], i); trk.remove(trk.size() - 1);
  }
}
```

LC 40 · Combination Sum II — MEDIUM

Pattern: Single Use + Dup Pruning
Key Insight: Sort + `i + 1` + skip duplicate.

```
public List<List<Integer>> combinationSum2(int[] cand, int target) {
  Arrays.sort(cand); List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), cand, target, 0); return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] c, int rem, int s) {
  if (rem < 0) return;
  if (rem == 0) { res.add(new ArrayList<>(trk)); return; }
  for (int i = s; i < c.length; i++) {
    if (i > s && c[i] == c[i - 1]) continue;
    trk.add(c[i]); dfs(res, trk, c, rem - c[i], i + 1); trk.remove(trk.size() - 1);
  }
}
```

LC 46 · Permutations — MEDIUM

Pattern: Visited Boolean Permutations
Key Insight: Loop $0..N-1$, skip if `visited[i]`.

```
public List<List<Integer>> permute(int[] nums) {
  List<List<Integer>> res = new ArrayList<>();
  dfs(res, new ArrayList<>(), nums, new boolean[nums.length]); return res;
}
private void dfs(List<List<Integer>> res, List<Integer> trk, int[] n, boolean[] vis) {
  if (trk.size() == n.length) { res.add(new ArrayList<>(trk)); return; }
  for (int i = 0; i < n.length; i++) {
    if (vis[i]) continue;
    vis[i] = true; trk.add(n[i]); dfs(res, trk, n, vis);
    trk.remove(trk.size() - 1); vis[i] = false;
  }
}
```

LC 79 · Word Search — MEDIUM

Pattern: Grid DFS Backtracking
Key Insight: Temp replace `board[r][c] = '#'`.

```
private boolean dfs(char[][] b, String w, int r, int c, int idx) {
  if (idx == w.length()) return true;
  if (r<0||r≥b.length||c<0||c≥b[0].length||b[r][c] ≠ w.charAt(idx)) return false;
  char tmp = b[r][c]; b[r][c] = '#';
  for (int[] d : DIRS) if (dfs(b, w, r + d[0], c + d[1], idx + 1)) return true;
  b[r][c] = tmp; return false;
}
```

LC 131 · Palindrome Partitioning — MEDIUM

Pattern: Substring Partitioning
Key Insight: Recurse if `isPalindrome(s, start, i)`.

```
private void dfs(List<List<String>> res, List<String> trk, String s, int start) {
  if (start == s.length()) { res.add(new ArrayList<>(trk)); return; }
  for (int i = start; i < s.length(); i++) {
    if (isPal(s, start, i)) {
      trk.add(s.substring(start, i + 1));
      dfs(res, trk, s, i + 1);
      trk.remove(trk.size() - 1);
    }
  }
}
```

LC 51 · N-Queens — HARD

Pattern: Constraint Grid DFS
Key Insight: Validate row, column, and diagonals.

```
private void dfs(List<List<String>> res, char[][] b, int row, int n) {
  if (row == n) { res.add(construct(b)); return; }
  for (int col = 0; col < n; col++) {
    if (isValid(b, row, col, n)) {
      b[row][col] = 'Q'; dfs(res, b, row + 1, n); b[row][col] = '.';
    }
  }
}
```

LC 17 · Phone Number Combinations — MEDIUM

Pattern: Digit Keypad Mapping
Key Insight: Map digits to letters string array.

```
private void dfs(List<String> res, StringBuilder sb, String digits, int idx, String[] map) {
  if (idx == digits.length()) { res.add(sb.toString()); return; }
  String letters = map[digits.charAt(idx) - '0'];
  for (char c : letters.toCharArray()) {
    sb.append(c); dfs(res, sb, digits, idx + 1, map); sb.deleteCharAt(sb.length() - 1);
  }
}
```

🔍 DRY RUN — Subsets ([1, 2])

Step	Track State	Action	Snapshot Added to Result
1	[]	`res.add([])`	`[]`
2	[1]	Add 1, recurse `start=1` -> `res.add([1])`	`[1]`
3	[1, 2]	Add 2, recurse `start=2` → `res.add([1, 2])`	`[1, 2]`
4	[1]	Pop 2 → Return to level 1	-
5	[]	Pop 1 → Loop `i=1` (add 2) → `res.add([2])`	`[2]` ✅

📐 MATHEMATICAL PROOF — 2^N SUBSETS DERIVATION

Claim: An array of N distinct elements produces exactly 2^N subsets.

Proof: For each element $x_i \in [1..N]$, we have binary choices: INCLUDE x_i or EXCLUDE x_i. Total subsets $= 2 \times 2 \times \dots \times 2 = 2^N$. Snapshot copy takes O(N) time → **O(N · 2^N) Time!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Write Subsets I in 3m
☐ Code Subsets II duplicate pruning
☐ Write Combination Sum (element reuse `i`)
☐ Code Permutations with `boolean[] visited`
☐ Write Permutations II duplicate check
☐ Code Word Search 2D Grid DFS
☐ Write N-Queens board placement DFS
☐ Prove why Subsets is O(N · 2^N)

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Always Make a Copy for Result
Always add a NEW copy to result list (`res.add(new ArrayList<>(track))`), NOT the original reference `res.add(track)`!

Rule 2 — Always Undo the Last Element
After the recursive call returns, immediately undo the choice: `track.remove(track.size() - 1)`!

DYNAMIC PROGRAMMING MASTERCLASS

The Universal SB → RBR → MTS Framework

🧠 THE UNIVERSAL DP PARADIGM: SB → RBR → MTS

Never jump straight to a DP array! Every single DP problem from Fibonacci & House Robber to Knapsack, LCS, and Tree DP is solved in this exact 3-phase order:

🟦 SB — State & Brainstorm

S — State: What minimum variables uniquely identify subproblem `dp[i]` or `dp[i][j]`?

B — Brainstorm Choices: *"If I am at this state, what choices do I have?"* (Take / Skip, 1 step / 2 steps)

🟧 RBR — Recurrence, Base, Result

R — Recurrence: Convert choices to equation e.g. `dp[i] = max(take, skip)`

B — Base Case: Where does recursion stop? (`dp[0] = 1`)

R — Result: What answer to return? (`dp[N]` or `dp[M][N]`)

🟩 MTS — Memo, Tab, Space

M — Memoization: Cache recursive calls Top-Down

T — Tabulation: Convert recursion to loops Bottom-Up

S — Space Optimization: Compress array `dp[i]` to `prev1, prev2` variables!

💡 GOLDEN RULE: THE COMPLETE SOLUTION FLOW

```
Brute Force Decision Tree
  ↓
Recursive Function
  ↓
SB (Define State & Choices)
  ↓
RBR (Recurrence + Base Case + Result)
  ↓
MTS (Memoization → Tabulation → Space Optimizat
ion O(1))
```

Aha Moment: DP is simply *Optimized Recursion*! Tabulation is Memoization turned upside-down!

⚡ CLUES THAT SCREAM DYNAMIC PROGRAMMING

1. *"Find max/min cost or total number of ways to reach X"* \rightarrow 1D Linear DP
2. *"Target sum with reusable vs 1-time items"* \rightarrow Unbounded vs 0/1 Knapsack DP
3. *"Longest common subsequence / Edit distance of 2 strings"* \rightarrow 2D String DP

🎬 STEP-BY-STEP EVOLUTION OF CLIMBING STAIRS (LC 70)

Phase 1: SB — State & Brainstorm

- **State:** `dp[i]` = total ways to reach step `i`.
- **Choices at step `i`:** Arrive from step `i-1` (climb 1 step) OR step `i-2` (climb 2 steps).

Phase 2: RBR — Recurrence, Base, Result

- **Recurrence:** `dp[i] = dp[i-1] + dp[i-2]`
- **Base Case:** `dp[1] = 1`, `dp[2] = 2`
- **Result:** `return dp[n]`

Phase 3: MTS — Memoization ➔ Tabulation ➔ Space Opt

We transform naive recursion O(2^N) into memoization O(N), then tabulation O(N), and finally compressed O(1) space!

PATTERN 1 HOUSE ROBBER (1D Include / Exclude Choice) e.g. House Robber (LC 198)

SB → RBR BREAKDOWN

State: `dp[i]` = max money robbed up to house `i`.
Brainstorm Choices: Rob house `i` (`nums[i] + dp[i-2]`) OR Skip house `i` (`dp[i-1]`).
Recurrence: `dp[i] = max(nums[i] + dp[i-2], dp[i-1])`.
Base: `dp[0] = nums[0]`, `dp[1] = max(nums[0], nums[1])`.

TEMPLATE — LC 198 (SPACE OPTIMIZED O(1))

```
public int rob(int[] nums) {
  if (nums == null || nums.length == 0) return 0;
  int prev2 = 0, prev1 = 0;
  for (int num : nums) {
    int tmp = prev1;
    prev1 = Math.max(prev2 + num, prev1); // Rob or Skip!
    prev2 = tmp;
  }
  return prev1;
}
```

PATTERN 2 COIN CHANGE (Unbounded Knapsack Minimum Coins) e.g. Coin Change (LC 322)

SB → RBR BREAKDOWN

State: `dp[a]` = min coins to make target amount `a`.
Brainstorm Choices: Try every coin `c` where `c <= a`.
Recurrence: `dp[a] = min(dp[a], 1 + dp[a - c])`.
Base: `dp[0] = 0`. Initialize all other entries to `amount + 1`.

TEMPLATE — LC 322

```
public int coinChange(int[] coins, int amount) {
  int[] dp = new int[amount + 1];
  Arrays.fill(dp, amount + 1); // Fill with infinity sentinel!
  dp[0] = 0; // Base case: 0 amount needs 0 coins
  for (int a = 1; a ⩽ amount; a++) {
    for (int c : coins) {
      if (a - c ⩾ 0) {
        dp[a] = Math.min(dp[a], 1 + dp[a - c]);
      }
    }
  }
  return dp[amount] > amount ? -1 : dp[amount];
}
```

PATTERN 3 LONGEST INCREASING SUBSEQUENCE (Patience Sorting Binary Search)

e.g. Longest Increasing Subsequence (LC 300)

PATIENCE SORTING O(N LOG N)

Maintain array `tails` storing smallest tail element of all increasing subsequences of length `i+1`. Use binary search `lowerBound` to replace or extend tails array!

TEMPLATE — LC 300

```
public int lengthOfLIS(int[] nums) {
  int[] tails = new int[nums.length];
  int len = 0;
  for (int x : nums) {
    int i = 0, j = len;
    while (i < j) {
      int mid = i + (j - i) / 2;
      if (tails[mid] < x) i = mid + 1;
      else j = mid;
    }
    tails[i] = x;
    if (i == len) len++;
  }
  return len;
}
```

PATTERN 4 UNIQUE PATHS (2D Grid Top-Left to Bottom-Right) e.g. Unique Paths (LC 62)

SB → RBR BREAKDOWN

State: `dp[r][c]` = unique paths to cell `(r, c)`.
Brainstorm Choices: Arrived from top `(r-1, c)` OR left `(r, c-1)`.
Recurrence: `dp[r][c] = dp[r-1][c] + dp[r][c-1]`.

TEMPLATE — LC 62 (SPACE OPTIMIZED O(N))

```
public int uniquePaths(int m, int n) {
  int[] row = new int[n];
  Arrays.fill(row, 1); // Top row base case: 1 path
  for (int r = 1; r < m; r++) {
    for (int c = 1; c < n; c++) {
      row[c] += row[c - 1]; // row[c] = top + left!
    }
  }
  return row[n - 1];
}
```

DYNAMIC PROGRAMMING MASTERCLASS

Dry Run, Proofs & Mastery Cheat Sheet

🔍 DRY RUN — Coin Change ([1, 2, 5], amount = 11)

Amount (a)	Coin Try	dp[a] Recurrence (`1 + dp[a - c]`)	dp[a] Result
1	1	`1 + dp[0] = 1`	1
2	1, 2	`min(1+dp[1], 1+dp[0]) = 1`	1
5	5	`1 + dp[0] = 1`	1
11	5	`1 + dp[6] = 1 + 2 = 3` (coins 5+5+1)	3 ✅

📐 MATHEMATICAL PROOF — PRINCIPLE OF OPTIMAL SUBSTRUCTURE

Claim: An optimal solution to a DP problem contains optimal solutions to its subproblems.

Proof: If a subproblem solution were non-optimal, we could replace it with a smaller subproblem cost, strictly decreasing the total cost. This contradicts the minimality of the optimal overall solution \rightarrow **Exact Proof of Correctness!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Apply the SB → RBR → MTS framework to any DP problem
- ☐ Code House Robber in O(1) space
- ☐ Code Coin Change unbounded knapsack in O(Amount)
- ☐ Write Longest Increasing Subsequence in O(N log N)
- ☐ Code 2D String LCS in O(M · N) time
- ☐ Explain why 0/1 Knapsack loops BACKWARDS in 1D array
- ☐ Prove the Principle of Optimal Substructure
- ☐ Convert Top-Down recursion to Bottom-Up tabulation

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — State Definition is Everything

Always define your DP state (`dp[i]`) in plain English BEFORE writing any code. If your state definition is clear, the recurrence relation falls into place effortlessly!

Rule 2 — Initialize DP Table with Infinity Sentinels

For minimum cost/coin problems, initialize all `dp[]` entries with an infinity sentinel (`amount + 1` or `1e9`), and set base case `dp[0] = 0`!

GREEDY ALGORITHMS

FAANG Master Guide — Java Edition

💡 WHY GREEDY?

Greedy algorithms make the **locally optimal choice at each step**, hoping that this choice leads to a globally optimal solution without ever looking back or backtracking!

💡 THE GREEDY PROOF AHA

Greedy works ONLY when local choices never restrict optimal future choices (Greedy Choice Property). If choices depend on future outcomes, use **Dynamic Programming**!

1 3 CORE PATTERNS

- **Farthest Reach (Jump Game):** Maintain `maxReach` boundary iteratively.
- **Gas Station Circuit:** If total gas >= total cost, a valid starting station is guaranteed!
- **Kadane's Algorithm:** Reset running subarray sum to 0 if it drops below 0!

2 VISUALIZING JUMP GAME

Nums = [2, 3, 1, 1, 4]:

```
Index 0 (val 2) ➔ maxReach = max (0, 0+2) = 2
Index 1 (val 3) ➔ maxReach = max (2, 1+3) = 4!
maxReach (4) >= Last Index (4) ➔ True!
```

📚 PREREQUISITES & COMPLEXITY REASONING

Algorithm	Time Complexity	Auxiliary Space
Jump Game I & II	**O(N) single pass**	**O(1) space**
Gas Station (LC 134)	**O(N) single pass**	**O(1) space**
Kadane's Max Subarray	**O(N) single pass**	**O(1) space**
Task Scheduler (LC 621)	**O(N) frequency**	**O(1) hashmap**

⚡ CLUES THAT SCREAM GREEDY

1. *"Minimum jumps to reach end" / "Can reach last index"* -> Farthest Reach
2. *"Complete circular gas circuit"* -> Cumulative Tank Reset
3. *"Maximum sum contiguous subarray"* -> Kadane's Algorithm

PATTERN 1 JUMP GAME I (Farthest Reach Boundary) e.g. Jump Game (LC 55)

WHEN TO USE

Check if last index is reachable in O(N) time and O(1) space.

TEMPLATE — LC 55

```
public boolean canJump(int[] nums) {
  int maxReach = 0;
  for (int i = 0; i < nums.length; i++) {
    if (i > maxReach) return false;
    maxReach = Math.max(maxReach, i + nums[i]);
  }
  return true;
}
```

PATTERN 2 JUMP GAME II (Minimum Jumps BFS Window) e.g. Jump Game II (LC 45)

BFS WINDOW STRATEGY

Maintain current jump boundary `currEnd`. When loop reaches `currEnd`, increment `jumps++` and update `currEnd = maxReach`!

TEMPLATE — LC 45

```
public int jump(int[] nums) {
  int jumps = 0, currEnd = 0, maxReach = 0;
  for (int i = 0; i < nums.length - 1; i++) {
    maxReach = Math.max(maxReach, i + nums[i]);
    if (i == currEnd) {
      jumps++;
      currEnd = maxReach;
    }
  }
  return jumps;
}
```

PATTERN 5 TASK SCHEDULER (Max Frequency Idle Slot Formula) e.g. Task Scheduler (LC 621)

FORMULA INTUITION

Find max frequency `maxFreq`. Minimum slots needed $= (\text{maxFreq} - 1) \times (n + 1) + \text{countMaxFreq}$. Return $\max(\text{tasks.length}, \text{slots})$.

TEMPLATE — LC 621

```
public int leastInterval(char[] tasks, int n) {
  int[] freq = new int[26];
  for (char t : tasks) freq[t - 'A']++;
  Arrays.sort(freq);
  int maxFreq = freq[25];
  int countMaxFreq = 0;
  for (int f : freq) if (f == maxFreq) countMaxFreq++;
  int partCount = maxFreq - 1;
  int partLength = n - (countMaxFreq - 1);
  int emptySlots = partCount * partLength;
  int availableTasks = tasks.length - maxFreq * countMaxFreq;
  int idles = Math.max(0, emptySlots - availableTasks);
  return tasks.length + idles;
}
```

PATTERN 6 VALID PARENTHESIS STRING (Min / Max Open Bounds) e.g. Valid Parenthesis String (LC 678)

RANGE BOUND TRACKING

Track `[cmin, cmax]` range of possible open brackets. `*` expands range `[cmin-1, cmax+1]`. Valid if `cmin == 0` at end!

TEMPLATE — LC 678

```
public boolean checkValidString(String s) {
  int cmin = 0, cmax = 0;
  for (char c : s.toCharArray()) {
    if (c == '(') { cmin++; cmax++; }
    else if (c == ')') { cmin--; cmax--; }
    else { cmin--; cmax++; } // '*' can be ')', empty, or '('
    if (cmax < 0) return false;
    cmin = Math.max(cmin, 0);
  }
  return cmin == 0;
}
```

COMPLETE GREEDY PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
53	Maximum Subarray	Kadane's Algorithm	O(N)	O(1)	E
55	Jump Game	Farthest Reach Boundary	O(N)	O(1)	M
45	Jump Game II	BFS Window Greedy	O(N)	O(1)	M
134	Gas Station	Circular Tank Reset	O(N)	O(1)	M
763	Partition Labels	Last Index Boundary	O(N)	O(1)	M
621	Task Scheduler	Max Frequency Formula	O(N)	O(1)	M
678	Valid Parenthesis String	Min/Max Open Bounds	O(N)	O(1)	M

LC 53 · Maximum Subarray

EASY

Pattern: Kadane's Algorithm
Key Insight: Reset sum if negative.

```
public int maxSubArray(int[] nums)
{
  int mx = nums[0], cur = 0;
  for (int n : nums) {
    cur = Math.max(n, cur + n); mx
= Math.max(mx, cur);
  }
  return mx;
}
```

LC 55 · Jump Game

MEDIUM

Pattern: Farthest Reach
Key Insight: Return false if `i > maxReach`.

```
public boolean canJump(int[] nums)
{
  int mx = 0;
  for (int i = 0; i < nums.length;
i++) {
    if (i > mx) return false;
    mx = Math.max(mx, i + nums
[i]);
  }
  return true;
}
```

LC 45 · Jump Game II

MEDIUM

Pattern: BFS Window Greedy
Key Insight: `jumps++` when `i == currEnd`.

```
public int jump(int[] nums) {
  int j = 0, curEnd = 0, mx = 0;
  for (int i = 0; i < nums.length
- 1; i++) {
    mx = Math.max(mx, i + nums
[i]);
    if (i == curEnd) { j++; curEnd
= mx; }
  }
  return j;
}
```

LC 134 · Gas Station

MEDIUM

Pattern: Tank Reset
Key Insight: Reset `start = i + 1` if `currTank < 0`.

```
public int canCompleteCircuit(int[] gas, int[] cost) {
  int tot = 0, cur = 0, st = 0;
  for (int i = 0; i < gas.length; i++) {
    int d = gas[i] - cost[i]; tot += d; cur += d;
    if (cur < 0) { st = i + 1; cur = 0; }
  }
  return tot >= 0 ? st : -1;
}
```

LC 763 · Partition Labels

MEDIUM

Pattern: Last Index Window
Key Insight: Partition when `i == end`.

```
public List<Integer> partitionLabels(String s) {
  int[] last = new int[26];
  for (int i = 0; i < s.length(); i++) last[s.charAt(i) - 'a'] = i;
  List<Integer> res = new ArrayList<>();
  int st = 0, end = 0;
  for (int i = 0; i < s.length(); i++) {
    end = Math.max(end, last[s.charAt(i) - 'a']);
    if (i == end) { res.add(end - st + 1); st = i + 1; }
  }
  return res;
}
```

LC 621 · Task Scheduler

MEDIUM

Pattern: Frequency Formula
Key Insight: Slots $= (\text{maxFreq} - 1) \times (n + 1) + \text{countMax}$.

```
public int leastInterval(char[] tasks, int n) {
  int[] f = new int[26]; for (char t : tasks) f[t - 'A']++;
  Arrays.sort(f); int maxF = f[25], countMax = 0;
  for (int v : f) if (v == maxF) countMax++;
  return Math.max(tasks.length, (maxF - 1) * (n + 1) + countMax);
}
```

LC 678 · Valid Parenthesis String

MEDIUM

Pattern: Min/Max Open Bounds
Key Insight: Track `[cmin, cmax]`. Return `cmin == 0`.

```
public boolean checkValidString(String s) {
  int cmin = 0, cmax = 0;
  for (char c : s.toCharArray()) {
    if (c == '(') { cmin++; cmax++; }
    else if (c == ')') { cmin--; cmax--; }
    else { cmin--; cmax++; }
    if (cmax < 0) return false;
    cmin = Math.max(cmin, 0);
  }
  return cmin == 0;
}
```

🔍 DRY RUN — Gas Station (gas=[1,2,3,4,5], cost=[3,4,5,1,2])

Index	gas[i] - cost[i]	currTank	start Candidate	Action
0	1 - 3 = -2	-2	1	Reset `currTank = 0`, candidate `start = 1`
1	2 - 4 = -2	-2	2	Reset `currTank = 0`, candidate `start = 2`
2	3 - 5 = -2	-2	3	Reset `currTank = 0`, candidate `start = 3`
3	4 - 1 = +3	+3	3	Valid tank (+3)
4	5 - 2 = +3	+6	3	Return `start = 3` ✅

📐 MATHEMATICAL PROOF — KADANE'S ALGORITHM

Claim: Resetting running sum to 0 when it drops below 0 yields the maximum subarray sum.

Proof: Any prefix subarray with a negative sum $\sum_{k=i}^j A[k] < 0 can ONLY decrease the sum of any future subarray starting at j+1$. Therefore, discarding this negative prefix is strictly optimal -> **O(N) Time!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Code Kadane's algorithm in O(1) space
☐ Write Jump Game I farthest reach in 2m
☐ Implement Jump Game II BFS window
☐ Code Gas Station circuit tank reset
☐ Write Partition Labels last occurrence map
☐ Code Task Scheduler max frequency formula
☐ Write Valid Parenthesis String min/max bounds
☐ Prove why Kadane's resets negative sums

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Verify Total Sum Before Returning

In Gas Station, if total gas >= total cost across the entire array, a valid starting station is guaranteed to exist!

Rule 2 — Loop to N-1 for Jump Game II

In Jump Game II, stop the loop at `N - 2` because once you reach `N - 1`, you are already at the destination and don't need another jump!

INTERVALS

FAANG Master Guide — Java Edition

💡 WHY INTERVALS?

Intervals represent ranges `[start, end]` (time slots, meeting rooms, line segments). Almost all interval problems reduce to **sorting by start or end time**, then scanning linearly in O(N \log N)!

💡 THE OVERLAP CONDITION AHA

Two intervals `A` and `B` overlap if and only if **`max(A.start, B.start) <= min(A.end, B.end)`**!

1 3 CORE SORTING STRATEGIES

- **Sort by Start Ascending (`a[0] - b[0]`):** Standard for Merge Intervals, Insert Interval, Meeting Rooms.
- **Sort by End Ascending (`a[1] - b[1]`):** Optimal greedy choice for Non-Overlapping Intervals (leave max room for future!).
- **Min-Heap PriorityQueue:** Track active ending times for Meeting Rooms II.

2 VISUALIZING INTERVAL MERGING

Merge [1, 3] and [2, 6]:

```
Interval A: [1 ── 3]
Interval B:   [2 ──── 6]
Overlap check: 2 <= 3 ➔ OVERLAP!
Merged Result: [1 ──────── 6] (ma
x(3, 6) = 6)
```

PREREQUISITES & COMPLEXITY REASONING

Operation / Algorithm	Time Complexity	Auxiliary Space
Merge Intervals (Sort + Scan)	**O(N log N)**	**O(N) output**
Insert Interval (Pre-sorted input)	**O(N) single pass**	**O(N) output**
Non-overlapping (Greedy Sort End)	**O(N log N)**	**O(1) space**
Meeting Rooms II (Min-Heap PQ)	**O(N log N)**	**O(N) PriorityQueue**

CLUES THAT SCREAM INTERVALS

1. *"Merge overlapping ranges"* -> Sort by Start + Link `end = Math.max(end, next.end)`
2. *"Minimum conference rooms required"* -> Min-Heap PQ of End Times
3. *"Minimum removals to eliminate overlaps"* -> Greedy Sort by End Time

PATTERN 1 INSERT INTERVAL (3-Phase Single Pass) e.g. Insert Interval (LC 57)

3-PHASE STRATEGY

1. Add all before new interval (`interval[1] < new[0]`).
2. Merge overlapping (`interval[0] <= new[1]`).
3. Add all after. O(N) time!

TEMPLATE — LC 57

```
public int[][] insert(int[][] intervals, int[] newInterval) {
  List<int[]> res = new ArrayList<>();
  int i = 0, n = intervals.length;
  while (i < n && intervals[i][1] < newInterval[0]) res.add(interv
als[i++]);
  while (i < n && intervals[i][0] <= newInterval[1]) {
    newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
    newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
    i++;
  }
  res.add(newInterval);
  while (i < n) res.add(intervals[i++]);
  return res.toArray(new int[res.size()][]);
}
```

PATTERN 2 MERGE INTERVALS (Sort by Start Time) e.g. Merge Intervals (LC 56)

WHEN TO USE

Combine all overlapping intervals into single continuous intervals.

TEMPLATE — LC 56

```
public int[][] merge(int[][] intervals) {
  Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
  List<int[]> res = new ArrayList<>();
  int[] curr = intervals[0]; res.add(curr);
  for (int[] in : intervals) {
    if (in[0] <= curr[1]) curr[1] = Math.max(curr[1], in[1]);
    else { curr = in; res.add(curr); }
  }
  return res.toArray(new int[res.size()][]);
}
```

PATTERN 5 MEETING ROOMS I (Single Person Attendance Check) e.g. Meeting Meetings (LC 252)

WHEN TO USE

Check if a single person can attend all meetings without overlap.

TEMPLATE — LC 252

```
public boolean canAttendMeetings(int[][] intervals) {
  Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
  for (int i = 0; i < intervals.length - 1; i++) {
    if (intervals[i][1] > intervals[i + 1][0]) return false;
  }
  return true;
}
```

PATTERN 6 MEETING ROOMS II (Min-Heap PriorityQueue) e.g. Meeting Rooms II (LC 253)

MIN-HEAP STRATEGY

Sort by start time. Maintain Min-Heap storing end times of active meetings. If `start >= pq.peek()`, reuse room (`pq.poll()`)!

TEMPLATE — LC 253

```
public int minMeetingRooms(int[][] intervals) {
  if (intervals.length == 0) return 0;
  Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
  PriorityQueue<Integer> minHeap = new PriorityQueue<>();
  minHeap.offer(intervals[0][1]);
  for (int i = 1; i < intervals.length; i++) {
    if (intervals[i][0] >= minHeap.peek()) {
      minHeap.poll(); // Reuse room!
    }
    minHeap.offer(intervals[i][1]);
  }
  return minHeap.size();
}
```

COMPLETE INTERVALS PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
252	Meeting Rooms	Single Person Check	O(N log N)	O(1)	E
56	Merge Intervals	Merge Overlapping	O(N log N)	O(N)	M
57	Insert Interval	3-Phase Single Pass	O(N)	O(N)	M
253	Meeting Rooms II	Min-Heap PriorityQueue	O(N log N)	O(N)	M
435	Non-overlapping Intervals	Greedy Sort by End	O(N log N)	O(1)	M
452	Minimum Arrows to Burst Balloons	Sort by End Coordinate	O(N log N)	O(1)	M

LC 252 · Meeting Rooms — EASY

Pattern: Single Person Check
Key Insight: Sort start. `curr.end > next.start` = false.

```
public boolean canAttendMeetings(int[][] intervals) {
  Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
  for (int i = 0; i < intervals.length - 1; i++) {
    if (intervals[i][1] > intervals[i + 1][0]) return false;
  }
  return true;
}
```

LC 56 · Merge Intervals — MEDIUM

Pattern: Merge Overlapping
Key Insight: Update `curr[1] = Math.max(...)`.

```
public int[][] merge(int[][] intervals) {
  Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
  List<int[]> res = new ArrayList<>();
  int[] curr = intervals[0]; res.add(curr);
  for (int[] in : intervals) {
    if (in[0] <= curr[1]) curr[1] = Math.max(curr[1], in[1]);
    else { curr = in; res.add(curr); }
  }
  return res.toArray(new int[res.size()][]);
}
```

LC 57 · Insert Interval — MEDIUM

Pattern: 3-Phase Single Pass
Key Insight: Before, overlap merge, after.

```
public int[][] insert(int[][] in, int[] newIn) {
  List<int[]> res = new ArrayList<>(); int i = 0, n = in.length;
  while (i < n && in[i][1] < newIn[0]) res.add(in[i++]);
  while (i < n && in[i][0] <= newIn[1]) {
    newIn[0] = Math.min(newIn[0], in[i][0]);
    newIn[1] = Math.max(newIn[1], in[i][1]); i++;
  }
  res.add(newIn);
  while (i < n) res.add(in[i++]);
  return res.toArray(new int[res.size()][]);
}
```

LC 253 · Meeting Rooms II — MEDIUM

Pattern: Min-Heap PQ
Key Insight: If `start >= pq.peek()`, reuse room (`pq.poll()`).

```
public int minMeetingRooms(int[][] intervals) {
  Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
  PriorityQueue<Integer> pq = new PriorityQueue<>();
  pq.offer(intervals[0][1]);
  for (int i = 1; i < intervals.length; i++) {
    if (intervals[i][0] >= pq.peek()) pq.poll();
    pq.offer(intervals[i][1]);
  }
  return pq.size();
}
```

LC 435 · Non-overlapping Intervals — MEDIUM

Pattern: Greedy Sort by End
Key Insight: If overlap, `removals++`.

```
public int eraseOverlapIntervals(int[][] intervals) {
  Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));
  int rem = 0, prevEnd = intervals[0][1];
  for (int i = 1; i < intervals.length; i++) {
    if (intervals[i][0] < prevEnd) rem++;
    else prevEnd = intervals[i][1];
  }
  return rem;
}
```

LC 452 · Minimum Arrows for Balloons — MEDIUM

Pattern: Sort by End Coordinate
Key Insight: Shoot arrow at `prevEnd`. `Integer.compare` avoids overflow!

```
public int findMinArrowShots(int[][] points) {
  if (points.length == 0) return 0;
  Arrays.sort(points, (a, b) -> Integer.compare(a[1], b[1]));
  int arrows = 1, prevEnd = points[0][1];
  for (int i = 1; i < points.length; i++) {
    if (points[i][0] > prevEnd) {
      arrows++; prevEnd = points[i][1];
    }
  }
  return arrows;
}
```

🔍 DRY RUN — Merge Intervals ([[1,3],[2,6],[8,10]])

Step	Interval	curr Interval	Overlap Check	Result List State
1	[1, 3]	[1, 3]	Initial	`[[1, 3]]`
2	[2, 6]	[1, 3]	2 <= 3 -> Overlap! `curr[1] = max(3, 6) = 6`	`[[1, 6]]`
3	[8, 10]	[1, 6]	$8 > 6 ->$ No overlap! `curr = [8, 10]`	`[[1, 6], [8, 10]]` ✅

📐 MATHEMATICAL PROOF — GREEDY END SORT FOR NON-OVERLAPPING

Claim: Sorting by end time E_i maximizes the number of non-overlapping intervals.

Proof: Choosing the interval with the EARLIEST end time E_1 leaves the maximum possible timeline available for all remaining intervals. Any alternative choice ending later can only restrict future choices -> **Strictly Optimal Greedy Choice!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Use `Integer.compare(a, b)` for sorting
- ☐ Write Merge Intervals (LC 56) in 3m
- ☐ Code Insert Interval 3-phase loop
- ☐ Write Meeting Rooms II with Min-Heap PQ
- ☐ Code Non-overlapping Intervals (sort by end)
- ☐ Implement Minimum Arrows to Burst Balloons
- ☐ Code Meeting Rooms II Line Sweep alternative
- ☐ Prove why sorting by end time is greedy optimal

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Integer.compare Prevents Overflow

Never use `(a, b) -> a[0] - b[0]`! If $a[0] = -2147483648$, subtraction overflows. Always use `Integer.compare(a[0], b[0])`!

Rule 2 — Strict vs Non-Strict Overlap

Meeting Rooms II considers `[1, 5]` and `[5, 10]` NON-overlapping (room reused at 5). Balloon shooting considers `[1, 5]` and `[5, 10]` OVERLAPPING (arrow at 5 bursts both)!

BIT MANIPULATION

FAANG Master Guide — Java Edition

💡 WHY BIT MANIPULATION?

Bitwise operations manipulate individual bits directly inside CPU registers in **O(1) time with 0 extra memory**! They power low-level systems, cryptographic algorithms, and optimal interview solutions.

💡 THE KERNIGHAN'S TRICK AHA

`n & (n - 1)` clears the lowest set bit (`1`) of `n` in exact O(1) time! Use this to count set bits in O(\text{set bits}) instead of 32 loop iterations!

1 6 ESSENTIAL BIT OPERATORS

- **AND (`&`):** Both bits 1 -> 1. Masking.
- **OR (`|`):** Either bit 1 -> 1. Setting bits.
- **XOR (`^`):** Different bits -> 1. $x \oplus x = 0$, $x \oplus 0 = x$!
- **NOT (`~`):** Bitwise inversion.
- **Left Shift (`<<`):** Multiply by 2^k.
- **Unsigned Right Shift (`>>>`):** Fills 0 on left.

2 VISUALIZING KERNIGHAN'S TRICK

Clearing lowest set bit of 12 (`1100`):

```
n     = 12 -> 1100 (binary)
n - 1 = 11 -> 1011 (binary)
n & (n - 1) = 1000 (binary) -> 8!
Lowest set bit at position 2 is C
LEARED!
```

PREREQUISITES & COMPLEXITY REASONING

Operation / Problem	Time Complexity	Auxiliary Space
Single Number (XOR Accumulation)	**O(N) single pass**	**O(1) space**
Hamming Weight (n & (n-1))	**O(set bits) <= 32**	**O(1) space**
Counting Bits (DP + Bitshift)	**O(N) single pass**	**O(N) result array**
Sum of 2 Integers (XOR + AND Carry)	**O(32) = O(1)**	**O(1) space**

CLUES THAT SCREAM BIT MANIPULATION

1. *"Every element appears twice except one"* -> Accumulative XOR (`a ^= num`)
2. *"Count total 1 bits in 32-bit integer"* -> Kernighan's `n = n & (n - 1)`
3. *"Add two numbers without + or -"* -> Bitwise XOR (`a ^ b`) + Carry (`(a & b) << 1`)

PATTERN 1 SINGLE NUMBER (XOR Self-Cancellation) e.g. Single Number (LC 136)

XOR CANCELLATION MATH

$x \oplus x = 0 and x \oplus 0 = x$. Since all duplicate pairs cancel to 0, XORing all array numbers leaves ONLY the unique single number!

TEMPLATE — LC 136

```
public int singleNumber(int[] nums) {
  int res = 0;
  for (int num : nums) {
    res ^= num; // Pair cancellation!
  }
  return res;
}
```

PATTERN 2 MISSING NUMBER (Index XOR Array Accumulation) e.g. Missing Number (LC 268)

INDEX MATCHING STRATEGY

XOR all indices $0..N$ with all array values `nums[i]`. Every present number cancels its matching index, leaving ONLY the missing number!

TEMPLATE — LC 268

```
public int missingNumber(int[] nums) {
  int res = nums.length;
  for (int i = 0; i < nums.length; i++) {
    res ^= i ^ nums[i];
  }
  return res;
}
```

PATTERN 5 SUM OF TWO INTEGERS (XOR Sum + AND Carry) e.g. Sum of Two Integers (LC 371)

HALF-ADDER HARDWARE LOGIC

`a ^ b` computes addition WITHOUT carry.
`(a & b) << 1` computes carry.
Repeat until carry `b == 0`!

TEMPLATE — LC 371

```
public int getSum(int a, int b) {
  while (b != 0) {
    int carry = (a & b) << 1;
    a = a ^ b;
    b = carry;
  }
  return a;
}
```

PATTERN 6 REVERSE INTEGER (32-Bit Overflow Guard) e.g. Reverse Integer (LC 7)

OVERFLOW GUARD FORMULA

Check `res > Integer.MAX_VALUE / 10` or `res < Integer.MIN_VALUE / 10` before multiplying by 10!

TEMPLATE — LC 7

```
public int reverse(int x) {
  int res = 0;
  while (x != 0) {
    int pop = x % 10; x /= 10;
    if (res > Integer.MAX_VALUE / 10 || (res == Integer.MAX_VALUE / 10
&& pop > 7)) return 0;
    if (res < Integer.MIN_VALUE / 10 || (res == Integer.MIN_VALUE / 10
&& pop < -8)) return 0;
    res = res * 10 + pop;
  }
  return res;
}
```

COMPLETE BIT MANIPULATION PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
136	Single Number	XOR Self-Cancellation	O(N)	O(1)	E
191	Number of 1 Bits	Kernighan's Trick	O(set bits)	O(1)	E
268	Missing Number	Index XOR Match	O(N)	O(1)	E
338	Counting Bits	DP Bitshift Recurrence	O(N)	O(N)	E
190	Reverse Bits	32-Bit Unsigned Shift	O(32)	O(1)	E
371	Sum of Two Integers	XOR + AND Carry	O(32)	O(1)	M
7	Reverse Integer	Overflow Protection	O(log10 X)	O(1)	M

LC 136 · Single Number

EASY

Pattern: XOR Self-Cancel
Key Insight: `res ^= num`.

```
public int singleNumber(int[] nums) {
  int res = 0;
  for (int n : nums) res ^= n;
  return res;
}
```

LC 191 · Number of 1 Bits

EASY

Pattern: Kernighan's Trick
Key Insight: `n = n & (n - 1)`.

```
public int hammingWeight(int n) {
  int c = 0;
  while (n != 0) { n &= (n - 1); c++; }
  return c;
}
```

LC 268 · Missing Number

EASY

Pattern: Index XOR Match
Key Insight: `res ^= i ^ nums[i]`.

```
public int missingNumber(int[] nums) {
  int res = nums.length;
  for (int i = 0; i < nums.length; i++) res ^= i ^ nums[i];
  return res;
}
```

LC 338 · Counting Bits — EASY

Pattern: DP Bitshift
Key Insight: `dp[i] = dp[i >> 1] + (i & 1)`.

```
public int[] countBits(int n) {
  int[] res = new int[n + 1];
  for (int i = 1; i <= n; i++) res
[i] = res[i >> 1] + (i & 1);
  return res;
}
```

LC 190 · Reverse Bits — EASY

Pattern: 32-Bit Unsigned Shift
Key Insight: `res = (res << 1) | (n & 1)`, `n >>>= 1`.

```
public int reverseBits(int n) {
  int res = 0;
  for (int i = 0; i < 32; i++) {
    res = (res << 1) | (n & 1); n >
>>= 1;
  }
  return res;
}
```

LC 371 · Sum of Two Integers — MEDIUM

Pattern: XOR + AND Carry
Key Insight: `a = a ^ b`, `b = (a & b) << 1`.

```
public int getSum(int a, int b) {
  while (b != 0) {
    int carry = (a & b) << 1;
    a = a ^ b; b = carry;
  }
  return a;
}
```

LC 7 · Reverse Integer — MEDIUM

Pattern: Overflow Protection
Key Insight: Guard `res > MAX / 10` before multiplying.

```
public int reverse(int x) {
  int res = 0;
  while (x != 0) {
    int pop = x % 10; x /= 10;
    if (res > Integer.MAX_VALUE / 10
|| res < Integer.MIN_VALUE / 10) retu
rn 0;
    res = res * 10 + pop;
  }
  return res;
}
```

🔍 DRY RUN — Single Number ([4, 1, 2, 1, 2])

Step	num	XOR Operation (`res ^= num`)	res Value
1	4	`0 ^ 4`	4
2	1	`4 ^ 1`	5
3	2	`5 ^ 2`	7
4	1	`7 ^ 1` (cancels previous 1)	6
5	2	`6 ^ 2` (cancels previous 2)	4 ✅

📐 MATHEMATICAL PROOF — KERNIGHAN'S CLEAR BIT FORMULA

Claim: `n & (n - 1)` clears the lowest set bit of `n`.

Proof: Subtracting 1 flips the rightmost `1` to `0` and all lower trailing `0`s to `1`s. Performing `n & (n - 1)` keeps all higher bits unchanged while forcing the rightmost `1` to `0` -> **Exact 1 bit cleared!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Code Single Number in 1m with XOR
☐ Write Kernighan's `n & (n - 1)` bit counter
☐ Code Missing Number with index XOR
☐ Write Counting Bits with DP bitshift
☐ Code Reverse Bits using `>>>` shift
☐ Write Sum of Two Integers (XOR + Carry)
☐ Write Reverse Integer with overflow guard
☐ Prove why `n & (n - 1) == 0` tests power of 2

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Operator Precedence Warning

Bitwise operators (`&`, `|`, `^`) have LOWER precedence than comparison operators (`==`, `<`)! ALWAYS wrap bitwise expressions in parentheses: `((n & 1) == 0)`!

Rule 2 — Always Use Unsigned Right Shift for Reversal

In 32-bit bit manipulation, use `>>>` instead of `>>` so negative numbers do not fill leading 1s!

MATH & GEOMETRY

FAANG Master Guide — Java Edition

💡 WHY MATH & GEOMETRY?

Matrix transformations (rotations, spirals, zeroing) and numerical algorithms (binary exponentiation, digit manipulation) rely on **pure mathematical symmetries** instead of complex data structures!

💡 THE 90° ROTATION AHA

Rotating an N \times N matrix 90° clockwise in-place is mathematically equivalent to: 1. **Transpose Matrix** (`swap(matrix[i][j], matrix[j][i])`) 2. **Reverse Each Row** (`swap(matrix[i][l], matrix[i][r])`)!

1 3 CORE GEOMETRIC PATTERNS

- **Matrix Transpose & Reverse:** 90° clockwise rotation in O(N^2) time and O(1) space.
- **4-Boundary Spiral Scan:** Shrink `top`, `bottom`, `left`, `right` boundaries after scanning each edge.
- **First Row/Col State Markers:** Mark zeroes in row 0 and col 0 for O(1) auxiliary space in Set Matrix Zeroes!

2 VISUALIZING MATRIX ROTATION

Original -> Transpose -> Reversed Rows:

```
Original:     Transpose:    Rever
sed Rows (90°):
[1 2]   ——>  [1 3]   ——>  [3 1]
[3 4]        [2 4]        [4 2]
```

🧩 PREREQUISITES & COMPLEXITY REASONING

Algorithm / Operation	Time Complexity	Auxiliary Space
Rotate Image (Transpose + Reverse)	**O(N^2)**	**O(1) in-place**
Spiral Matrix (4 Boundaries)	**O(M · N)**	**O(1) space**
Set Matrix Zeroes (Row/Col 0 Markers)	**O(M · N)**	**O(1) space**
Pow(x, n) (Binary Exponentiation)	**O(log N)**	**O(log N) stack**

⚡ CLUES THAT SCREAM MATH & GEOMETRY

1. *"Rotate 2D matrix 90 degrees in-place"* -> Transpose + Reverse Rows
2. *"Traverse 2D matrix in spiral order"* -> 4 Boundary Pointers (`top`, `bot`, `left`, `right`)
3. *"Compute x^n in O(log n) time"* -> Binary Exponentiation (`n / 2` recursive squarings)

PATTERN 1 ROTATE IMAGE (Transpose + Row Reversal) e.g. Rotate Image (LC 48)

WHEN TO USE

Rotate N \times N 2D matrix 90° clockwise in-place (O(1) space).

TEMPLATE — LC 48

```
public void rotate(int[][] matrix) {
  int n = matrix.length;
  // Step 1: Transpose matrix (swap matrix[i][j] with matrix[j]
[i])
  for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
      int tmp = matrix[i][j]; matrix[i][j] = matrix[j][i]; matrix
[j][i] = tmp;
    }
  }
  // Step 2: Reverse each row
  for (int i = 0; i < n; i++) {
    for (int l = 0, r = n - 1; l < r; l++, r--) {
      int tmp = matrix[i][l]; matrix[i][l] = matrix[i][r]; matrix
[i][r] = tmp;
    }
  }
}
```

PATTERN 2 SPIRAL MATRIX (4-Boundary Shrinking Loop) e.g. Spiral Matrix (LC 54)

BOUNDARY SHRINK LOGIC

Traverse top row, right col, bottom row, left col. Increment `top++`, `left++`, decrement `bot--`, `right--`!

TEMPLATE — LC 54

```
public List<Integer> spiralOrder(int[][] matrix) {
  List<Integer> res = new ArrayList<>();
  int top = 0, bot = matrix.length - 1, left = 0, right = matrix
[0].length - 1;
  while (top <= bot && left <= right) {
    for (int c = left; c <= right; c++) res.add(matrix[top][c]); t
op++;
    for (int r = top; r <= bot; r++) res.add(matrix[r][right]); ri
ght--;
    if (top <= bot) { for (int c = right; c >= left; c--) res.add
(matrix[bot][c]); bot--; }
    if (left <= right) { for (int r = bot; r >= top; r--) res.add
(matrix[r][left]); left++; }
  }
  return res;
}
```

PATTERN 5 PLUS ONE (Backwards Carry Propagation) e.g. Plus One (LC 66)

CARRY PROPAGATION

Scan backwards. If digit < 9, increment and return! If all digits were 9 (e.g. 999), return array `[1, 0, 0, 0]`.

TEMPLATE — LC 66

```
public int[] plusOne(int[] digits) {
  for (int i = digits.length - 1; i >= 0; i--) {
    if (digits[i] < 9) { digits[i]++; return digits; }
    digits[i] = 0;
  }
  int[] res = new int[digits.length + 1];
  res[0] = 1;
  return res;
}
```

PATTERN 6 MULTIPLY STRINGS (Position Index Product Array) e.g. Multiply Strings (LC 43)

POSITION MATH (`I + J + 1`)

Product of `num1[i]` and `num2[j]` maps to positions `p1 = i + j` and `p2 = i + j + 1` in result array of size M + N.

TEMPLATE — LC 43

```
public String multiply(String num1, String num2) {
  int m = num1.length(), n = num2.length();
  int[] pos = new int[m + n];
  for (int i = m - 1; i >= 0; i--) {
    for (int j = n - 1; j >= 0; j--) {
      int mul = (num1.charAt(i) - '0') * (num2.charAt(j) - '0');
      int p1 = i + j, p2 = i + j + 1;
      int sum = mul + pos[p2];
      pos[p1] += sum / 10;
      pos[p2] = sum % 10;
    }
  }
  StringBuilder sb = new StringBuilder();
  for (int p : pos) if (!(sb.length() == 0 && p == 0)) sb.append
(p);
  return sb.length() == 0 ? "0" : sb.toString();
}
```

COMPLETE MATH & GEOMETRY PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
66	Plus One	Backwards Carry	O(N)	O(1)	E
48	Rotate Image	Transpose + Reverse Rows	O(N^2)	O(1)	M
54	Spiral Matrix	4-Boundary Shrinking Loop	O(M · N)	O(1)	M
73	Set Matrix Zeroes	First Row/Col Markers	O(M · N)	O(1)	M
202	Happy Number	Floyd's Fast/Slow Digits	O(log N)	O(1)	E
50	Pow(x, n)	Binary Exponentiation	O(log N)	O(log N)	M
43	Multiply Strings	Product Position Map	O(M · N)	O(M + N)	M
2013	Detect Squares	Diagonal Point Match	O(N)	O(N)	M

LC 66 · Plus One

EASY

Pattern: Backwards Carry

Key Insight: Increment and return if digit < 9.

```
public int[] plusOne(int[] digits)
{
  for (int i = digits.length - 1;
i >= 0; i--) {
    if (digits[i] < 9) { digits[i]
++; return digits; }
    digits[i] = 0;
  }
  int[] res = new int[digits.lengt
h + 1]; res[0] = 1;
  return res;
}
```

LC 48 · Rotate Image

MEDIUM

Pattern: Transpose + Reverse

Key Insight: `swap(i,j)` then reverse each row.

```
public void rotate(int[][] m) {
  int n = m.length;
  for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j+
+) { int t = m[i][j]; m[i][j] = m
[j][i]; m[j][i] = t; }
  for (int i = 0; i < n; i++)
    for (int l = 0, r = n - 1; l <
r; l++, r--) { int t = m[i][l]; m
[i][l] = m[i][r]; m[i][r] = t; }
}
```

LC 54 · Spiral Matrix

MEDIUM

Pattern: 4-Boundary Loop

Key Insight: Shrink `top`, `bot`, `left`, `right`.

```
public List<Integer> spiralOrder(i
nt[][] m) {
  List<Integer> res = new ArrayLis
t<>();
  int t = 0, b = m.length - 1, l =
0, r = m[0].length - 1;
  while (t <= b && l <= r) {
    for (int c = l; c <= r; c++) r
es.add(m[t][c]); t++;
    for (int row = t; row <= b; ro
w++) res.add(m[row][r]); r--;
    if (t <= b) { for (int c = r;
c >= l; c--) res.add(m[b][c]); b-
-; }
    if (l <= r) { for (int row =
b; row >= t; row--) res.add(m[row]
[l]); l++; }
  }
  return res;
}
```

LC 73 · Set Matrix Zeroes

MEDIUM

Pattern: First Row/Col Marker
Key Insight: Use row 0 & col 0 as flag storage.

```
public void setZeroes(int[][] m) {
  int R = m.length, C = m[0].lengt
h; boolean r0 = false, c0 = false;
  for (int r = 0; r < R; r++) if (m
[r][0] == 0) c0 = true;
  for (int c = 0; c < C; c++) if (m
[0][c] == 0) r0 = true;
  for (int r = 1; r < R; r++)
    for (int c = 1; c < C; c++)
      if (m[r][c] == 0) { m[r][0] =
0; m[0][c] = 0; }
  for (int r = 1; r < R; r++)
    for (int c = 1; c < C; c++)
      if (m[r][0] == 0 || m[0][c] =
= 0) m[r][c] = 0;
  if (c0) for (int r = 0; r < R; r+
+) m[r][0] = 0;
  if (r0) for (int c = 0; c < C; c+
+) m[0][c] = 0;
}
```

LC 50 · Pow(x, n)

MEDIUM

Pattern: Binary Exponentiation
Key Insight: `fastPow(x, n/2)`. Cast `long N = n`.

```
public double myPow(double x, int
n) {
  long N = n; if (N < 0) { x = 1 /
x; N = -N; }
  return fastPow(x, N);
}
private double fastPow(double x, lo
ng n) {
  if (n == 0) return 1.0;
  double half = fastPow(x, n / 2);
  return (n % 2 == 0) ? half * half
: half * half * x;
}
```

LC 43 · Multiply Strings

MEDIUM

Pattern: Product Position Map
Key Insight: `num1[i] * num2[j]` maps to `i+j` and `i+j+1`.

```
public String multiply(String num1, S
tring num2) {
  int m = num1.length(), n = num2.len
gth();
  int[] pos = new int[m + n];
  for (int i = m - 1; i >= 0; i--) {
    for (int j = n - 1; j >= 0; j--)
{
      int mul = (num1.charAt(i) -
'0') * (num2.charAt(j) - '0');
      int p1 = i + j, p2 = i + j + 1,
sum = mul + pos[p2];
      pos[p1] += sum / 10; pos[p2] =
sum % 10;
    }
  }
  StringBuilder sb = new StringBuilde
r();
  for (int p : pos) if (!(sb.length()
== 0 && p == 0)) sb.append(p);
  return sb.length() == 0 ? "0" : sb.
toString();
}
```

LC 2013 · Detect Squares

MEDIUM

Pattern: Diagonal Match
Key Insight: Match `|px - x| == |py - y| > 0`.

```
public int count(int[] point) {
  int px = point[0], py = point[1], a
ns = 0;
  for (int[] pt : points) {
    int x = pt[0], y = pt[1];
    if (Math.abs(px - x) != Math.abs
(py - y) || px == x || py == y) conti
nue;
    ans += count.getOrDefault(px +
"@" + y, 0) * count.getOrDefault(x +
"@" + py, 0);
  }
  return ans;
}
```

🔍 DRY RUN — Rotate Image 90° Clockwise

Step	Input State	Operation	Output State
1	`[[1,2],[3,4]]`	Transpose `swap(matrix[0][1], matrix[1][0])`	`[[1,3],[2,4]]`
2	`[[1,3],[2,4]]`	Reverse row 0: `[1, 3] -> [3, 1]`	`[[3,1],[2,4]]`
3	`[[3,1],[2,4]]`	Reverse row 1: `[2, 4] -> [4, 2]`	`[[3,1],[4,2]]` ✅

📐 MATHEMATICAL PROOF — 90° CLOCKWISE MATRIX ROTATION

Claim: Transposing a matrix and reversing each row rotates the matrix 90° clockwise.

Proof: Transpose maps element $(r, c) -> (c, r)$. Reversing rows maps element $(c, r) -> (c, N - 1 - r)$. Under 90° clockwise rotation, original coordinate $(r, c) moves to (c, N - 1 - r) ->$ **Exact Mathematical Identity!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Rotate matrix 90° in-place with Transpose + Reverse
- ☐ Code Spiral Matrix 4-boundary loop
- ☐ Set Matrix Zeroes in O(1) space with row/col 0 flags
- ☐ Code Pow(x, n) via binary exponentiation
- ☐ Write Happy Number with Floyd's fast/slow digits
- ☐ Code Plus One backwards carry propagation
- ☐ Write Multiply Strings position index mapping
- ☐ Prove why Transpose + Reverse = 90° Rotation

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Cast Negative Powers to Long

In `Pow(x, n)`, if $n = -2147483648$, taking `-n` causes 32-bit signed integer overflow! Cast to `long N = n` before negation!

Rule 2 — Guard Extra Loops in Spiral Matrix

In Spiral Matrix, check `if (top <= bot)` before scanning bottom row and `if (left <= right)` before scanning left column to prevent duplicate row scans on odd dimensions!

ADVANCED DATA STRUCTURES

FAANG Master Guide — Java Edition

💡 WHY ADVANCED DATA STRUCTURES?

Standard data structures break down under dynamic constraints. Advanced structures combine multiple techniques (e.g., Doubly Linked List + HashMap for LRU Cache) to guarantee **O(1) or O(log N) operations** under complex workloads!

💡 THE INVERSE ACKERMANN AHA

Disjoint Set Union (DSU) with **Path Compression** and **Union by Rank** reduces `find()` and `union()` operations to near-constant alpha(N) ≈ O(1) time!

1 4 CORE ADVANCED STRUCTURES

- **Union-Find (DSU):** Dynamic connectivity, cycle detection in O(alpha(N)).
- **Segment Tree:** Range minimum / sum queries & point updates in O(\log N).
- **Fenwick Tree (BIT):** Prefix sums and updates in O(\log N) with zero tree overhead.
- **LRU Cache:** Doubly Linked List + HashMap for O(1) get/put eviction.

2 VISUALIZING PATH COMPRESSION

Before vs After Path Compression:

```
Before: 4 → 3 → 2 → 1 (roo
t)
find(4) flattens tree!
After:  4 → 1, 3 → 1, 2 → 1
Tree height becomes 1!
```

📚 PREREQUISITES & COMPLEXITY REASONING

Data Structure	Query Time	Update Time	Space
Union-Find (DSU)	**O(α(N)) ≈ O(1)**	**O(α(N)) ≈ O(1)**	**O(N)**
Segment Tree	**O(log N) range**	**O(log N) point**	**O(4N) array**
Fenwick Tree (BIT)	**O(log N) prefix**	**O(log N) point**	**O(N) array**
LRU Cache	**O(1) get**	**O(1) put**	**O(Capacity)**

⚡ CLUES THAT SCREAM ADVANCED DS

1. *"Dynamic graph connectivity / Detect cycle in undirected graph"* → Union-Find (DSU)
2. *"Dynamic range sum queries with point updates"* → Segment Tree or Fenwick Tree
3. *"Design O(1) cache with least recently used eviction"* → LRU Cache (DLL + Map)

PATTERN 1 — UNION-FIND CYCLE DETECTION (Redundant Connection)

e.g. Redundant Connection (LC 684), Number of Connected Components

CYCLE DETECTION MECHANICS

For each edge (u, v), if `union(u, v) == false` (meaning `find(u) == find(v)`), the current edge creates a cycle and is redundant!

TEMPLATE — LC 684

```
public int[] findRedundantConnection(int[][] edges) {
  int n = edges.length;
  DSU dsu = new DSU(n + 1);
  for (int[] edge : edges) {
    if (!dsu.union(edge[0], edge[1])) return edge; // Cycle edge
found!
  }
  return new int[0];
}
```

PATTERN 2 SEGMENT TREE (Range Sum Query & Point Update) e.g. Range Sum Query - Mutable (LC 307)

SEGMENT TREE ARRAY SIZE 4N

Divide range in half recursively. Left child $= 2i+1$, Right child $= 2i+2$. Point updates & range sum queries run in O(\log N)!

TEMPLATE — LC 307

```
class NumArray {
  int[] tree, nums; int n;
  public NumArray(int[] nums) {
    this.nums = nums; n = nums.length;
    if (n > 0) { tree = new int[4 * n]; build(0, 0, n - 1); }
  }
  private void build(int node, int start, int end) {
    if (start == end) { tree[node] = nums[start]; return; }
    int mid = (start + end) / 2;
    build(2 * node + 1, start, mid); build(2 * node + 2, mid + 1,
end);
    tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
  }
  public void update(int index, int val) { updateTree(0, 0, n -
1, index, val); }
  private void updateTree(int node, int start, int end, int idx,
int val) {
    if (start == end) { nums[idx] = val; tree[node] = val; retur
n; }
    int mid = (start + end) / 2;
    if (start <= idx && idx <= mid) updateTree(2 * node + 1, star
t, mid, idx, val);
    else updateTree(2 * node + 2, mid + 1, end, idx, val);
    tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
  }
  public int sumRange(int left, int right) { return query(0, 0, n
- 1, left, right); }
  private int query(int node, int start, int end, int L, int R) {
    if (R < start || end < L) return 0;
    if (L <= start && end <= R) return tree[node];
    int mid = (start + end) / 2;
    return query(2 * node + 1, start, mid, L, R) + query(2 * node
+ 2, mid + 1, end, L, R);
  }
}
```

PATTERN 3 LRU CACHE (Doubly Linked List + HashMap) e.g. LRU Cache (LC 146)

SENTINEL HEAD & TAIL GUARD

HashMap maps `key → Node`. Dummy `head` and `tail` nodes eliminate edge case checks! Most recently used node is moved to head; least recently used at tail is evicted!

TEMPLATE — LC 146

```
class LRUCache {
  class Node { int key, val; Node prev, next; Node(int k, int v)
{ key=k; val=v; } }
  private final int capacity;
  private final Map<Integer, Node> map = new HashMap<>();
  private final Node head = new Node(0, 0), tail = new Node(0,
0);

  public LRUCache(int capacity) {
    this.capacity = capacity; head.next = tail; tail.prev = head;
  }
  public int get(int key) {
    if (!map.containsKey(key)) return -1;
    Node node = map.get(key); remove(node); insertToHead(node);
    return node.val;
  }
  public void put(int key, int value) {
    if (map.containsKey(key)) remove(map.get(key));
    if (map.size() == capacity) { map.remove(tail.prev.key); remo
ve(tail.prev); }
    Node node = new Node(key, value); insertToHead(node); map.put
(key, node);
  }
  private void remove(Node n) { n.prev.next = n.next; n.next.prev
= n.prev; }
  private void insertToHead(Node n) { n.next = head.next; n.next.
prev = n; head.next = n; n.prev = head; }
}
```

PATTERN 4 LFU CACHE (Frequency HashMap + Frequency Lists) e.g. LFU Cache (LC 460)

MIN FREQUENCY STATE

Map `key → Node`. Map `freq → DoublyLinkedList`. Maintain `minFreq`. When capacity reached, evict from `freqMap.get(minFreq).tail.prev`!

TEMPLATE — LC 460

```
class LFUCache {
  class Node { int key, val, freq = 1; Node prev, next; Node(int
k, int v) { key=k; val=v; } }
  class DoubleLinkedList {
    Node head = new Node(0, 0), tail = new Node(0, 0); int size =
0;
    DoubleLinkedList() { head.next = tail; tail.prev = head; }
    void add(Node n) { n.next = head.next; n.next.prev = n; head.
next = n; n.prev = head; size++; }
    void remove(Node n) { n.prev.next = n.next; n.next.prev = n.p
rev; size--; }
    Node removeLast() { if (size == 0) return null; Node n = tai
l.prev; remove(n); return n; }
  }
  int cap, minFreq = 0;
  Map<Integer, Node> keyMap = new HashMap<>();
  Map<Integer, DoubleLinkedList> freqMap = new HashMap<>();

  public LFUCache(int capacity) { cap = capacity; }
  public int get(int key) {
    if (!keyMap.containsKey(key)) return -1;
    Node node = keyMap.get(key); updateFreq(node); return node.va
l;
  }
  private void updateFreq(Node n) {
    DoubleLinkedList oldList = freqMap.get(n.freq); oldList.remov
e(n);
    if (n.freq == minFreq && oldList.size == 0) minFreq++;
    n.freq++;
    freqMap.computeIfAbsent(n.freq, k -> new DoubleLinkedList()).
add(n);
  }
}
```

📊 COMPLETE ADVANCED DATA STRUCTURES PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
684	Redundant Connection	Union-Find DSU	O(N · α(N))	O(N)	M
307	Range Sum Query - Mutable	Fenwick Tree / Segment Tree	O(log N) query/update	O(N)	M
146	LRU Cache	Doubly Linked List + Map	O(1) get/put	O(Capacity)	M
460	LFU Cache	Frequency Map + DLL Lists	O(1) get/put	O(Capacity)	H

💡 MASTERING ADVANCED STRUCTURES — QUICK SUMMARY

1. **DSU:** Use Path Compression (`parent[i] = find(parent[i])`) and Union by Rank for O(alpha(N)) time.
2. **Fenwick Tree (BIT):** Use lowbit `i & -i` to advance or retreat index. `add()` moves forward, `query()` moves backward.
3. **LRU & LFU Caches:** Always use dummy `head` and `tail` nodes in your Doubly Linked List to eliminate `null` pointer checks during node removal!

LC 684 · Redundant Connection — MEDIUM

Pattern: DSU Cycle Check
Key Insight: Return edge if `union == false`.

```
public int[] findRedundantConnection(int[][] edges) {
  DSU dsu = new DSU(edges.length + 1);
  for (int[] e : edges)
    if (!dsu.union(e[0], e[1])) return e;
  return new int[0];
}
```

LC 307 · Range Sum Query - Mutable — MEDIUM

Pattern: Fenwick Tree (BIT)
Key Insight: `add()` and `query()` with `i & -i`.

```
class NumArray {
  int[] tree, nums;
  public NumArray(int[] nums) {
    this.nums = nums; tree = new int[nums.length + 1];
    for (int i = 0; i < nums.length; i++) add(i, nums[i]);
  }
  public void update(int i, int val) { add(i, val - nums[i]); nums[i] = val; }
  private void add(int i, int d) { for (i++; i < tree.length; i += i & -i) tree[i] += d; }
  public int sumRange(int l, int r) { return q(r) - q(l - 1); }
  private int q(int i) { int s = 0; for (i++; i > 0; i -= i & -i) s += tree[i]; return s; }
}
```

LC 146 · LRU Cache — MEDIUM

Pattern: Map + DLL
Key Insight: Move to head on `get`, evict tail on full.

```
public int get(int key) {
  if (!map.containsKey(key)) return -1;
  Node n = map.get(key); remove(n); insertHead(n); return n.val;
}
```

LC 460 · LFU Cache — HARD

Pattern: Freq Map + DLL

Key Insight: Evict from `freqMap.get(minFreq)`.

```
private void updateFreq(Node n)
{
  DoubleLinkedList oldL = freqM
ap.get(n.freq); oldL.remove(n);
  if (n.freq == minFreq && old
L.size == 0) minFreq++;
  n.freq++; freqMap.computeIfAb
sent(n.freq, k→new DoubleLinke
dList()).add(n);
}
```

🔍 DRY RUN — DSU Path Compression (`find(4)`)

Step	Node	parent[i] Before	Action / Compression	parent[i] After
1	4	3	Recurse `find(3)`	1
2	3	2	Recurse `find(2)`	1
3	2	1	Recurse `find(1)` → Root is 1!	1
4	4	3	`parent[4] = 1` (Direct link to root!)	1 ✅

📐 MATHEMATICAL PROOF — LOWBIT OPERATOR (`i & -i`)

Claim: `i & -i` isolates the lowest set bit (power of 2) of integer `i`.

Proof: `-i` is computed as two's complement `~i + 1`. Inverting `i` flips all bits. Adding 1 flips trailing 1s back to 0 until reaching the first 0 (which was the lowest 1 in `i`). Performing `i & (~i + 1)` cancels all higher bits → **Exact Lowest Bit Power of 2!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Write DSU with Path Compression & Union by Rank
- ☐ Code Redundant Connection cycle check
- ☐ Implement Fenwick Tree (BIT) add & query
- ☐ Code Segment Tree build, update, & range query
- ☐ Implement LRU Cache (DLL + HashMap) in 10m
- ☐ Write LFU Cache min-frequency map eviction
- ☐ Use dummy head/tail sentinels for DLL
- ☐ Prove why `i & -i` extracts lowbit

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Always Perform Path Compression

In DSU `find()`, write `return parent[i] = find(parent[i])`! Without assignment, trees can degrade to O(N) height!

Rule 2 — Always Check Map Size in LRU Put

In `LRUCache.put()`, if key ALREADY EXISTS, remove old node first BEFORE checking if `map.size() == capacity`!

CH 20: INTERVIEW MINDSET

How FAANG interviewers think • Evaluation rubric • What gets you hired or rejected

1 HOW INTERVIEWERS THINK

They spend 45 mins deciding:

- **Can this person break down a problem?**
- **Can they communicate their thinking?**
- **Do they handle ambiguity well?**
- **Can they write clean production code?**
- **Do they verify their own work?**
- **Are they a Senior Engineer or an order-taker?**

The interview is a pair-programming session disguised as an exam. Your job is to make the interviewer's job easy.

2 THE 4 FAANG EVALUATION PILLARS

Pillar	What They Look For	✅ Green Flag	❌ Red Flag
Problem Solving	Pattern recognition, trade-off analysis	Clarifies before coding	Jumps straight to code
Technical Skill	Correct DS choice, clean Java code	Modular functions, good names	Monolithic spaghetti
Verification	Proactive dry run, edge case thinking	Traces manually, finds bugs	"It looks right to me"
Communication	Thinks out loud, receptive to hints	Narrates reasoning constantly	Silent 10+ min coding

3 HIRING RUBRIC SCORES

Score	Verdict	Signal
1	No Hire	Couldn't solve or communicate
2	Lean No	Partial solve, weak communication
3	Lean Yes	Solved with hints, OK communication
4	**Strong Hire**	Optimal + clean code + narrated fully

💡 Score 4 = interviewer writes: *"Would love to work with them."*

4 5 INSTANT REJECTION TRIGGERS

❌ **Silent coding for 10+ minutes.** Interviewers can't evaluate what they can't hear. Even wrong ideas spoken out loud are better than silence.

❌ **Jumping to code before clarifying.** Interviewer: "The problem said integers — did you ask if they could be negative?"

❌ **"I'm done" without dry-running.** Submitting buggy code with confidence is worse than finding your own bug.

❌ **Arguing against interviewer hints.** When they say "what about this edge case?" — they are giving you the answer. Listen!

❌ **Using variable names like a, b, temp2, x1.** `leftPtr`, `maxSumSoFar`, `visitedNodes` signal senior-level thinking.

5 WHAT IMPRESSES FAANG INTERVIEWERS

✅ **Proactively names constraints before being asked** — "I'll assume N ≤ 10^5, so O(N log N) is fine"

✅ **Compares two approaches** — "Heap vs QuickSelect: heap is O(N log K), QuickSelect O(N) avg"

✅ **Gets interviewer agreement before coding** — "Does this approach make sense before I code it?"

✅ **Writes defensive guards at top** — null check, empty array check before main logic

✅ **Dry-runs with a specific example** — traces pointer values step by step on paper/screen

✅ **Announces follow-ups unprompted** — "If the input was a stream, I'd use a sliding window instead"

⏳ **GOLDEN RULE:** Every minute you speak, every constraint you clarify, every trade-off you name = signal. Silence = no signal. No signal = No Hire.

👑 MID vs SENIOR vs STAFF EXPECTATIONS

The exact behavioral differences that determine your level

1 LEVEL EXPECTATION COMPARISON MATRIX

Rubric Dimension	Mid-Level (L4 / IC4)	Senior Engineer (L5 / IC5)	Staff Engineer (L6 / IC6)
🔍 **Requirement Clarification**	Asks basic: "Is input sorted?"	Asks: constraints, scale N, edge cases, return type for invalid	Identifies hidden scale (streaming, distributed), asks about memory limits, SLA
💡 **Algorithm Choice**	Proposes first working approach	Compares 2-3 trade-offs explicitly: Heap vs QuickSelect, BFS vs DFS	Proposes optimal + discusses cache locality, concurrency, and real-world gotchas
💻 **Code Structure**	Monolithic single method, ad-hoc names	Modular helpers, Java records, descriptive names, handles overflow	Production-grade: extensible interfaces, handles all error cases, comments WHY
🧪 **Verification / Testing**	Tests happy path when asked	Proactively dry-runs edge cases: empty, single element, all-same, max N	Constructs adversarial tests: worst-case input, stress test thinking, invariant checking
🗣️ **Communication Style**	Responds to questions	Thinks out loud continuously, narrates every decision	Leads the interview gracefully, guides discussion, makes interviewer feel like a colleague
📈 **Proactiveness**	Solves exactly what is asked	Asks "what if N is 10^9?" before interviewer does	Reframes the problem: "Is this actually a shortest path on a graph of states?"
🔁 **Follow-up Handling**	Waits for follow-up question	Proactively says "a harder version of this would be..."	Presents 3 alternative approaches with trade-offs, discusses system-level implications
🏗️ **Engineering Ownership**	Implementer mentality	Takes ownership of correctness and efficiency	Thinks about team impact, API design, long-term maintainability

2 SAME QUESTION — L4 vs L5 RESPONSE

Question: "Find the k-th largest element in an array."

L4 Candidate:
"I'll sort the array and return arr[n-k]." *(begins coding immediately)*

L5 Candidate:
"Before I code — a few clarifications. Can the input have duplicates? Are elements guaranteed integers? What is the range of k — is k always valid?

For approach: sorting gives O(N log N). A better approach is a min-heap of size k — O(N log K) time, O(K) space. QuickSelect can do O(N) average but $O(N^2)$ worst-case. Given no worst-case guarantee needed, I'll use the heap approach for predictability. Does that sound good?"

3 COMMON MISTAKES BY LEVEL

L4 Mistakes (getting rejected for L5):

- Solves the easy version without checking constraints
- Never mentions time/space until asked
- Thinks "it works" = done (no dry run)
- Doesn't handle null / empty / single element

L5 Mistakes (getting rejected for L6):

- Solves correctly but doesn't proactively scale
- Doesn't anticipate follow-ups before being asked
- Presents one solution without comparing alternatives
- Good code but doesn't discuss extensibility / API design

📌 **KEY INSIGHT:** To pass as L5, demonstrate engineering ownership from the first 30 seconds. To pass as L6, make the interviewer feel they're collaborating with a peer, not evaluating a candidate.

🎯 CH 21: PATTERN RECOGNITION MASTERY

30-second recognition • Constraint mapping • Hidden keywords • Wrong pattern traps

1 THE 30-SECOND RECOGNITION SYSTEM

Step 1: Read problem → underline nouns (input type) and verbs (operation type)
Step 2: Look at N constraint → maps to max allowed time complexity
Step 3: Match keyword phrases → map to pattern family
Step 4: Recall generic template → customize for problem

CONSTRAINT → MAX COMPLEXITY MAPPING

N Constraint	Max Complexity	Pattern Hint
N ≤ 20	$O(2^N)$ or O(N!)	Backtracking, Bitmask DP
N ≤ 100	$O(N^3)$	3D DP, Floyd-Warshall
N ≤ 1,000	$O(N^2)$	2D DP, Bubble Sort
N ≤ 100,000	O(N log N)	Sort + Heap + Binary Search
N ≤ 10,000,000	O(N)	Two Pointers, Sliding Window, Prefix Sum
N ≤ 10^{18}	O(log N)	Binary Search on Answer, Math

2 HIDDEN KEYWORD → PATTERN MAP

If you see these words...	Think this pattern
"contiguous subarray", "max/min sum"	Sliding Window / Prefix Sum
"two numbers that sum to target"	Two Pointers or HashMap
"sorted array", "find index of"	Binary Search
"top K", "K largest/smallest", "frequent"	Min-Heap of size K
"shortest path", "fewest steps"	BFS (unweighted) or Dijkstra
"all combinations", "all subsets", "generate all"	Backtracking
"minimum cost", "optimal choice at each step"	Greedy or DP
"overlapping subproblems", "how many ways"	Dynamic Programming
"merge intervals", "schedule", "overlap"	Intervals + Sort by start
"connected components", "union", "same group"	Union-Find (DSU)
"course prerequisites", "dependency order"	Topological Sort
"word prefix", "autocomplete", "starts with"	Trie
"parentheses", "valid brackets", "daily temperature"	Monotonic Stack
"range query", "point update"	Segment Tree / Fenwick BIT

3 COMPLETE PATTERN RECOGNITION MATRIX (15 CORE PATTERNS)

Pattern	Trigger Signal	Key DS	Time	Space	Classic Problems
Two Pointers	Sorted array, pair finding, palindrome	int left, right	O(N)	O(1)	LC 15, 125, 167, 11
Sliding Window	Contiguous subarray/substring, max/min K window	int[128] freq, l/r pointers	O(N)	O(1)	LC 3, 76, 424, 567
Binary Search	Sorted space, minimise/maximise feasibility	int lo, hi, mid	O(log N)	O(1)	LC 33, 875, 410, 4
Prefix Sum	Subarray sum = k, range queries	int[] prefix, HashMap	O(N)	O(N)	LC 560, 523, 1248
Monotonic Stack	Next greater/smaller, histogram bars	Deque<Integer>	O(N)	O(N)	LC 739, 84, 85, 42
Top-K Heap	K largest/smallest/frequent	PriorityQueue min-heap	O(N log K)	O(K)	LC 215, 347, 295, 973
Tree DFS	Tree path, depth, subtree, LCA	Recursion stack	O(N)	O(H)	LC 104, 124, 236, 543
Tree BFS	Level order, row by row, zigzag	Queue<TreeNode>	O(N)	O(W)	LC 102, 199, 103
Graph DFS/BFS	Connected components, flood fill, island	boolean[] visited	O(V+E)	O(V)	LC 200, 695, 130
Topo Sort	Dependencies, DAG ordering	int[] indegree, Queue	O(V+E)	O(V)	LC 207, 210, 269
Union-Find	Grouping, connectivity, cycle detection	int[] parent, rank	O(α(N))	O(N)	LC 547, 684, 1319
1D DP	Min/max cost, count ways, linear state	int[] dp	O(N)	O(N)	LC 70, 198, 300, 139
2D DP	Grid paths, string matching, knapsack	int[][] dp	O(N·M)	O(N·M)	LC 62, 1143, 72, 416
Backtracking	All combinations, permutations, N-Queens	List<T> path, recursion	$O(2^N)$ / O(N!)	O(N)	LC 78, 39, 46, 51
Greedy + Intervals	Merge, schedule, min arrows, activity	Arrays.sort + scan	O(N log N)	O(N)	LC 56, 435, 452, 253

4 WRONG PATTERN TRAPS — PIVOT IMMEDIATELY

Looks Like	Actually Is	Failure Signal	Pivot To
Greedy (Coin Change)	DP (Unbounded Knapsack)	Non-canonical coins → greedy fails	dp[i] = min(dp[i], dp[i-coin]+1)
DFS (Shortest Path)	BFS Level Order	DFS explores deep first, misses short	Queue + visited[] BFS
Sort (Top K)	Min-Heap size K	Sort is O(N log N), heap is O(N log K)	PriorityQueue minHeap
DP (Jump Game)	Greedy (Farthest Reach)	DP is $O(N^2)$, greedy O(N)	maxReach = max(maxReach, i+nums[i])

5 WHEN NOT TO USE — COMMON CONFUSIONS

- ❌ **Binary Search on unsorted array** — must sort first or use Two Pointers
- ❌ **DFS for shortest path** — always use BFS for unweighted shortest path
- ❌ **Greedy on Coin Change with arbitrary coins** — use DP
- ❌ **Backtracking when N > 20** — exponential, look for DP or Greedy
- ❌ **Two Pointers on unsorted array** — only valid on sorted or monotone arrays
- ❌ **Union-Find for directed graphs** — only correct for undirected connectivity

TREE + GRAPH TEMPLATE LIBRARY

All 8 Tree Templates + 11 Graph Templates — the complete reusable set

T1 DFS — PREORDER / INORDER / POSTORDER

```
// PREORDER: Root → Left → Right
void preorder(TreeNode node) {
    if (node == null) return;
    process(node);    // ← Root FIRST
    preorder(node.left);
    preorder(node.right);
}
// INORDER: Left → Root → Right (BST = sorted!)
void inorder(TreeNode node) {
    if (node == null) return;
    inorder(node.left);
    process(node);    // ← Root MIDDLE
    inorder(node.right);
}
// POSTORDER: Left → Right → Root
int postorder(TreeNode node) {
    if (node == null) return 0;
    int left = postorder(node.left);
    int right = postorder(node.right);
    return combine(left, right, node); // ← Root LAST
}
```

💡 Preorder = serialize. Inorder on BST = sorted. Postorder = compute from leaves up.

T2 BFS — LEVEL ORDER

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size(); // FREEZE level size
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            level.add(node.val);
            if (node.left ≠ null) q.offer(node.left);
            if (node.right ≠ null) q.offer(node.right);
        }
        res.add(level);
    }
    return res;
}
```

T3 BOTTOM-UP: HEIGHT / BALANCED / DIAMETER

```
// HEIGHT (returns int)
int height(TreeNode node) {
    if (node == null) return 0;
    int L = height(node.left);
    int R = height(node.right);
    return 1 + Math.max(L, R); // ← combine
}
// BALANCED (returns bool)
boolean isBalanced(TreeNode node) {
    return checkHeight(node) ≠ -1;
}
int checkHeight(TreeNode node) {
    if (node == null) return 0;
    int L = checkHeight(node.left);
    if (L == -1) return -1;
    int R = checkHeight(node.right);
    if (R == -1) return -1;
    if (Math.abs(L - R) > 1) return -1;
    return 1 + Math.max(L, R);
}
// DIAMETER (global max with postorder)
int maxDia = 0;
int diameter(TreeNode node) {
    if (node == null) return 0;
    int L = diameter(node.left);
    int R = diameter(node.right);
    maxDia = Math.max(maxDia, L + R);
    return 1 + Math.max(L, R);
}
```

T5 BST — SEARCH / VALIDATE / LCA

```
// BST SEARCH
TreeNode search(TreeNode root, int target) {
    if (root == null || root.val == target) return root;
    return target < root.val ?
        search(root.left, target) :
        search(root.right, target);
}
// VALIDATE BST (pass min/max bounds)
boolean validate(TreeNode node, long min, long max) {
    if (node == null) return true;
    if (node.val ≤ min || node.val ≥ max) return false;
    return validate(node.left, min, node.val) &&
           validate(node.right, node.val, max);
}
// LCA (Binary Tree)
TreeNode lca(TreeNode node, TreeNode p, TreeNode q) {
    if (node == null || node == p || node == q) return node;
    TreeNode L = lca(node.left, p, q);
    TreeNode R = lca(node.right, p, q);
    return (L ≠ null && R ≠ null) ? node : (L ≠ null ? L : R);
}
```

G1 BUILD GRAPH — ALL TYPES

```
// UNDIRECTED GRAPH from edge list
List<List<Integer>> buildGraph(int n, int[][] edges) {
    List<List<Integer>> g = new ArrayList<>();
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    for (int[] e : edges) {
        g.get(e[0]).add(e[1]);
        g.get(e[1]).add(e[0]); // remove for directed
    }
    return g;
}
// WEIGHTED GRAPH
List<List<int[]>> buildWeighted(int n, int[][] edges) {
    List<List<int[]>> g = new ArrayList<>();
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    for (int[] e : edges)  // e = [from, to, weight]
        g.get(e[0]).add(new int[]{e[1], e[2]});
    return g;
}
// GRID UTILITIES
int[][] DIR4 = {{-1,0},{1,0},{0,-1},{0,1}};
int[][] DIR8 = {{-1,-1},{-1,0},{-1,1},{0,-1},{0,1},{1,-1},{1,0},{1,1}};
boolean isValid(int r, int c, int rows, int cols) {
    return r ⩾ 0 && r < rows && c ⩾ 0 && c < cols;
}
```

G3 BFS + DIJKSTRA

```
// BFS (unweighted shortest path)
int bfs(List<List<Integer>> g, int src, int dst) {
    int[] dist = new int[g.size()];
    Arrays.fill(dist, -1); dist[src] = 0;
    Queue<Integer> q = new ArrayDeque<>();
    q.offer(src);
    while (!q.isEmpty()) {
        int u = q.poll();
        if (u == dst) return dist[u];
        for (int v : g.get(u)) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.offer(v);
            }
        }
    }
    return -1;
}
// DIJKSTRA (weighted)
int[] dijkstra(List<List<int[]>> g, int src) {
    int[] dist = new int[g.size()];
    Arrays.fill(dist, Integer.MAX_VALUE); dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) → a[0]-b[0]);
    pq.offer(new int[]{0, src});
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int d = cur[0], u = cur[1];
        if (d > dist[u]) continue; // stale
        for (int[] e : g.get(u)) {
            int newDist = dist[u] + e[1];
            if (newDist < dist[e[0]]) {
                dist[e[0]] = newDist;
                pq.offer(new int[]{newDist, e[0]});
            }
        }
    }
    return dist;
}
```

G9 UNION-FIND (DSU) — PATH COMPRESSION + UNION BY RANK

```
int[] parent, rank;
void init(int n) { parent = new int[n]; rank = new int[n];
    for (int i = 0; i < n; i++) parent[i] = i; }
int find(int x) { // path compression
    if (parent[x] ≠ x) parent[x] = find(parent[x]);
    return parent[x]; }
boolean union(int x, int y) { // union by rank
    int px = find(x), py = find(y);
    if (px == py) return false; // already same component (cycle!)
    if (rank[px] < rank[py]) { int t = px; px = py; py = t; }
    parent[py] = px;
    if (rank[px] == rank[py]) rank[px]++;
    return true; }
```

G8 TOPOLOGICAL SORT — KAHN'S ALGORITHM

```
List<Integer> topoSort(int n, int[][] prereqs) {
    List<List<Integer>> g = new ArrayList<>();
    int[] indegree = new int[n];
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    for (int[] p : prereqs) {
        g.get(p[1]).add(p[0]);
        indegree[p[0]]++;
    }
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < n; i++) if (indegree[i] == 0) q.offer(i);
    List<Integer> order = new ArrayList<>();
    while (!q.isEmpty()) {
        int u = q.poll(); order.add(u);
        for (int v : g.get(u))
            if (--indegree[v] == 0) q.offer(v);
    }
    return order.size() == n ? order : new ArrayList<>(); // empty = cycle
}
```

🛠️ CH 22: THE COMPLETE PROBLEM SOLVING FRAMEWORK

7-step workflow • Time budget • Clarification scripts • Full worked example

1 THE 7-STEP WORKFLOW + TIME BUDGET

Min	Step	Goal
0–3	🔍 **Clarify + Constraints**	N range, negative ints, return type, edge cases
3–7	📝 **Work Examples**	Concrete 3-4 element example, trace manually
7–12	🔨 **Brute Force + Why It Fails**	State O(N²), explain bottleneck
12–17	💡 **Optimal Insight + Pitch**	Name data structure, get interviewer nod
17–35	💻 **Write Clean Java Code**	Modular, defensive guards first
35–42	▶️ **Dry Run Edge Cases**	Empty, single, duplicates, max N
42–45	📊 **Complexity + Follow-ups**	Time/space analysis, scale question

💡 **Never start coding before minute 17.** Agreement on approach = no wasted code resets.

2 CLARIFYING QUESTIONS SCRIPT

For ARRAYS:

- "Can input contain negative numbers or duplicates?"
- "Is the array sorted? Can I sort it?"
- "What is the max value of N? Max element value?"

For GRAPHS:

- "Is the graph directed or undirected?"
- "Can it have cycles? Multiple components?"
- "Are edge weights positive or can they be negative?"

For STRINGS:

- "Only lowercase letters, or ASCII or Unicode?"
- "Is the string empty a valid input?"

For DESIGN:

- "What are the most frequent operations — get vs put?"
- "Thread-safe? Persistence? Memory constraints?"

3 FULL WORKED EXAMPLE: LC 1 — TWO SUM (L5-Level Execution)

Interviewer: "Given an integer array nums and target, return indices of two numbers that add to target."

Candidate: "Before I code — quick clarifications: Can there be multiple valid answers? Is there exactly one solution guaranteed? Can we reuse the same element? And what's the range of N and element values?"

Interviewer: "Exactly one solution. No reuse. N ≤ 10⁴."

Candidate: "Got it. Brute force would be O(N²) with two nested loops — too slow for N=10⁴ in a tight time limit.

Optimal insight: use a HashMap to store complement. For each num, check if (target - num) already exists in the map. If yes, return [map.get(complement), i]. Otherwise, store nums[i] → i. This is O(N) time, O(N) space. Does that sound good?"

Interviewer: "Yes, go ahead."

```
// GUARD: always handle edge cases first
public int[] twoSum(int[] nums, int target) {
    if (nums == null || nums.length < 2)
        return new int[]{};

    // complement → index mapping
    Map<Integer, Integer> seen = new HashMap<>();

    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (seen.containsKey(complement)) {
            // DRY RUN: nums=[2,7,11,15], target=9
            // i=0: complement=7, seen={} → miss
            // i=1: complement=2, seen={2:0} → HIT!
            return new int[]{seen.get(complement), i};
        }
        seen.put(nums[i], i);
    }
    return new int[]{}; // never reached per problem spec
}
// Time: O(N) — one pass, O(1) HashMap lookup
// Space: O(N) — at most N entries in HashMap
// Edge: nums=[3,3], target=6 → works! (different indices)
```

▶️ DRY RUN TABLE: nums=[2,7,11,15], target=9

i	nums[i]	complement	seen contains?	action
0	2	7	❌ No	put 2→0
1	7	2	✅ Yes! (index 0)	return [0,1]

🔁 FOLLOW-UP QUESTIONS:

- "What if there can be multiple pairs?" → collect all, return List<int[]>
- "What if array is sorted?" → Two Pointers, O(1) space!
- "What if input is a data stream?" → Keep HashMap, process one-by-one
- "What if we need 3Sum?" → Sort + Two Pointers outer loop

💡 **Always volunteer follow-ups!** It signals Senior+ thinking.

CH 25: JAVA INTERVIEW TOOLKIT + TEMPLATE LIBRARY

Collections cheat sheet • Utility APIs • Comparators • DP + BS templates

1 JAVA COLLECTIONS MASTER REFERENCE

Collection	Key Operations	Time	Interview Use	⚠️ Gotcha
HashMap<K,V>	put, get, getOrDefault, containsKey, computeIfAbsent	O(1) avg	Frequency, complement lookup, grouping	getOrDefault not same as get+putIfAbsent
HashSet<T>	add, contains, remove	O(1) avg	Dedup, existence check, visited	add() returns false if already present
TreeMap<K,V>	floorKey, ceilingKey, firstKey, lastKey, subMap	O(log N)	Ordered range queries, calendar, skyline	NavigableMap API not in HashMap!
TreeSet<T>	floor, ceiling, lower, higher, headSet, tailSet	O(log N)	K-th element, range existence	floor/ceiling return null if not found
PriorityQueue	offer, poll, peek	O(log N)	Top-K, Dijkstra, two-heap median	Default is MIN heap! Use (a,b)→b-a for MAX
ArrayDeque	push/pop (Stack), offer/poll (Queue), peekFirst/Last	O(1)	Stack, Queue, Monotonic DS	Faster than LinkedList and Stack class
Arrays utility	sort, binarySearch, fill, copyOf, toString	O(N log N) sort	Sort + BS, initialize arrays	Use Integer.compare(a,b) not a-b (overflow!)

2 COMPARATOR / LAMBDA QUICK REFERENCE

```
// MIN HEAP (default)
PriorityQueue<Integer> minPQ = new PriorityQueue<>();
// MAX HEAP
PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Collections.reverseOrder());
// Sort by second element (SAFE — no overflow)
PriorityQueue<int[]> pq = new PriorityQueue<>((a,b)→Integer.compare(a[1],b[1]));
// Sort array of arrays by first col asc, then second desc
Arrays.sort(intervals, (a,b) → a[0]==b[0] ? b[1]-a[1] : a[0]-b[0]);
// Sort strings by length, then alphabetically
list.sort(Comparator.comparingInt(String::length).thenComparing(Comparator.naturalO
rder()));
// GroupBy (anagram key)
Map<String, List<String>> map = new HashMap<>();
map.computeIfAbsent(key, k → new ArrayList<>()).add(word);
```

3 JAVA TYPE CONVERSION CHEAT SHEET

From	To	Java Code
String	char[]	`s.toCharArray()`
char[]	String	`new String(arr)`
int	String	`String.valueOf(n)`
String	int	`Integer.parseInt(s)`
char	int (index)	`c - 'a'`
int	char	`(char)('a' + i)`
List<Integer>	int[]	`list.stream().mapToInt(x→x).toArray()`
int[]	List	`Arrays.stream(arr).boxed().collect(Collectors.toList())`

4 DP TEMPLATE — MEMO + TABULATION

```
// ═══ TOP-DOWN MEMOIZATION ═══
Map<String, Integer> memo = new HashMap<>();
int dp(int i, int j, String s, String t) {
    if (i == s.length()) return 0; // base case
    if (j == t.length()) return s.length() - i;
    String key = i + "," + j;
    if (memo.containsKey(key)) return memo.get(key);
    int res;
    if (s.charAt(i) == t.charAt(j)) res = dp(i+1, j+1, s, t);
    else res = 1 + Math.min(dp(i+1,j,s,t), Math.min(dp(i,j+1,s,t), dp(i+1,j+
1,s,t)));
    memo.put(key, res);
    return res;
}
// ═══ BOTTOM-UP TABULATION ═══
// dp[i][j] = min edit distance for s[0..i-1], t[0..j-1]
int editDistance(String s, String t) {
    int n = s.length(), m = t.length();
    int[][] dp = new int[n+1][m+1];
    for (int i = 0; i ≤ n; i++) dp[i][0] = i;
    for (int j = 0; j ≤ m; j++) dp[0][j] = j;
    for (int i = 1; i ≤ n; i++)
        for (int j = 1; j ≤ m; j++)
            dp[i][j] = s.charAt(i-1) == t.charAt(j-1) ? dp[i-1][j-1] :
                1 + Math.min(dp[i-1][j], Math.min(dp[i][j-1], dp[i-1][j-1]));
    return dp[n][m];
}
```

5 BINARY SEARCH — 3 TEMPLATES

```
// ═══ CLASSIC: Find exact target ═══
int binarySearch(int[] arr, int target) {
    int lo = 0, hi = arr.length - 1;
    while (lo ≤ hi) {
        int mid = lo + (hi - lo) / 2; // NEVER (lo+hi)/2 (overflow!)
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
// ═══ LEFT BOUNDARY: First position ≥ target ═══
int lowerBound(int[] arr, int target) {
    int lo = 0, hi = arr.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] < target) lo = mid + 1;
        else hi = mid;          // ← move right boundary LEFT
    }
    return lo;
}
// ═══ BINARY SEARCH ON ANSWER ═══
// Minimize x such that feasible(x) = true
int lo = minPossible, hi = maxPossible;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (feasible(mid)) hi = mid;  // mid works, try smaller
    else lo = mid + 1;            // too small, go bigger
}
// lo = answer
```

💻 CH 23: CODING EXECUTION + DEBUGGING

CoderPad • Whiteboard • Clean code rules • Debugging protocol • Recovery when stuck

1 CODERPAD STRATEGY

- ✅ Keep imports at top: `import java.util.*;`
- ✅ Write a skeleton class + method signature first
- ✅ Use // comments as your coding plan before filling in
- ✅ Call helper methods before implementing them (top-down)
- ✅ Use `System.out.println()` for debug traces during dry run
- ❌ Don't use System.exit() or scanner — not needed
- ❌ Don't use static fields unless asked for design
- ❌ Don't rename variables halfway through (confusing)

💡 Write the function signature + guard clauses + helper stubs BEFORE any logic. It signals organized thinking.

2 CLEAN CODE CHECKLIST (10 RULES)

✓Return type and method name clearly describe purpose
✓First line: null/empty guard (`if (nums == null || nums.length == 0) return ...;`)
✓Descriptive variable names: `leftPtr` not `l`, `maxSoFar` not `m`
✓One responsibility per function — extract helpers for complex logic
✓Use `Math.max()`, `Math.min()` over verbose if-else
✓Use `Integer.compare(a, b)` in comparators — never `a - b` (overflow!)
✓Loop variable names match their domain: `row, col` not `i, j` for grids
✓Explicit base case for recursion at the top
✓Use `long` for sums that may overflow 32-bit int
✓Maximum 25 lines per method — refactor if longer

3 5-STEP RECOVERY WHEN STUCK

1. VERBALIZE: Explain what you know and what you're stuck on

↓

2. DRAW: Sketch a 3-4 element concrete example on paper/screen

↓

3. SIMPLIFY: "What if N=1? What if all elements are the same?"

↓

4. DATASTRUCTURE: "Can sorting / a HashMap / a queue help here?"

↓

5. ASK: "I'm thinking X or Y — does either direction seem promising to you?"

❌ NEVER sit silent for more than 90 seconds. Always verbalize your mental state, even if stuck!

4 DEBUGGING PROTOCOL — WHEN CODE FAILS

The 5-step debug process:

1. **Re-read the failing test case** — what is the expected output?
2. **Trace with print statements** — add System.out.println() for key variables
3. **Check loop bounds** — off-by-one errors are #1 source of bugs
4. **Check null/empty handling** — is the guard at the top correct?
5. **Check data structure initialization** — HashMap cleared? Arrays zeroed?

💡 **Say out loud:** "Let me trace through this specific failing case step by step..." — interviewers love seeing systematic debugging!

5 VARIABLE NAMING GUIDE

❌ Weak Name	✅ Strong Name	Context
l, r	leftPtr, rightPtr	Two Pointers
m	maxSoFar, maxLen	Sliding Window
i, j	row, col	Grid traversal
n	nodeCount, totalNodes	Graph
tmp	prevNode, nextNode	Linked List
h	maxHeap, minHeap	Priority Queue
dp	minCostDp, waysCount	DP table
vis	visitedNodes, seen	Graph DFS/BFS

CH 24: COMMUNICATION MASTERY

Thinking out loud • Senior scripts • Handling hints • Trade-off language

1 SENIOR ENGINEER COMMUNICATION MATRIX — EXACT PHRASES

Interview Stage	L4 Says (❌ Avoid)	L5 Says (✅ Target)	L6 Says (☀️ Aim For)
Opening	"Let me just solve it."	"Before I start — a few clarifications about constraints..."	"Interesting problem — this reminds me of shortest path in a DAG. Let me verify: is the input always a valid DAG?"
Naming approach	"I'll use a map."	"I'll use a HashMap here because I need O(1) complement lookup — this brings time from $O(N^2)$ to O(N)."	"Two approaches: HashMap for O(N) average, or Two Pointers on sorted input for O(N) worst-case with O(1) space. Given no sort cost, HashMap is simpler."
Complexity	"It's O(N)."	"Time is O(N log K) because each of the N elements does at most one heap operation costing O(log K). Space is O(K) for the heap."	"Time O(N log K), space O(K). Note: if K ≈ N, heap approach degrades to O(N log N) — in that case, nth_element (QuickSelect) gives O(N) average with O(1) extra space."
Dry Run	"I think it works."	"Let me trace through my example: nums=[3,1,4,1], target=5. At i=0, complement=2, seen={} miss, store 3→0. At i=1, complement=4..."	"Let me construct an adversarial case: what if all elements are the same? nums=[3,3,3], target=6 — my code handles this correctly because each index is distinct in the map."
Receiving hints	"Oh OK." (continues same approach)	"That's a great point — you're suggesting the sorted input enables Two Pointers? Let me think about that: we could sort in O(N log N) then use O(N) two-pointer scan..."	"Yes, that edge case I hadn't considered — if the input contains Long.MAX_VALUE, my subtraction overflows. Let me fix the guard: use target - nums[i] computed in long first."

2 THINKING OUT LOUD — SCRIPT LIBRARY

When recognizing a pattern:

"This looks like a sliding window problem because we're asked for the longest contiguous subarray satisfying a condition — that's the classic signal."

When comparing approaches:

"I see two ways: sort + two pointers in O(N log N) time and O(1) space, or hash map in O(N) time and O(N) space. Given N is 10^5 and space isn't constrained, I'll go with the hash map for simplicity."

When writing a helper:

"I'll extract this into a helper `isValid(row, col)` to keep the DFS clean — I'll implement it right after."

When stuck:

"I know the brute force, but I'm trying to see if there's a way to avoid the inner loop. Does caching prefix sums help here?"

3 TRADE-OFF DISCUSSION FRAMEWORK

Always structure trade-offs as:

"Approach A gives [Time A] time and [Space A] space. Approach B gives [Time B] time and [Space B] space. Given [constraint from clarification], I'll choose [A or B] because [specific reason]."

Trade-off Dimension	What to Say
Time vs Space	"Trading O(N) space for O(1) better time — worth it here because N ≤ 10^5"
Worst vs Average	"QuickSelect is O(N) average but $O(N^2)$ worst — for guaranteed O(N log K) use heap"
Simplicity vs Optimality	"The simpler O(N log N) solution is bug-free and fast enough; QuickSelect saves a log factor but adds complexity"
Recursion vs Iteration	"Recursion is cleaner here. For very deep trees (N=10^5), stack overflow is a risk — iterative DFS with explicit stack is safer"

🎯 **COMMUNICATION FORMULA:** Clarify → Example → Brute Force → Optimal Insight → Get Agreement → Code → Dry Run → Complexity → Follow-up. *Never skip a step.*

MOCK 1: GOOGLE HARD — COURSE SCHEDULE II

Full 45-minute mock with evaluation rubric and score

PROBLEM

🔴 Hard 🔵 Google LC 210

There are numCourses courses (0 to numCourses-1). You are given prerequisites where prerequisites[i] = [a, b] means to take course a you must first take b. Return a valid course ordering. If impossible, return [].

CANDIDATE INTERVIEW EXECUTION

Candidate: "First, clarifications: Can there be duplicate edges? What is the max numCourses — 2000? Can courses have no prerequisites? What should I return if numCourses=0?"

Interviewer: "No duplicates, max 2000, return [] for 0 courses."

Candidate: "This is a topological sort problem on a DAG. I'll model each prerequisite [a,b] as a directed edge b→a (b before a). Then run Kahn's algorithm with in-degree counting. If all N courses are processed, return the order; if any course remains (cycle detected), return []."

Time: O(V+E), Space: O(V+E). Shall I code this?"

Interviewer: "Yes, go ahead."

💡 **AHA:** The trick is modeling prerequisites as directed edges correctly — b→a means b is prerequisite of a.

DRY RUN: numCourses=4, prerequisites=[[1,0],[2,0],[3,1],[3,2]]

Step	Queue	inDegree	order
Init	[0]	[0,1,1,2]	[]
Poll 0	[1,2]	[0,0,0,2]	[0]
Poll 1	[2]	[0,0,0,1]	[0,1]
Poll 2	[3]	[0,0,0,0]	[0,1,2]
Poll 3	[]	[0,0,0,0]	[0,1,2,3] ✅

COMPLETE JAVA SOLUTION

```
public int[] findOrder(int numCourses, int[][] prerequisites) {
    // GUARD: edge case
    if (numCourses == 0) return new int[]{};

    // BUILD GRAPH + compute in-degrees
    List<List<Integer>> g = new ArrayList<>();
    int[] inDegree = new int[numCourses];
    for (int i = 0; i < numCourses; i++) g.add(new ArrayList<>());

    for (int[] prereq : prerequisites) {
        int course = prereq[0], pre = prereq[1];
        g.get(pre).add(course);   // edge: pre → course
        inDegree[course]++;
    }

    // INIT QUEUE with zero in-degree courses
    Queue<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < numCourses; i++)
        if (inDegree[i] == 0) q.offer(i);

    // PROCESS: Kahn's BFS topological sort
    int[] order = new int[numCourses];
    int idx = 0;
    while (!q.isEmpty()) {
        int course = q.poll();
        order[idx++] = course;
        for (int next : g.get(course)) {
            if (--inDegree[next] == 0)
                q.offer(next);
        }
    }
    // If all courses processed → no cycle
    return idx == numCourses ? order : new int[]{};
}
// Time:  O(V + E) — V=numCourses, E=prerequisites.length
// Space: O(V + E) — adjacency list + in-degree array
```

FOLLOW-UP QUESTIONS

1. **"What if we need all possible orderings?"** — DFS backtracking, return all valid topological sorts
2. **"What if some courses are elective (no path from source)?"** — disconnected graph; still works since we init all zero-indegree nodes
3. **"If N = 10^6 courses, any issue?"** — Java recursion depth for DFS approach could overflow. Kahn's iterative BFS is safe.
4. **"How do you detect which specific cycle exists?"** — Color-based DFS: white/gray/black states; gray node re-visit = back edge = cycle

EVALUATION RUBRIC + SCORE

Dimension	Score (1-4)	Feedback
Problem Solving	4	Identified graph + topo sort in under 2 min
Code Quality	4	Clean, modular, proper naming, guard clause
Verification	4	Full step-by-step dry run with trace table
Communication	4	Narrated full 7-step framework
OVERALL	🌟 **STRONG HIRE (L5/L6)**	

🧪 MOCK 2: META MEDIUM + MOCK 3: AMAZON DESIGN

MOCK 2: META — MERGE INTERVALS (LC 56)

🟡 Medium 🟣 Meta

Given an array of intervals, merge all overlapping intervals.

Candidate: "Are intervals guaranteed non-negative? Can end < start exist? Is output order important?"
Interviewer: "Valid intervals, any order."
Candidate: "Sort by start time. Then single pass: if current interval's start ≤ last merged end, merge by taking max of ends. Otherwise, add as new interval. O(N log N) time, O(N) space for output."

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) → Integer.compare(a[0], b[0]));
    List<int[]> merged = new ArrayList<>();
    for (int[] inv : intervals) {
        if (merged.isEmpty() || merged.get(merged.size()-1)[1] < inv[0])
            merged.add(inv);          // no overlap: add as-is
        else
            merged.get(merged.size()-1)[1] =   // overlap: extend end
                Math.max(merged.get(merged.size()-1)[1], inv[1]);
    }
    return merged.toArray(new int[merged.size()][]);
}
// Time: O(N log N)  Space: O(N)
```

DRY RUN: [[1,3],[2,6],[8,10],[15,18]]

inv	last.end	action	merged
[1,3]	—	add	[[1,3]]
[2,6]	3	2≤3 → extend to 6	[[1,6]]
[8,10]	6	8>6 → add	[[1,6],[8,10]]
[15,18]	10	15>10 → add	[[1,6],[8,10],[15,18]]

💡 Meta style: speed matters. Aim to code this in under 10 minutes with zero bugs.

MOCK 3: AMAZON — LRU CACHE (LC 146)

🟡 Medium 🟠 Amazon

Design an LRU cache with O(1) get and put operations.

Candidate: "Key insight: O(1) get + put means I need a data structure that supports O(1) ordered access. HashMap + Doubly Linked List: HashMap for O(1) key lookup, DLL for O(1) order maintenance. Sentinel head/tail nodes avoid null checks."

```
class LRUCache {
    class Node { int key, val; Node prev, next;
        Node(int k, int v){ key=k; val=v; } }
    private final Map<Integer,Node> map = new HashMap<>();
    private final Node head = new Node(0,0), tail = new Node(0,0);
    private final int cap;
    public LRUCache(int capacity) {
        this.cap = capacity;
        head.next = tail; tail.prev = head;  // sentinel setup
    }
    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        Node node = map.get(key);
        remove(node); insertFront(node);   // move to MRU position
        return node.val;
    }
    public void put(int key, int value) {
        if (map.containsKey(key)) remove(map.get(key));
        if (map.size() == cap) {   // evict LRU (tail.prev)
            map.remove(tail.prev.key); remove(tail.prev); }
        Node node = new Node(key, value);
        insertFront(node); map.put(key, node);
    }
    private void remove(Node n) { n.prev.next=n.next; n.next.prev=n.prev; }
    private void insertFront(Node n) {
        n.next=head.next; n.next.prev=n; head.next=n; n.prev=head; }
}
// ALL ops: O(1) time  |  Space: O(capacity)
```

💡 Amazon LP integration: "This is a classic ownership problem — one oversight in the eviction logic (wrong pointer) causes production cache corruption."

🧪 MOCK 4: UBER HARD + MOCK 5: STAFF-LEVEL

MOCK 4: UBER — N-QUEENS (LC 51)

🔴 Hard 🚙 Uber

Place N queens on N×N board so none attacks another. Return all valid configurations.

Candidate: "Classic backtracking. Each row gets exactly one queen. For each row, try each column — skip if column, diagonal, or anti-diagonal already occupied. Use three HashSets for O(1) conflict checking. Choose → place → recurse → undo."

```
public List<List<String>> solveNQueens(int n) {
    List<List<String>> res = new ArrayList<>();
    int[] queens = new int[n]; // queens[row] = col
    Arrays.fill(queens, -1);
    Set<Integer> cols = new HashSet<>(),
        diag = new HashSet<>(),    // row - col
        antiDiag = new HashSet<>(); // row + col
    backtrack(0, n, queens, cols, diag, antiDiag, res);
    return res;
}
void backtrack(int row, int n, int[] queens,
    Set<Integer> cols, Set<Integer> diag, Set<Integer> antiDiag,
    List<List<String>> res) {
    if (row == n) { res.add(buildBoard(queens, n)); return; }
    for (int col = 0; col < n; col++) {
        if (cols.contains(col) || diag.contains(row-col)
            || antiDiag.contains(row+col)) continue;
        // CHOOSE
        queens[row] = col;
        cols.add(col); diag.add(row-col); antiDiag.add(row+col);
        // EXPLORE
        backtrack(row+1, n, queens, cols, diag, antiDiag, res);
        // UNDO (state restoration)
        queens[row] = -1;
        cols.remove(col); diag.remove(row-col); antiDiag.remove(row+col);
    }
}
// Time: O(N!) worst  Space: O(N) recursion depth
```

FOLLOW-UPS:

- "Count unique solutions only?" → just increment counter, not build board
- "Solve for N=100?" → not feasible with backtracking; use optimization (branch-and-bound, symmetry)
- "How is this used at Uber?" → dispatch routing = constraint satisfaction at scale

MOCK 5: STAFF LEVEL — FIND MEDIAN FROM DATA STREAM (LC 295)

🔴 Hard 👑 Staff Level

Design a MedianFinder that supports addNum(int num) and findMedian() in O(log N) and O(1) respectively.

Candidate: "Two heaps: max-heap for lower half, min-heap for upper half. Invariant: maxHeap.size() == minHeap.size() or maxHeap.size() == minHeap.size()+1. Median = maxHeap.peek() if odd, or average of both tops if even. Staff-level extension: if data is mostly skewed to one side, discuss memory-efficient approaches using reservoir sampling."

```
class MedianFinder {
    // maxHeap holds lower half (negated for max behavior)
    private final PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Col
lections.reverseOrder());
    // minHeap holds upper half
    private final PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    public void addNum(int num) {
        maxHeap.offer(num);              // 1. always add to maxHeap first
        minHeap.offer(maxHeap.poll());   // 2. balance: push max of lower
to upper
        if (maxHeap.size() < minHeap.size())     // 3. maintain size invar
iant
            maxHeap.offer(minHeap.poll());
    }
    public double findMedian() {
        if (maxHeap.size() > minHeap.size())
            return maxHeap.peek();       // odd total: lower half has one
more
        return (maxHeap.peek() + minHeap.peek()) / 2.0; // even: average o
f midpoints
    }
}
// addNum: O(log N)  findMedian: O(1)  Space: O(N)
```

STAFF-LEVEL EXTENSIONS

1. **99% data within range [0,100]?** → Use integer frequency count array, O(1) add, O(1) median
2. **Remove elements?** → Lazy deletion with HashSet of "removed" keys, recalculate on access
3. **Sliding window median?** → Two TreeMaps with frequency counts (LC 480)

🏢 CH 27: COMPANY GUIDES — GOOGLE · META · AMAZON · MICROSOFT

🔵 GOOGLE — Interview Guide

Google Interview Profile

Rounds	5–6 rounds: 4-5 coding + 1 system design (Sr+)
Difficulty	Hard. Expect unseen variant problems, not LeetCode exact copies
Top Patterns	Graph BFS/DFS, Trees, DP (intervals, 2D), Topo Sort, Segment Tree
Style	Mathematical correctness over speed; expect proofs for greedy choices
Communication	Think out loud constantly; they evaluate process as much as solution
Common Mistakes	Solving easy version without handling edge cases; not proving correctness
Preparation	NeetCode Hard problems; graph variants; BST edge cases; practice verbal explanation
Time Management	40min coding + 5min follow-up; never run over without checking time

💡 Google cares deeply about correctness and mathematical rigor. If you choose greedy, you should be able to explain the exchange argument.

🟣 META — Interview Guide

Meta Interview Profile

Rounds	2 coding rounds (45 min each) — often 2 problems per round!
Difficulty	Medium-Hard but known problems; speed is crucial
Top Patterns	Arrays, Strings, Two Pointers, BFS/DFS, Trees, HashMap, Sliding Window
Style	Velocity. They expect clean, bug-free code quickly (aim 15 min per problem)
Communication	Brief clarification, then code fast. Less conversation than Google.
Common Mistakes	Too much talking; running out of time on second problem
Preparation	Blind 75 + Meta-tagged LeetCode; practice writing clean code in <15 min
Behavioral	Meta Leadership Principles: Move fast, Be bold, Focus on impact

💡 Meta = speed + accuracy. Practice coding common patterns in under 12 minutes, bug-free.

🟠 AMAZON — Interview Guide

Amazon Interview Profile

Rounds	4–5 rounds: coding + LP behavioral in every single round
Difficulty	Medium. Breadth over depth — practical DS problems
Top Patterns	Heap/PQ, HashMap, BFS/DFS, Sliding Window, Design (LRU, LFU)
Style	LP integration every round; always have STAR stories ready
Communication	Frame technical decisions using Leadership Principles language
Common Mistakes	Ignoring LP questions; not connecting code to real-world impact
Preparation	Map all 16 LPs to your STAR stories; practice out loud
LP Focus	Bias for Action, Customer Obsession, Dive Deep, Ownership, Frugality

💡 Amazon = Coding + Storytelling. Prepare 3-5 rich STAR stories that map to multiple LPs.

🟢 MICROSOFT — Interview Guide

Microsoft Interview Profile

Rounds	4–5 rounds: coding + behavioral + hiring manager final
Difficulty	Easy-Medium. Strong emphasis on code clarity and testing
Top Patterns	Linked List, Trees (BST), Stack, Arrays, String, Basic DP
Style	Code quality + verbal explanation + "How would you test this?"
Communication	Collaborative; interviewer may guide you more than at Google
Common Mistakes	Not discussing testing strategy; ignoring null edge cases
Preparation	LeetCode Easy-Medium; practice explaining code after writing
Special	"How would you test this function?" — always prepare an answer

💡 Microsoft rewards thoroughness. Voluntarily discuss testing strategy and null handling.

🏢 COMPANY GUIDES — UBER · STRIPE · NETFLIX · DATADOG · AIRBNB

🚗 UBER

Rounds	4 rounds: coding, SD, behavioral, HM
Top Patterns	Graphs (Dijkstra!), Geospatial, Concurrency, Heap
Style	Practical, production-ready; expect large N problems
Special	Geospatial indexing, real-time matching algorithms
Prep Tip	Master Dijkstra, geohash concepts, surge pricing algorithms

💳 STRIPE

Rounds	4 rounds: coding in real IDE (not CoderPad)
Top Patterns	Practical parsing, String manipulation, System design
Style	Real IDE + real tests; expected to write unit tests!
Special	JSON parsing, rate limiting, idempotency key design
Prep Tip	Practice writing clean, testable Java in your IDE, not online editors

📺 NETFLIX

Rounds	3-4 rounds; strong system design focus at senior
Top Patterns	Data pipeline, Streaming algorithms, LRU/Cache design
Style	"Freedom and Responsibility" culture; ownership is key
Special	Chaos engineering mindset; resilience + failure handling
Prep Tip	Read Netflix Tech Blog; understand their chaos engineering

📊 DATADOG

Rounds	4 rounds: coding + metrics system design
Top Patterns	Sliding Window, Time Series, Heap, HashMap, Rate limiting
Style	Observability-focused; expect questions about metrics at scale
Special	Moving average, percentile calculation (P99), anomaly detection
Prep Tip	Understand time-series data structures, circular buffers, rolling windows

💡 Datadog loves questions about "how would you compute P99 latency efficiently from a stream?"

🏠 AIRBNB

Rounds	4 rounds: coding + infra design + cross-functional interview
Top Patterns	Intervals, Calendar scheduling, Booking availability, Graph traversal
Style	Practical domain-specific questions (booking, calendar, pricing)
Special	Merge intervals with availability calendars; conflict detection
Prep Tip	Master interval problems; practice booking system design

💡 Airbnb loves: "Given N bookings, find all available time slots between check-ins."

COMPANY COMPARISON MATRIX

Company	Speed	Difficulty	Behavioral	System Design	Top Skill to Practice
🔵 Google	Medium	⭐⭐⭐⭐⭐	Moderate	Sr+ required	Graph algorithms, algorithmic correctness
🟣 Meta	🔥 Fast	⭐⭐⭐⭐	Light	Sr+ required	Speed + accuracy, no time waste
🟠 Amazon	Medium	⭐⭐⭐	🔥 Heavy (LP)	Sr+ required	STAR stories + practical DS
🟢 Microsoft	Relaxed	⭐⭐⭐	Moderate	Optional	Code quality + testing strategy
🚗 Uber	Medium	⭐⭐⭐⭐	Light	Sr+ required	Dijkstra, spatial indexing
💳 Stripe	Medium	⭐⭐⭐	Moderate	Sr required	Clean code + unit testing in IDE

📜 CH 28: REVISION HANDBOOK + STUDY PLANS

Pattern matrix • Blind 75 map • NeetCode 150 map • Last month/week/day plans

ULTIMATE COMPLEXITY REVISION MATRIX — 20 CORE PATTERNS

Pattern	Time	Space	DS Used	When to Use	Top FAANG Problems
Two Pointers	O(N)	O(1)	Two int pointers	Sorted array, pair/triple sum	LC 15, 11, 42, 125
Sliding Window	O(N)	O(1)/O(K)	int[128] freq, l/r	Contiguous subarray/string	LC 3, 76, 424, 239
Binary Search	O(log N)	O(1)	lo, hi, mid	Sorted space or BS on answer	LC 33, 875, 410, 4
Prefix Sum	O(N)	O(N)	int[] prefix + HashMap	Subarray sum queries	LC 560, 525, 1248
Monotonic Stack	O(N)	O(N)	Deque<Integer>	Next greater/smaller element	LC 739, 84, 85, 42
Top-K Heap	O(N log K)	O(K)	PriorityQueue	K largest/smallest/frequent	LC 347, 295, 215, 973
Tree DFS	O(N)	O(H)	Call stack	Path, depth, subtree problems	LC 104, 236, 124, 543
Tree BFS	O(N)	O(W)	Queue<TreeNode>	Level-order, row traversal	LC 102, 199, 103
Graph DFS	O(V+E)	O(V)	boolean[] visited	Flood fill, components	LC 200, 130, 695, 417
Graph BFS	O(V+E)	O(V)	Queue + visited	Shortest path unweighted	LC 994, 1162, 127
Topo Sort	O(V+E)	O(V)	int[] indegree + Queue	DAG ordering, dependencies	LC 207, 210, 269
Dijkstra	O(E log V)	O(V+E)	PriorityQueue<int[]>	Shortest path weighted	LC 743, 1514, 787
Union-Find	O(α(N))	O(N)	int[] parent, rank	Grouping, cycle detect	LC 547, 684, 1319
1D DP	O(N)	O(N)→O(1)	int[] dp	Min cost, count ways, LIS	LC 70, 198, 300, 322
2D DP	O(NM)	O(NM)→O(M)	int[][] dp	Grid, string match, knapsack	LC 62, 1143, 72, 416
Backtracking	$O(2^N)$/O(N!)	O(N)	List path + recursion	All combos, permutations	LC 78, 39, 46, 51
Greedy + Sort	O(N log N)	O(1)/O(N)	Arrays.sort	Optimal local → global	LC 55, 134, 621, 406
Intervals	O(N log N)	O(N)	Sort by start + scan	Merge, insert, overlap	LC 56, 57, 435, 253
Trie	O(L)	O(N·L)	TrieNode[26]	Prefix, word search, autocomplete	LC 208, 211, 212, 421
Bit Manipulation	O(1)/O(N)	O(1)	int bitmask	XOR tricks, subsets, masking	LC 136, 191, 338, 78

LAST MONTH PLAN (4 WEEKS)

Week	Focus	Problems/Day
Week 1	Arrays, Strings, HashMap, Two Pointers, Sliding Window	3 Medium
Week 2	Trees, BST, Heap, Binary Search, Linked List	3 Medium
Week 3	Graphs, DP (1D/2D), Backtracking, Greedy, Intervals	2 Medium + 1 Hard
Week 4	Mock interviews, revision, weak areas, system design	Full mocks

LAST WEEK PLAN (7 DAYS)

Day	Focus
Day 1	Arrays, HashMap, Two Pointers top 10
Day 2	Trees: DFS + BFS + BST top 10
Day 3	Graphs: BFS/DFS/Topo/Dijkstra top 10
Day 4	DP: 1D, 2D, Knapsack top 10
Day 5	Heap, Binary Search, Backtracking top 10
Day 6	Full mock interview (timed, 45 min)
Day 7	Review mistakes, cheat sheets only — rest!

LAST DAY PLAN (24 HRS)

✓Review pattern cheat sheet — 30 min
✓Review complexity table above
✓Practice 3 favourite templates from memory
✓Review one mock interview out loud
✓Prepare 3 LP stories for Amazon/behavioral
✓Sleep 8 hours — seriously!
✓Light exercise in the morning
❌Do NOT solve new hard problems
❌Do NOT stay up late "just in case"

💎 CH 29: ULTIMATE INTERVIEW PLAYBOOK + OFFER NEGOTIATION

Before • During • After • If stuck • Offer strategy

✅ BEFORE INTERVIEW CHECKLIST

Night Before:

✓Review pattern cheat sheet (20 min max)
✓Prepare IDE / CoderPad snippet (import java.util.*;)
✓Review your STAR stories for LP questions
✓Sleep 8 hours — peak cognitive performance requires sleep
❌Don't solve new hard problems night before

Morning Of:

✓Light exercise (30 min walk)
✓Good breakfast, no caffeine overload
✓Join 5 min early, test audio/video
✓Have water and scratch paper ready
✓Remind yourself: this is a conversation, not an exam

🖥️ DURING INTERVIEW PROTOCOL

0-3 min: Read problem. NEVER code yet.

↓

Ask clarifying questions about constraints

↓

Work through a small concrete example

↓

State brute force + why it's suboptimal

↓

Name optimal approach + get agreement

↓

Code: Guard → Helper stubs → Logic → Return

↓

Dry run your code on the example

↓

State complexity (time + space) + edge cases

↓

Volunteer follow-up: "A harder version would be..."

🆘 IF COMPLETELY STUCK

Say out loud: "Let me think step by step about what I know..."

Strategy 1: Solve a simpler version — "What if N=1?"
Strategy 2: Draw a concrete example — visualize the state machine
Strategy 3: Try brute force out loud — it may reveal the optimization
Strategy 4: "I'm thinking of X or Y — does either feel right to you?"
Strategy 5: "What if I sort first / use a HashMap / use a queue?"

💡 **Interviewers can HELP you if you verbalize where you're stuck!** Silent struggling = unsalvageable.

If you take a hint:

Say: "That's a great point — let me think about how that changes the approach. If I use [hint], then I can avoid [bottleneck] because..."
Taking a hint gracefully = demonstrates intellectual humility = good signal!

💰 OFFER NEGOTIATION — 4 GOLDEN LAWS

Law 1: Never Give First Number
"I'd love to understand the full compensation structure first — what's the range for this level?"

Law 2: Leverage Competing Offers
"I have an offer from [Company] for $X TC. I'm more excited about [your company] — can you match or improve?"

Law 3: Negotiate TC Not Salary
Focus on RSU cliff, vesting schedule, sign-on bonus — base salary cap is harder to move than equity/sign-on.

Law 4: Anchor with Enthusiasm
"I'm very excited about this role. If we can reach $X TC total, I'll sign immediately — I'm not shopping around."

💡 **Script:** "I'm very excited about the opportunity. The offer is $Y — I was expecting around $X based on my research and the competing offer. Is there any flexibility in the RSU grant?"

✅ FINAL INTERVIEW MASTER CHECKLIST

Before coding (must do all 5):

✓Clarify input type, constraints, N range
✓Work through a concrete example
✓Name brute force + explain why it's slow
✓Pitch optimal approach + get agreement
✓State time/space complexity before coding

While coding (must do all 5):

✓Guard clause at top (null, empty check)
✓Descriptive variable names throughout
✓Think out loud while writing every loop
✓Extract helpers for complex sub-problems
✓Use Integer.compare() in all comparators

After coding (must do all 5):

✓Dry run with your own concrete example
✓Test edge cases: empty, single, duplicates
✓State final time and space complexity
✓Volunteer follow-up extension unprompted
✓Ask a thoughtful question about the team

🧠 THE FAANG LEARNING FRAMEWORK — OUR UNIQUE TEACHING SYSTEM

13 phases • Mentor style principles • Signature workflow • Template-first learning

THE 13-PHASE FAANG LEARNING FRAMEWORK (Applied to EVERY Topic)

Phase	Name	Goal	Key Activity
Phase 0	🎯 Mindset	WHY does this exist?	Why do companies ask this? What does it solve? Interview frequency?
Phase 1	💡 Intuition First	Build the feeling	Story → Real-world analogy → Draw it → Build from scratch → AHA moment
Phase 2	📖 Core Understanding	Lock in definitions	Formal definitions → Properties → Rules → Edge cases → Common confusions
Phase 3	🎯 Pattern Recognition	Brain learns to spot it	Recognition keywords → When NOT to use → Similar pattern comparison
Phase 4	🔨 Build Algorithm Together	Derive, don't memorize	Brute Force → Identify bottleneck → Optimize step by step → Why it works
Phase 5	⚙️ Template Building	Create reusable code	Generic template → Java template → Interview skeleton → Memory tricks
Phase 6	▶️ Dry Run	See variables move	Small example → Medium example → Visual trace → State change table
Phase 7	💻 Coding Together	One line at a time	Explain every variable → Every loop → Every condition → Complete Java code
Phase 8	📊 Complexity Thinking	Analyze rigorously	Time O() → Space O() → Why? → Can it be improved? → Trade-offs
Phase 9	🐛 Debugging Mindset	Find and fix bugs	Common bugs → Edge cases → Dry-run mistakes → Debug checklist
Phase 10	🔁 Reverse Thinking	Handle follow-ups	Constraint changes → Solution changes → Alternative approaches → Why not X?
Phase 11	📈 Pattern Expansion	Generalize mastery	Easy → Medium → Hard → Similar pattern → Mixed pattern problems
Phase 12	✅ Mastery	Own it completely	Explain without notes → Draw from memory → Write template → Teach it back

🧠 OUR MENTOR STYLE PRINCIPLES

- 🚫 **Never memorize** — Derive the algorithm from first principles
- 🚫 **Never jump to code** — Build intuition first, always
- 📖 **Every algorithm has a story** — Dijkstra as GPS, BFS as ripple in water
- 💡 **Every topic has ONE Aha Moment** — the mental shift that unlocks understanding
- 🖍️ **Draw before coding** — visualize nodes, pointers, windows
- 🔨 **Build brute force first** — understand the problem before optimizing
- 📐 **Template-first** — Intuition → Algorithm → Template → Java code
- 🎓 **Master the pattern, not the problem** — solve 100 new problems from 15 templates

🔁 OUR SIGNATURE WORKFLOW

📖 Read Problem Carefully
↓
🔍 Find Clues (Keywords + N constraints)
↓
🎯 Recognize Pattern Family
↓
📋 Recall Generic Template
↓
🎀 Customize Template for This Problem
↓
▶️ Dry Run with Small Example
↓
☕ Write Clean Java Code
↓
📊 Analyze Time + Space Complexity
↓
🧪 Test Edge Cases
↓
🚀 Optimize + Follow-ups

🌳 TREE TEMPLATES (8 Total)

#	Template	Key Insight
T1	**DFS Preorder**	Root first → Copy/Serialize
T2	**DFS Inorder**	Left→Root→Right → BST sorted order
T3	**DFS Postorder**	Leaves-up → Height/Diameter
T4	**BFS Level Order**	Freeze level size, Queue
T5	**Bottom-Up Height**	1 + max(L, R)
T6	**BST Search/Validate**	min/max bounds recursion
T7	**LCA**	Return node if == p or q
T8	**Serialize/Deserialize**	Preorder + null markers

💡 Teaching flow: Draw → Parent/Child → Recursive Faith → Dry Run Stack → Template → Customize

🌐 GRAPH TEMPLATES (11 Total)

#	Template	Key DS
G1	**Build Undirected**	List<List<Integer>>
G2	**Build Weighted**	List<List<int[]>>
G3	**DFS (recursive)**	boolean[] visited
G4	**BFS shortest path**	Queue + dist[]
G5	**Multi-Source BFS**	All sources in Queue init
G6	**Grid DFS**	DIR4 + isValid()
G7	**Grid BFS**	Queue + visited[][]
G8	**Topological Sort**	indegree[] + Queue
G9	**Union-Find (DSU)**	parent[] + rank[]
G10	**Dijkstra**	PQ<int[]> + dist[]
G11	**Kruskal MST**	Sort edges + DSU

💡 AHA: Grids = Graphs! Trees ⊂ Graphs! Most graph problems hide as word problems.

🌲 TRIE TEMPLATES (9 Total)

#	Template	Key Insight
Tr1	**TrieNode Design**	TrieNode[26] + isEnd boolean
Tr2	**Insert**	Create nodes char by char
Tr3	**Search (exact)**	Traverse + check isEnd
Tr4	**StartsWith**	Traverse + return true (no isEnd check)
Tr5	**Prefix Count**	countChildren at prefix end
Tr6	**Word Search II**	Trie + DFS board backtrack
Tr7	**MapSum**	TrieNode with int val
Tr8	**Max XOR (Binary Trie)**	Bit-by-bit 31→0
Tr9	**Delete**	Postorder cleanup

💡 Teaching flow: String → Characters → One char = One node → Build path → End marker → Insert → Search

🎯 **THE PHILOSOPHY:** Master the 13-phase FAANG Learning Framework for every pattern. Derive the algorithm from first principles. Build the template. Write the code. The goal is to solve *any* unseen problem — not to memorize 300 solutions. **Master the pattern. Master any problem.**